单周期CPU设计

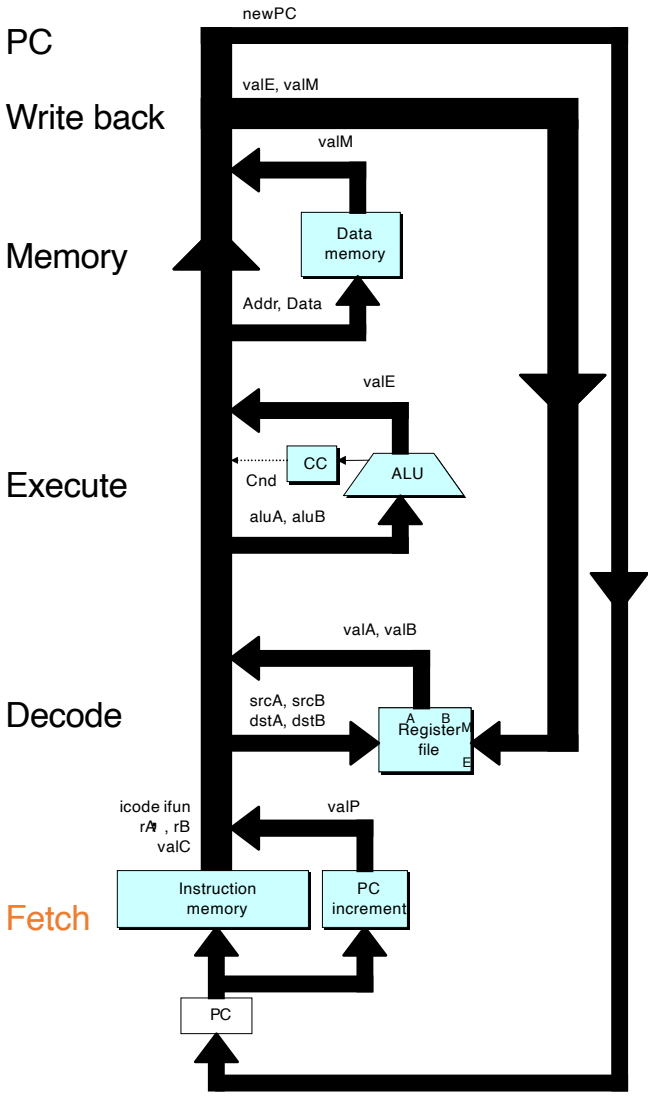
Y86-64 SEQ处理器

时钟周期=指令周期（一个时钟周期做完指令的所有阶段，时钟周期很长）

SEQ阶段

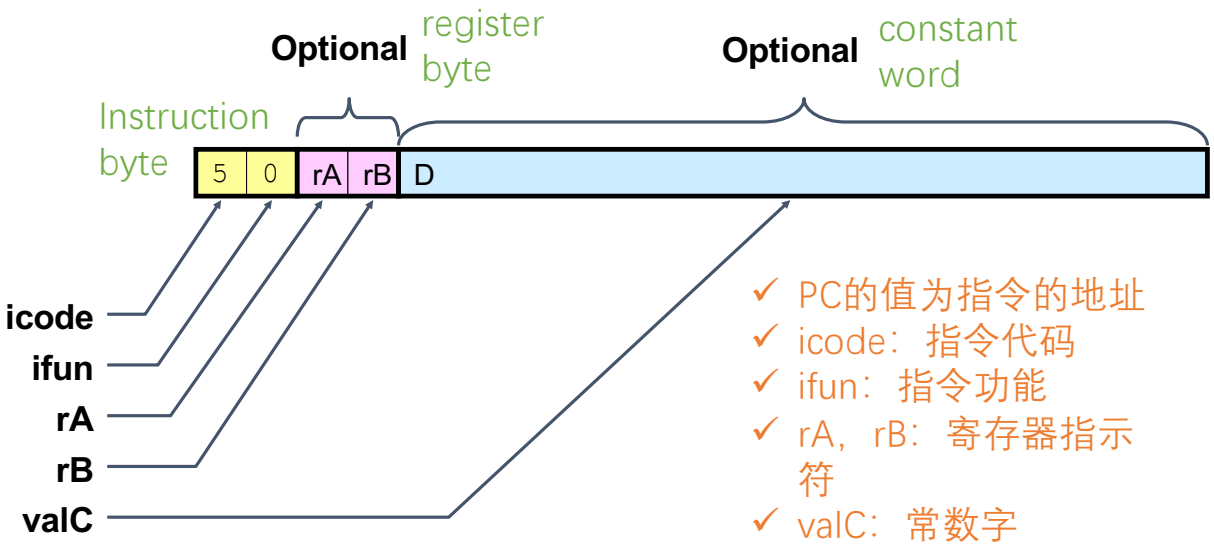
- 取指Fetch
  - Read instruction from instruction memory
- 译码Decode
  - Read program registers
- 执行Execute
  - Compute value or address
- 访存Memory
  - Read or write data
- 写回Write Back
  - Write program registers
- 更新PC
  - Update program counter

✓ PC的值为指令的地址
✓ icode：指令代码
✓ ifun：指令功能
✓ rA，rB：寄存器指示符
✓ valC：常数字
✓ valP=PC值+指令长度

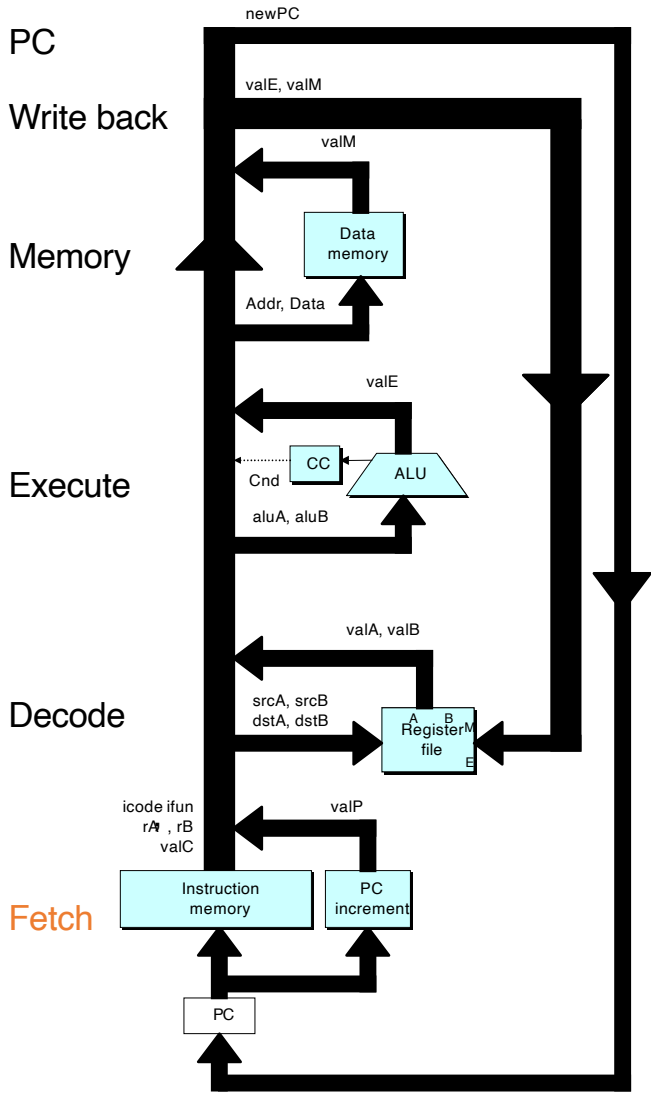


SEQ阶段

- 取指Fetch
  - Read instruction from instruction memory



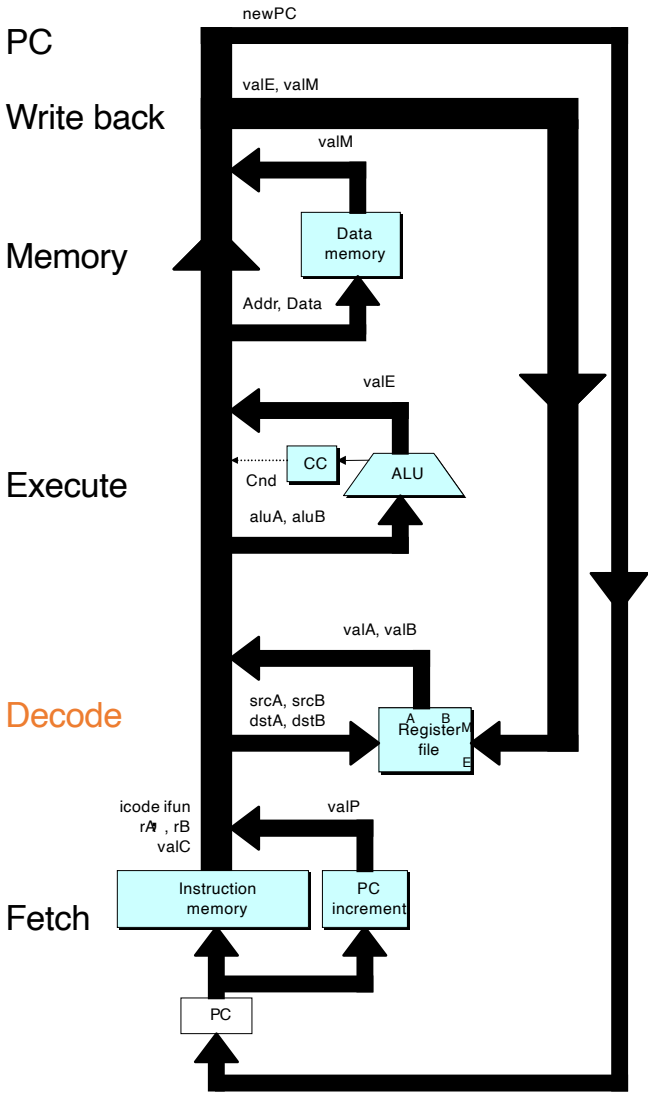
- ✓ PC的值为指令的地址
- ✓ icode：指令代码
- ✓ ifun：指令功能
- ✓ rA，rB：寄存器指示符
- ✓ valC：常数字
- ✓ valP=PC值+指令长度



SEQ阶段

- 取指Fetch
  - Read instruction from instruction memory
- 译码Decode
  - Read program registers
- 执行Execute
  - Compute value or address
- 访存Memory
  - Read or write data
- 写回Write Back
  - Write program registers
- 更新PC
  - Update program counter

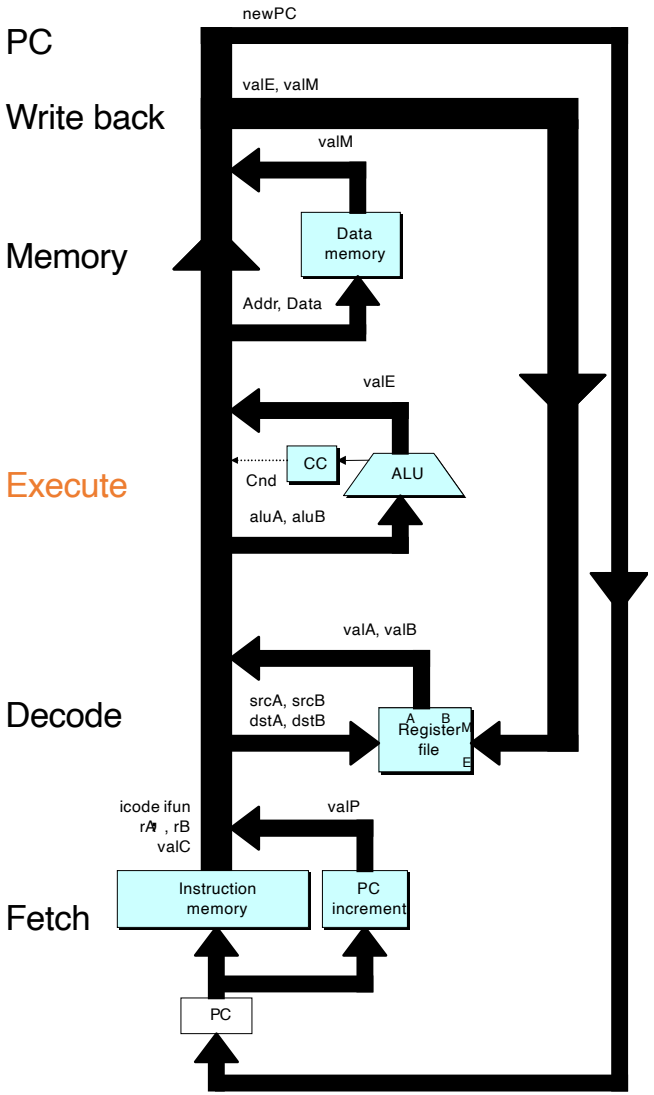
✓ 从寄存器文件的读端口读入rA、rB指明的操作数valA、valB，或读寄存器%rsp



SEQ阶段

- 取指Fetch
  - Read instruction from instruction memory
- 译码Decode
  - Read program registers
- 执行Execute
  - Compute value or address
- 访存Memory
  - Read or write data
- 写回Write Back
  - Write program registers
- 更新PC
  - Update program counter

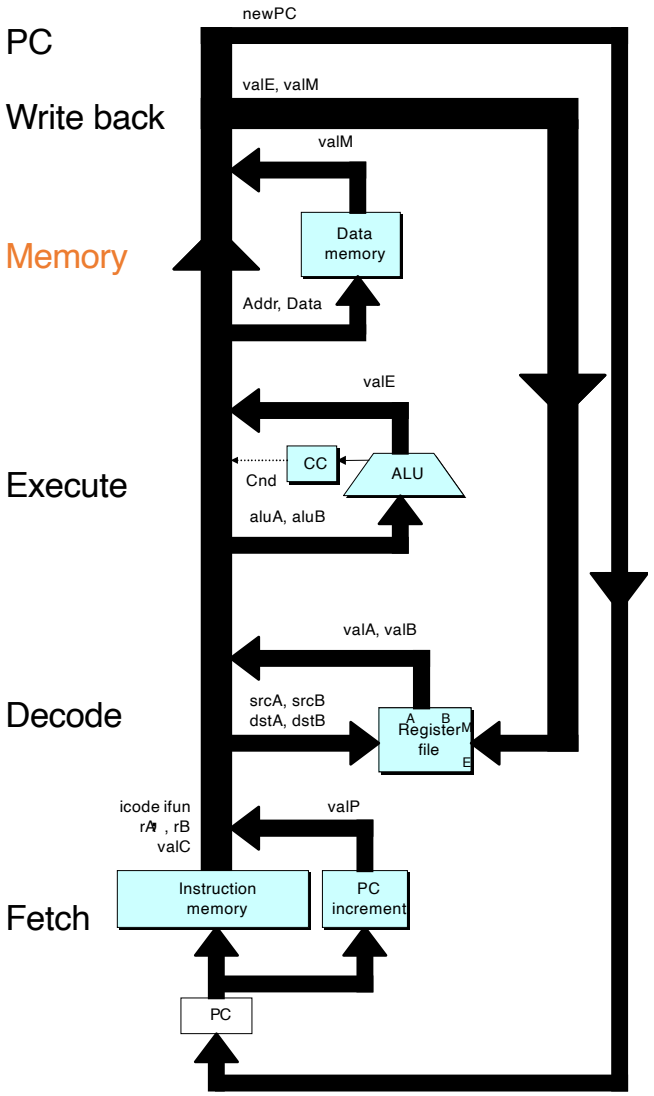
- ✓ ALU执行ifun操作
- ✓ 计算内存引用的有效地址 或 增减栈指针
- ✓ 得到结果valE
- ✓ 设置条件码
- ✓ （ifun条件码成立）更新目标寄存器 或 计算跳转的分支信号Cnd



SEQ阶段

- 取指Fetch
  - Read instruction from instruction memory
- 译码Decode
  - Read program registers
- 执行Execute
  - Compute value or address
- 访存Memory
  - Read or write data
- 写回Write Back
  - Write program registers
- 更新PC
  - Update program counter

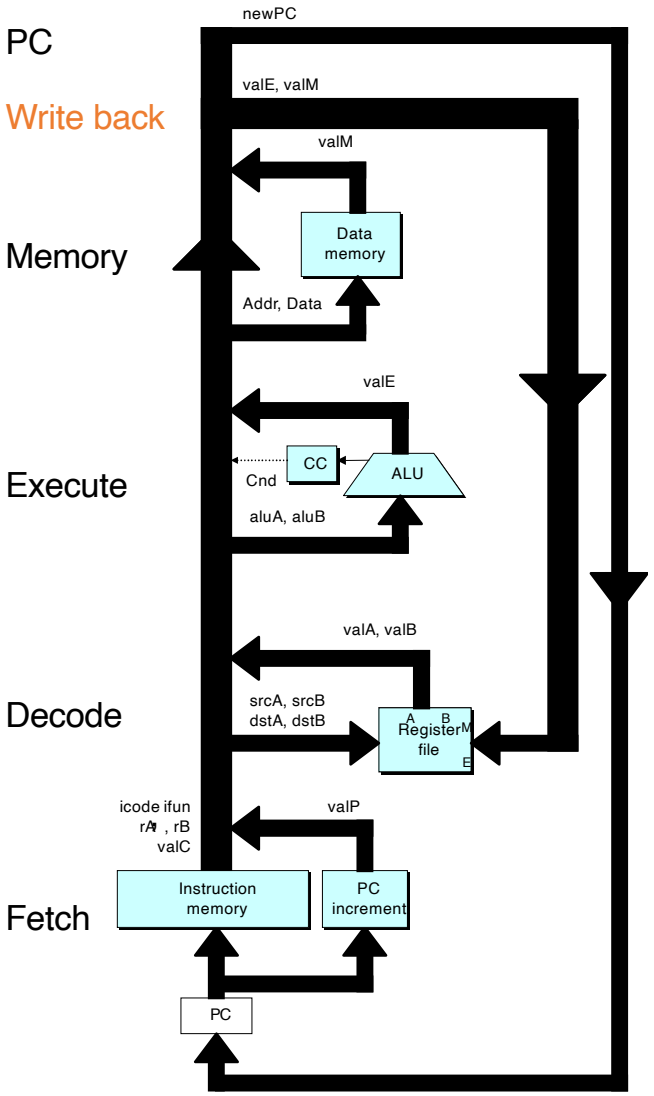
✓ 数据写入内存 或 内存读出数据valM



SEQ阶段

- 取指Fetch
  - Read instruction from instruction memory
- 译码Decode
  - Read program registers
- 执行Execute
  - Compute value or address
- 访存Memory
  - Read or write data
- 写回Write Back
  - Write program registers
- 更新PC
  - Update program counter

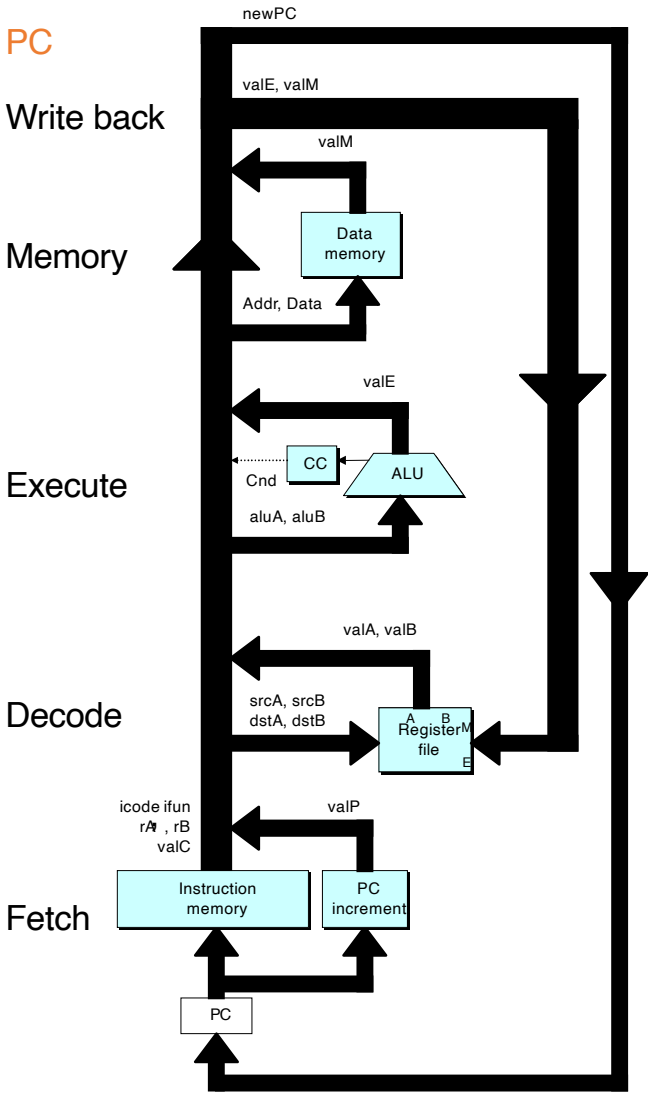
✓ valE、valM写回寄存器文件（两个写端口M、E）



SEQ阶段

- 取指Fetch
  - Read instruction from instruction memory
- 译码Decode
  - Read program registers
- 执行Execute
  - Compute value or address
- 访存Memory
  - Read or write data
- 写回Write Back
  - Write program registers
- 更新PC
  - Update program counter

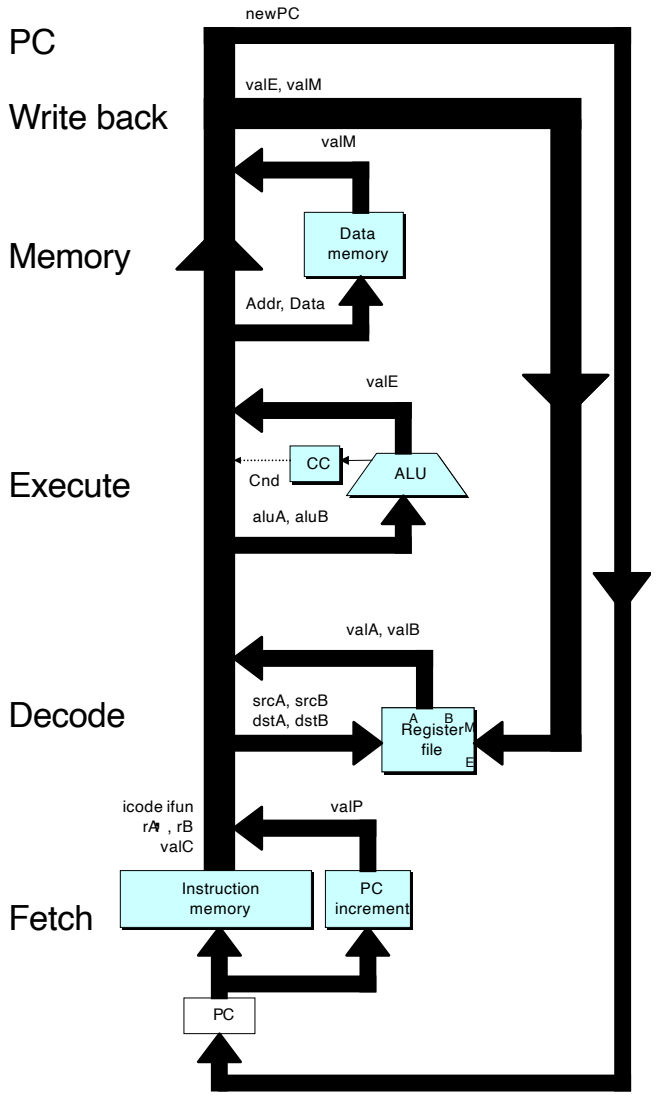
✓ valP：下一条指令地址
✓ valC：调用或跳转的目标地址
✓ valM：内存读取的返回地址



SEQ阶段

- 循环执行以上阶段

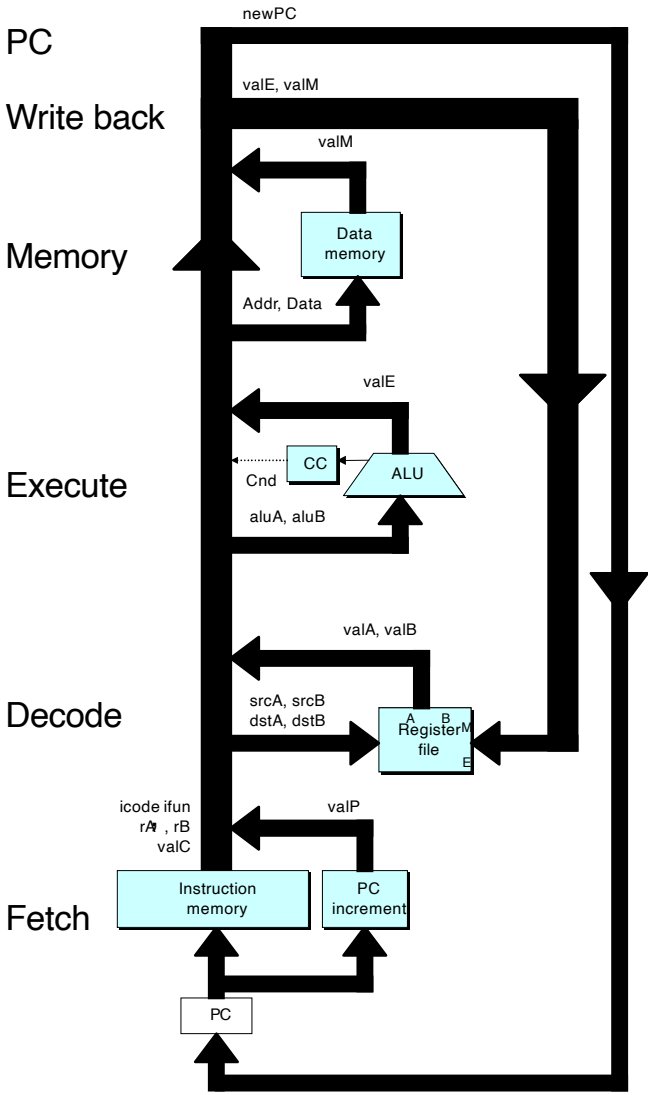
- 状态
  - 程序计数器寄存器（PC）
  - 条件码寄存器（CC）
  - 寄存器文件
  - 内存
    - 指令
    - 数据



SEQ阶段

- 硬件上复制逻辑块的成本远大于软件中重复代码
- 硬件中处理特殊情况的难度远大于软件来处理

- 硬件数量尽可能少
- 结构尽可能简单一致
- 不同指令共享尽可能多的硬件
  - 每个处理器只有一个ALU，指令类型不同，使用方式不同
- 不同指令需要的计算尽可能放入一个通用框架



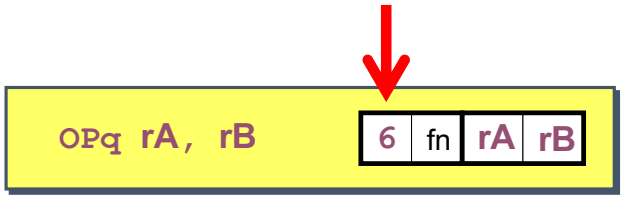
OPq（整数和逻辑运算）

- $M_n[x]$: 访问内存位置x处的n个字节



	OPq rA, rB	
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	Read instruction byte Read register byte Compute next PC
Decode	valA ← R[rA] valB ← R[rB]	Read operand A Read operand B
Execute	valE ← valB OP valA Set CC	Perform ALU operation Set condition code register
Memory		
Write back	R[rB] ← valE	Write back result
PC update	PC ← valP	Update PC

OPq（整数和逻辑运算）



- Mn[x]: 访问内存位置x处的n个字节

- 四个整数操作（addq, subq, andq, xorq）
  - icode相同，顺序处理步骤相同
  - ifun不同，只改变ALU计算

- 访存阶段什么也不做

	OPq rA, rB	
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	Read instruction byte Read register byte Compute next PC
Decode	valA ← R[rA] valB ← R[rB]	Read operand A Read operand B
Execute	valE ← valB OP valA Set CC	Perform ALU operation Set condition code register
Memory		
Write back	R[rB] ← valE	Write back result
PC update	PC ← valP	Update PC

rrmovq（RR传送）

	OPq rA, rB	rrmovq rA, rB
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2
Decode	valA ← R[rA] valB ← R[rB]	valA ← R[rA]
Execute	valE ← valB OP valA Set CC	valE ← 0 + valA
Memory		
Write back	R[rB] ← valE	R[rB] ← valE
PC update	PC ← valP	PC ← valP

- 不取第二个操作数
- ALU第二个输入设为0，实现输出等于第一个输入

irmovq（IR传送）

- ALU第一个输入为常数
- 长指令格式

	OPq rA, rB	rrmovq rA, rB	irmovq V, rB
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valC ← M_8[PC+2] valP ← PC+10
Decode	valA ← R[rA] valB ← R[rB]	valA ← R[rA]	
Execute	valE ← valB OP valA Set CC	valE ← 0 + valA	valE ← 0 + valC
Memory			
Write back	R[rB] ← valE	R[rB] ← valE	R[rB] ← valE
PC update	PC ← valP	PC ← valP	PC ← valP

rmmovq（RM传送）

- 访存
- 不写回



- ALU用作有效地址计算

	rmmovq rA, D(rB)	
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valC ← M_8[PC+2] valP ← PC+10	Read instruction byte Read register byte Read displacement D Compute next PC
Decode	valA ← R[rA] valB ← R[rB]	Read operand A Read operand B
Execute	valE ← valB + valC	Compute effective address
Memory	M_8[valE] ← valA	Write value to memory
Write back		
PC update	PC ← valP	Update PC

popq

- 先读栈内存，写到寄存器，再出栈
  - 读两次栈指针：后续流程和其他指令更一致
  - ALU给栈指针+8（向高地址回退）
  - 访存 地址是**没有加8**的原始值
  - 写回 用**加8**的栈指针更新栈指针寄存器



	popq rA	
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	Read instruction byte Read register byte Compute next PC
Decode	valA ← R[%rsp] valB ← R[%rsp]	Read stack pointer Read stack pointer
Execute	valE ← valB + 8	Increment stack pointer
Memory	valM ← M_8[valA]	Read from stack
Write back	R[%rsp] ← valE R[rA] ← valM	Update stack pointer Write back result
PC update	PC ← valP	Update PC

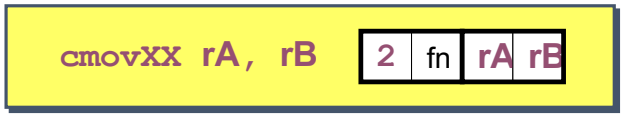
pushq



	popq rA	pushq rA
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2
Decode	valA ← R[%rsp] valB ← R[%rsp]	valA ← R[rA] valB ← R[%rsp]
Execute	valE ← valB + 8	valE ← valB + (-8)
Memory	valM ← M_8[valA]	M_8[valE] ← valA
Write back	R[%rsp] ← valE R[rA] ← valM	R[%rsp] ← valE
PC update	PC ← valP	PC ← valP

- 栈指针-8
- 访存写入
- 栈指针更新

cmovXX（条件传送）



	cmovXX rA, rB	
Fetch	icode:ifun ← M_1[PC] rA:rB ← M_1[PC+1] valP ← PC+2	Read instruction byte Read register byte Compute next PC
Decode	valA ← R[rA] valB ← 0	Read operand A
Execute	valE ← valB + valA If ! Cond(CC,ifun) rB ← 0xF	Pass valA through ALU (Disable register update)
Memory		
Write back	R[rB] ← valE	Write back result
PC update	PC ← valP	Update PC

- Cancel move by setting destination register to 0xF
  - If condition codes & move condition indicate no move

jXX



	jXX Dest	
Fetch	icode:ifun ← M_1[PC] valC ← M_8[PC+1] valP ← PC+9	Read instruction byte Read destination address Fall through address
Decode		
Execute	Cnd ← Cond(CC,ifun)	Take branch?
Memory		
Write back		
PC update	PC ← Cnd ? valC : valP	Update PC

- Compute both addresses
- Choose based on setting of condition codes and branch condition

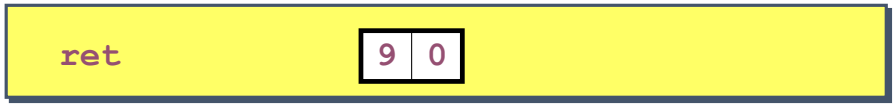
call



	call Dest	
Fetch	icode:ifun ← M_1[PC]	Read instruction byte
	valC ← M_8[PC+1]	Read destination address
	valP ← PC+9	Compute return point
Decode	valB ← R[%rsp]	Read stack pointer
Execute	valE ← valB + (-8)	Decrement stack pointer
Memory	M_8[valE] ← valP	Write return value on stack
Write back	R[%rsp] ← valE	Update stack pointer
PC update	PC ← valC	Set PC to destination

- Use ALU to decrement stack pointer
- Store incremented PC

ret



	ret	
Fetch	icode:ifun ← M_1[PC]	Read instruction byte
Decode	valA ← R[%rsp] valB ← R[%rsp]	Read operand stack pointer Read operand stack pointer
Execute	valE ← valB + 8	Increment stack pointer
Memory	valM ← M_8[valA]	Read return address
Write back	R[%rsp] ← valE	Update stack pointer
PC update	PC ← valM	Set PC to return address

- Use ALU to increment stack pointer
- Read return address from memory

计算步骤

- 所有指令遵循一个通用的模式
- 每一步计算的内容有所差别

		OPq rA, rB	
Fetch	icode,ifun	icode:ifun ← M_1[PC]	Read instruction byte
	rA,rB	rA:rB ← M_1[PC+1]	Read register byte
	valC		[Read constant word]
	valP	valP ← PC+2	Compute next PC
Decode	valA, srcA	valA ← R[rA]	Read operand A
	valB, srcB	valB ← R[rB]	Read operand B
Execute	valE	valE ← valB OP valA	Perform ALU operation
	Cond code	Set CC	Set/use cond. code reg
Memory	valM		[Memory read/write]
Write back	dstE	R[rB] ← valE	Write back ALU result
	dstM		[Write back memory result]
PC update	PC	PC ← valP	Update PC

计算步骤

- 所有指令遵循一个通用的模式
- 每一步计算的内容有所差别

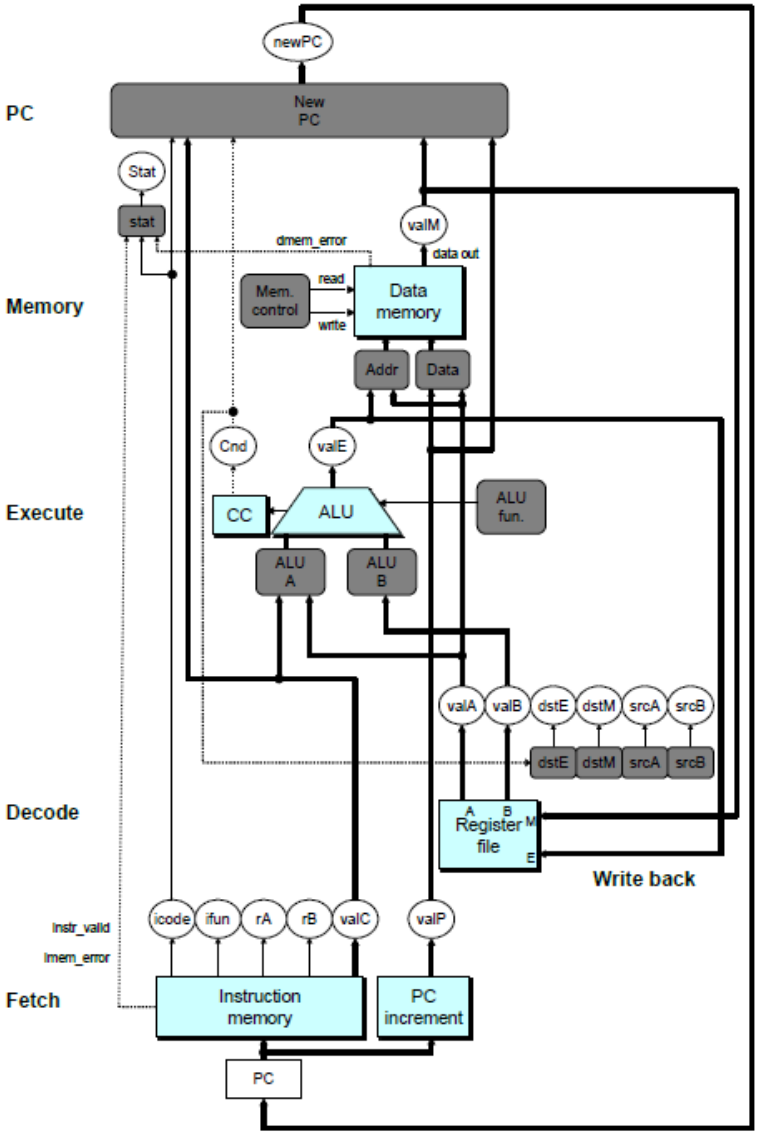
		`call` Dest	
Fetch	icode,ifun	icode:ifun ← M_1[PC]	Read instruction byte
	rA,rB		[Read register byte]
	valC	valC ← M_8[PC+1]	Read constant word
	valP	valP ← PC+9	Compute next PC
Decode	valA, srcA		[Read operand A]
	valB, srcB	valB ← R[`%rsp`]	Read operand B
Execute	valE	valE ← valB + –8	Perform ALU operation
	Cond code		[Set /use cond. code reg]
Memory	valM	M_8[valE] ← valP	Memory read/write
Write back	dstE	R[`%rsp`] ← valE	Write back ALU result
	dstM		[Write back memory result]
PC update	PC	PC ← valC	Update PC

计算的值

- Fetch
  - icode Instruction code
  - ifun Instruction function
  - rA Instr. Register A
  - rB Instr. Register B
  - valC Instruction constant
  - valP Incremented PC
- Decode
  - srcA Register ID A
  - srcB Register ID B
  - dstE Destination Register E
  - dstM Destination Register M
  - valA Register value A
  - valB Register value B
- Execute
  - valE ALU result
  - Cnd Branch/move flag
- Memory
  - valM Value from memory

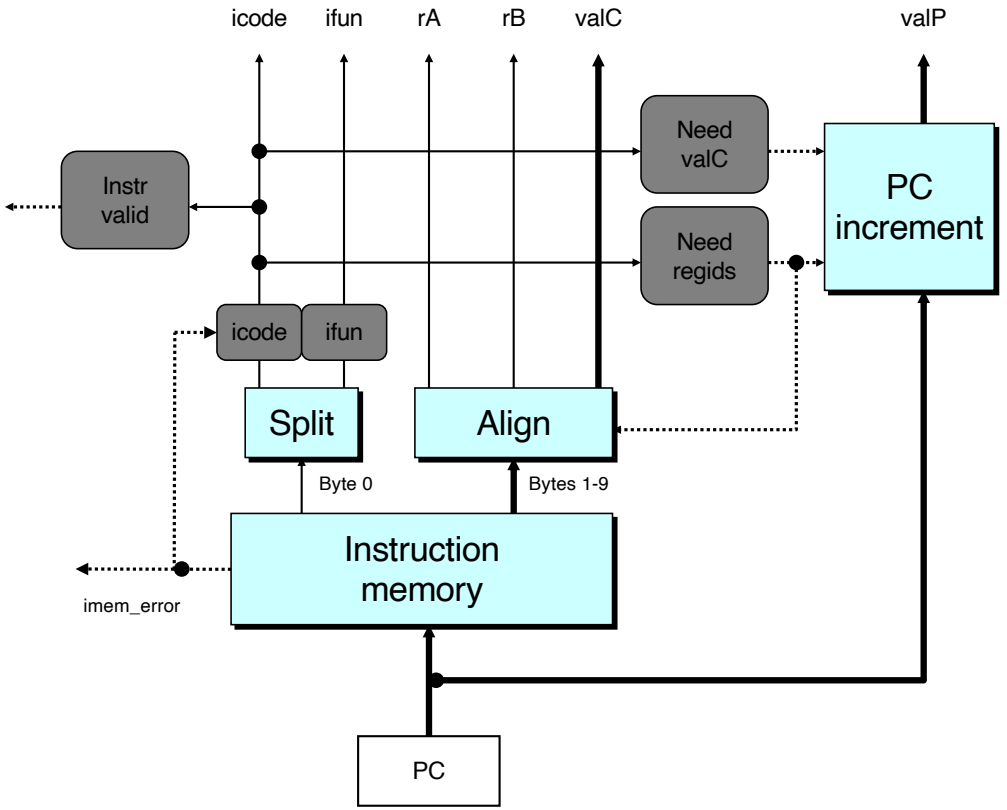
SEQ硬件

- 蓝色框
  - 预先设计好的硬件块
    - 内存、ALU
- 灰色框
  - 控制逻辑块
    - 使用HCL描述
- 白色圆
  - 线路标识
- 白色框（PC）
  - 时钟（时钟周期=指令周期，极慢）
- 粗实线（64根线）
  - 字数据传输
- 细实线（4或8根线）
  - 字节或更小数据传输
- 虚线
  - 单个位的控制值



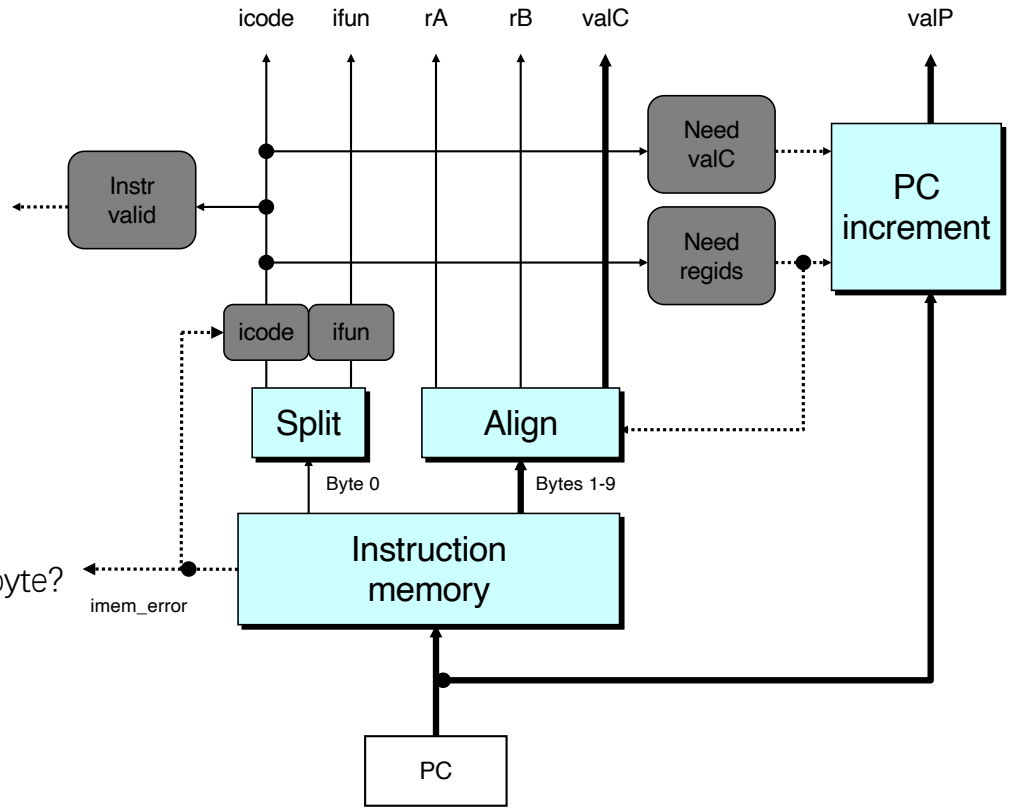
取指逻辑

- 预定义硬件块
  - PC
    - Register containing PC
  - Instruction memory
    - Read 10 bytes (PC to PC+9)
    - Signal invalid address (imem_error)
  - Split
    - Divide instruction byte into icode and ifun
  - Align
    - Get fields for rA, rB, and valC



取指逻辑

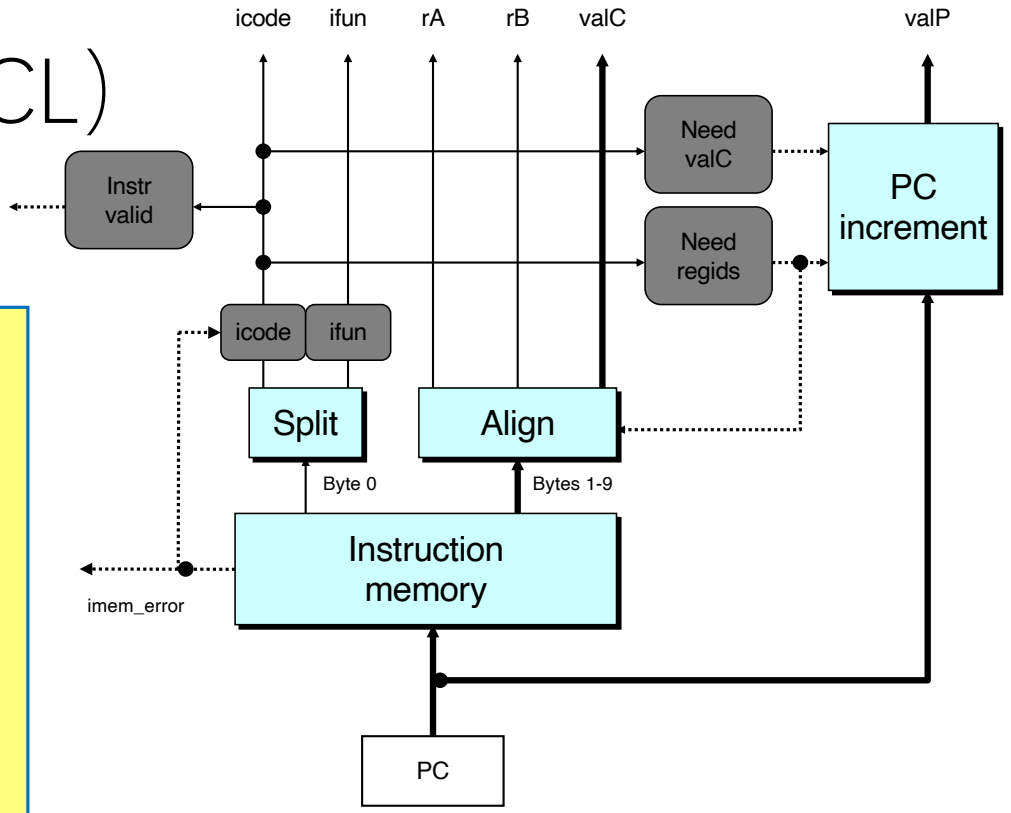
- 控制逻辑
  - Instr. Valid
    - Is this instruction valid?
  - icode, ifun
    - Generate **nop** if invalid address
  - Need regids
    - Does this instruction have a register byte?
  - Need valC
    - Does this instruction have a constant word?



取指控制逻辑（HCL）

```
# Determine instruction code
int icode = [
        imem_error: INOP;
        1: imem_icode;
];

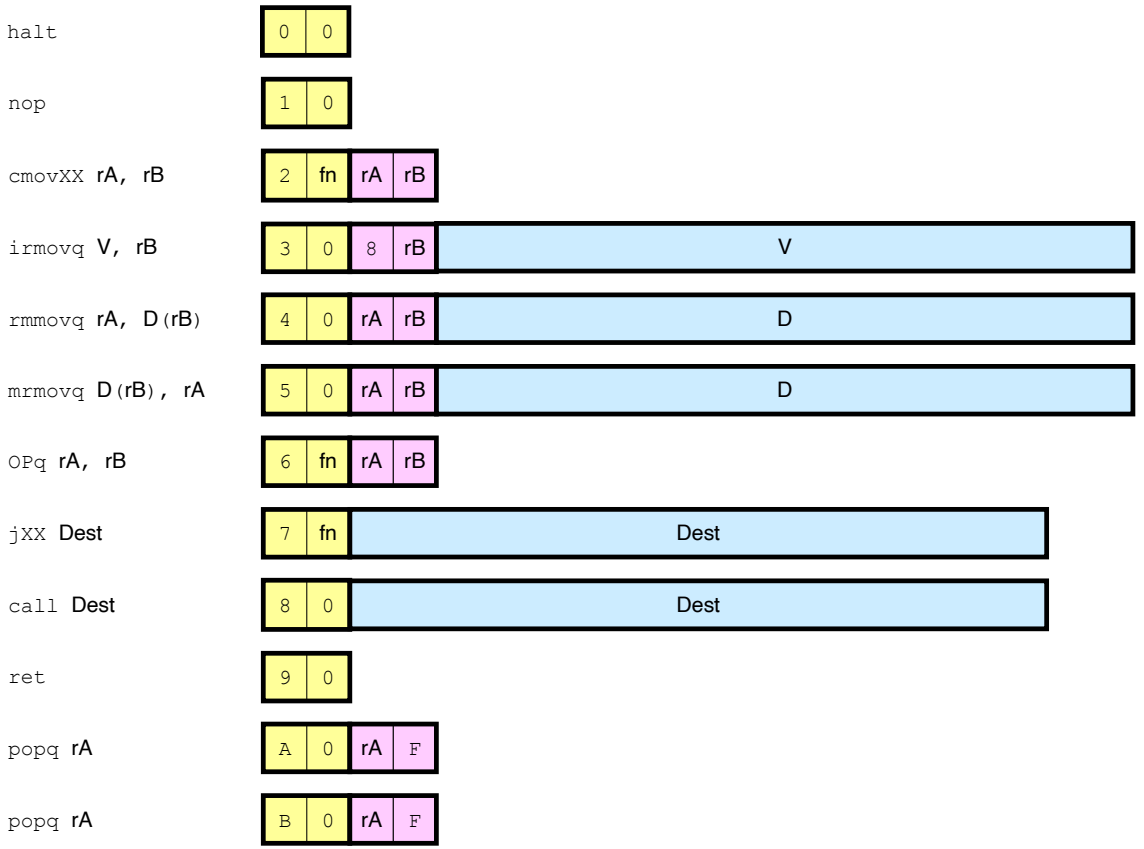
# Determine instruction function
int ifun = [
        imem_error: FNONE;
        1: imem_ifun;
];
```



取指控制逻辑（HCL）

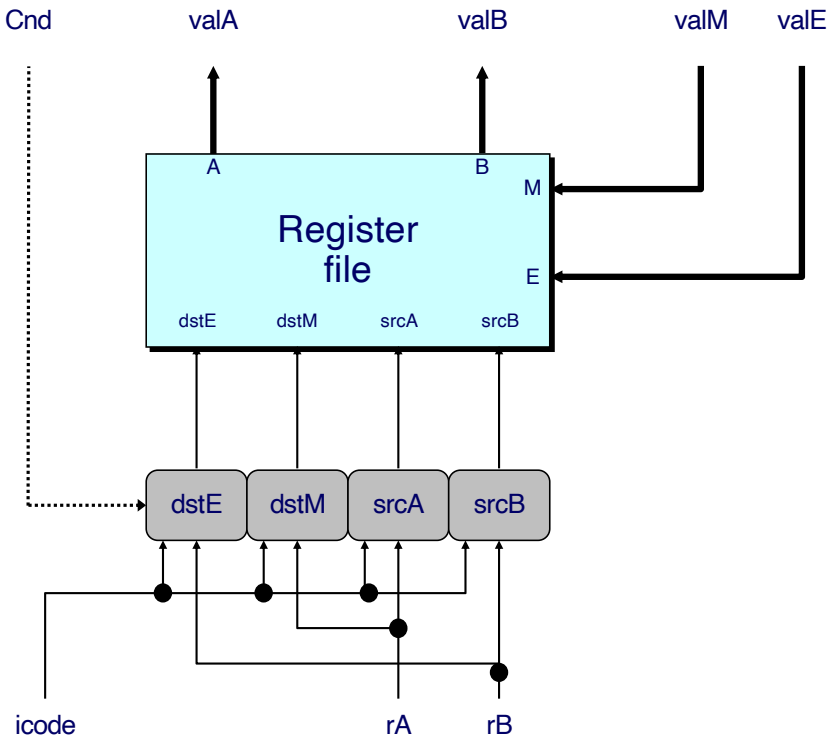
```
bool instr_valid = icode in
       { INOP, IHALT, IRRMOVQ,
IIRMOVQ, IRMMOVQ, IMRMOVQ, IOPQ,
IJXX, ICALL, IRET, IPUSHQ, IPOPQ };

bool need_regids =
       icode in { IRRMOVQ, IOPQ,
IPUSHQ, IPOPQ, IIRMOVQ, IRMMOVQ,
IMRMOVQ };
```



译码和写回逻辑

- Register File
  - Read ports A, B
  - Write ports E, M
  - Addresses are register IDs or 15 (0xF) (no access)
- Control Logic
  - srcA, srcB: read port addresses
  - dstE, dstM: write port addresses
- 控制信号
  - Cnd: Indicate whether or not to perform conditional move
    - Computed in Execute stage



译码和写回逻辑

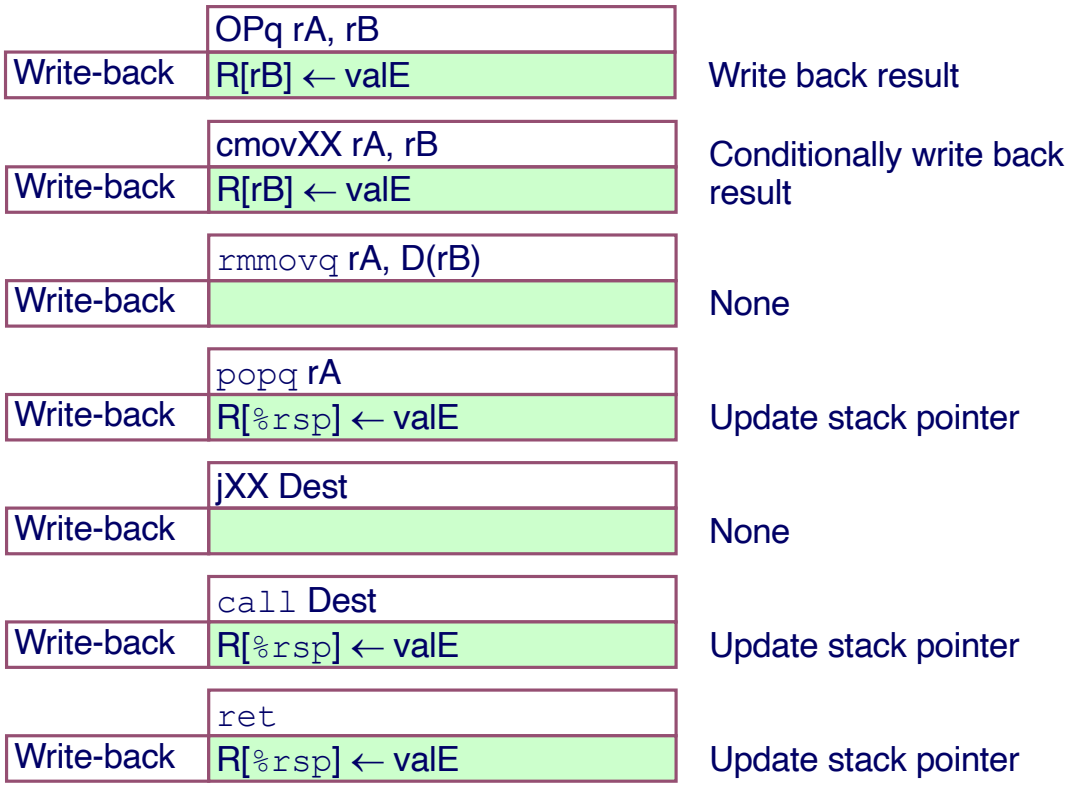
- srcA

	OPq rA, rB	
Decode	valA ← R[rA]	Read operand A
	cmovXX rA, rB	
Decode	valA ← R[rA]	Read operand A
	`rmmovq` rA, D(rB)	
Decode	valA ← R[rA]	Read operand A
	`popq` rA	
Decode	valA ← R[`%rsp`]	Read stack pointer
	jXX Dest	
Decode		No operand
	`call` Dest	
Decode		No operand
	`ret`	
Decode	valA ← R[`%rsp`]	Read stack pointer

```
int srcA = [
        icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ  } : rA;
        icode in { IPOPQ, IRET } : RRSP;
        1 : RNONE; # Don't need register
];
```

译码和写回逻辑

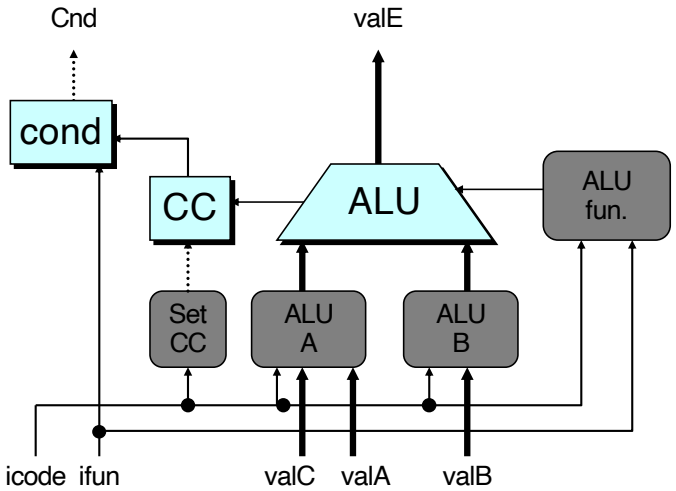
- dstE




```
int dstE = [
        icode in { IRRMOVQ } && Cnd : rB;
        icode in { IIRMOVQ, IOPQ } : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # Don't write any register
];
```

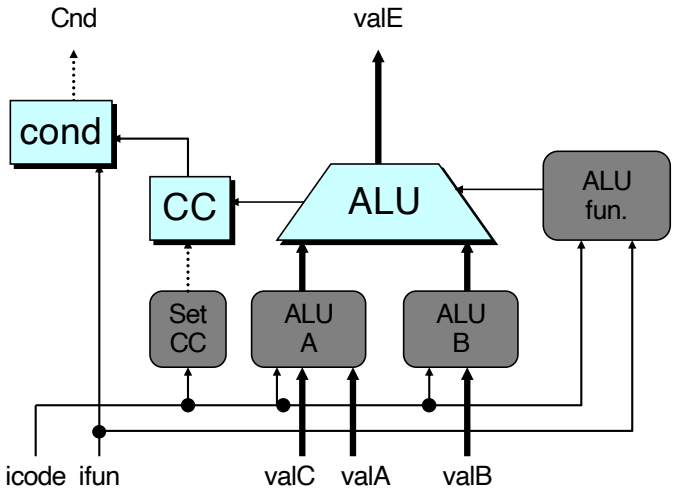
执行逻辑

- 硬件单元
  - ALU
    - Implements 4 required functions (add/sub/and/xor)
    - Generates condition code values
  - CC
    - Register with 3 condition code bits (ZF/SF/OF)
  - cond
    - Computes conditional jump/move flag



执行逻辑

- 控制逻辑
  - ALU A: Input A to ALU
  - ALU B: Input B to ALU
  - ALU fun: What function should ALU compute?
  - Set CC: Should condition code register be loaded?



执行逻辑

- ALUA

	OPq rA, rB	
Execute	valE ← valB OP valA	Perform ALU operation
	cmovXX rA, rB	
Execute	valE ← 0 + valA	Pass valA through ALU
	`rmmovq` rA, D(rB)	
Execute	valE ← valB + valC	Compute effective address
	`popq` rA	
Execute	valE ← valB + 8	Increment stack pointer
	jXX Dest	
Execute		No operation
	`call` Dest	
Execute	valE ← valB + –8	Decrement stack pointer
	`ret`	
Execute	valE ← valB + 8	Increment stack pointer

```
int aluA = [
      icode in { IRRMOVQ, IOPQ } : valA;
      icode in { IIRMOVQ, IRMMOVQ, IMRMOVQ } : valC;
      icode in { ICALL, IPUSHQ } : -8;
      icode in { IRET, IPOPQ } : 8;
      # Other instructions don't need ALU ];
```

执行逻辑

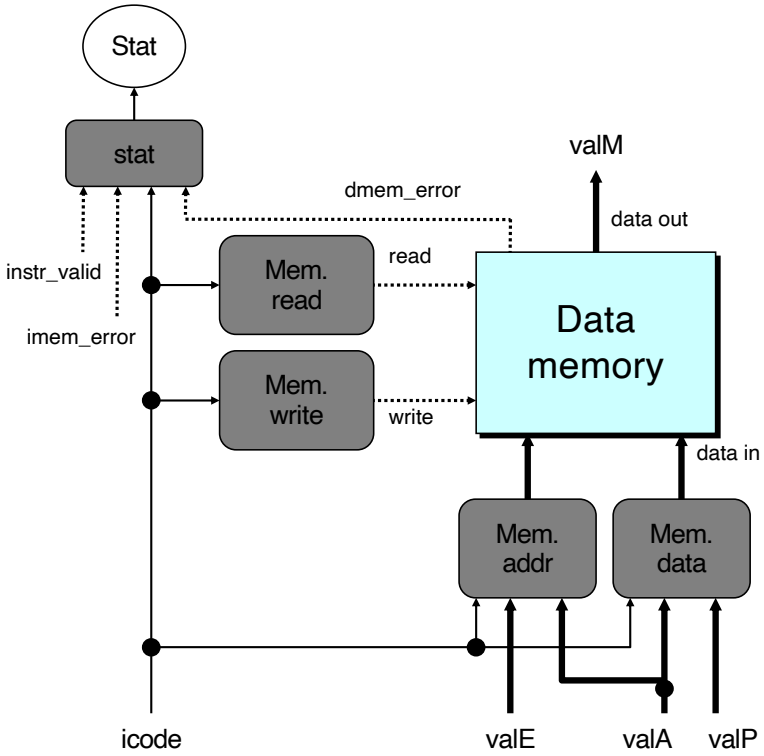
- alufun

```
int alufun = [
        icode == IOPQ : ifun;
        1 : ALUADD;
];
```

	OPl rA, rB	
Execute	valE ← valB OP valA	Perform ALU operation
	cmovXX rA, rB	
Execute	valE ← 0 + valA	Pass valA through ALU
	`rmmovl` rA, D(rB)	
Execute	valE ← valB + valC	Compute effective address
	`popq` rA	
Execute	valE ← valB + 8	Increment stack pointer
	jXX Dest	
Execute		No operation
	`call` Dest	
Execute	valE ← valB + –8	Decrement stack pointer
	`ret`	
Execute	valE ← valB + 8	Increment stack pointer

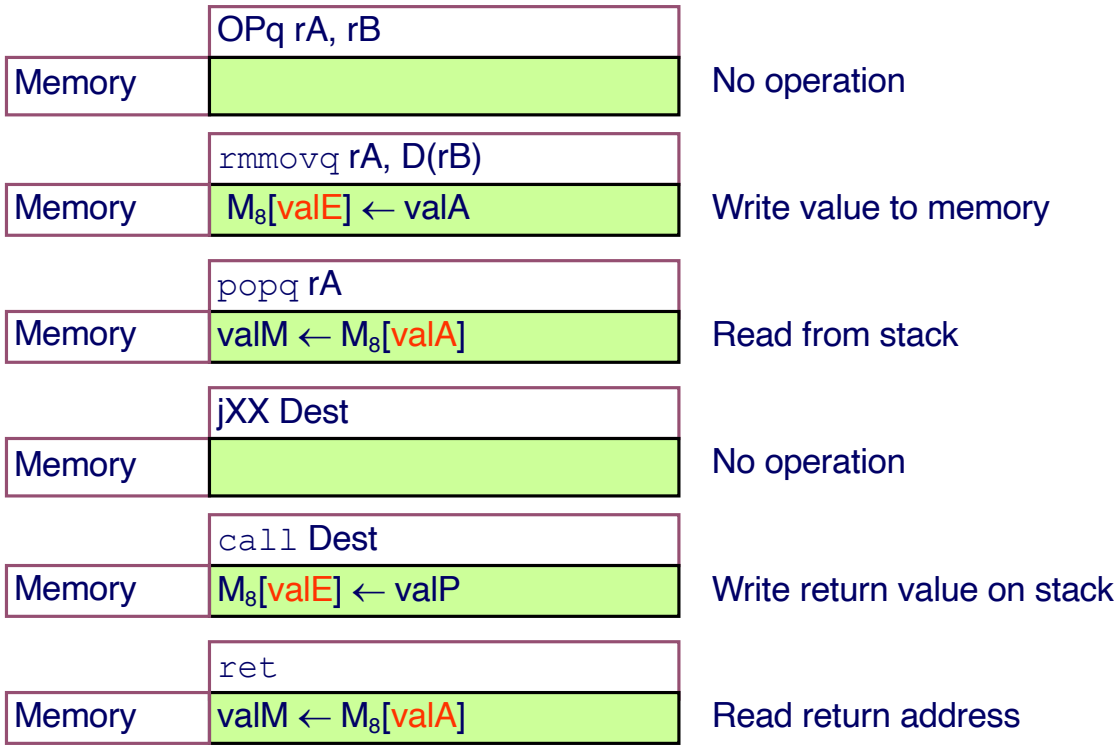
访存逻辑

- 内存
  - 按地址读或写内存字
- 控制逻辑
  - stat: What is instruction status?
  - Mem. read: should word be read?
  - Mem. write: should word be written?
  - Mem. addr.: Select address
  - Mem. data: Select data



访存逻辑

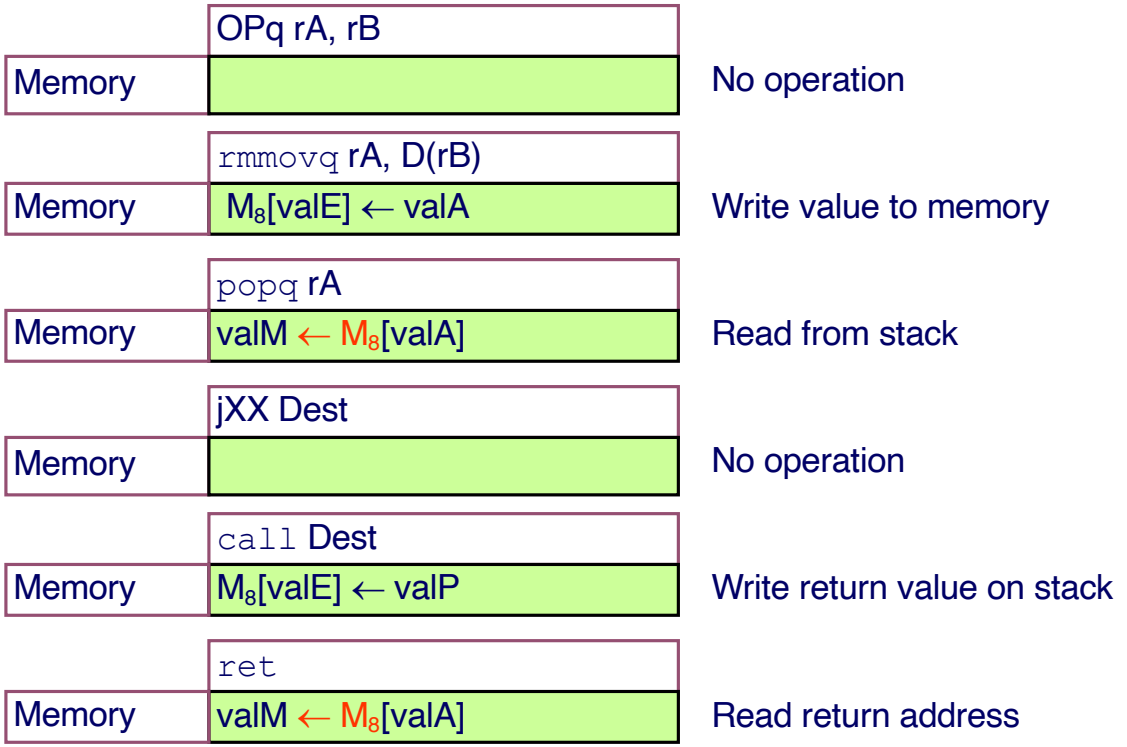
- Mem. addr




```
int mem_addr = [
        icode in { IRMMOVQ, IPUSHQ, ICALL, IMRMOVQ } : valE;
        icode in { IPOPQ, IRET } : valA;
        # Other instructions don't need address
];
```

访存逻辑

- Mem. read

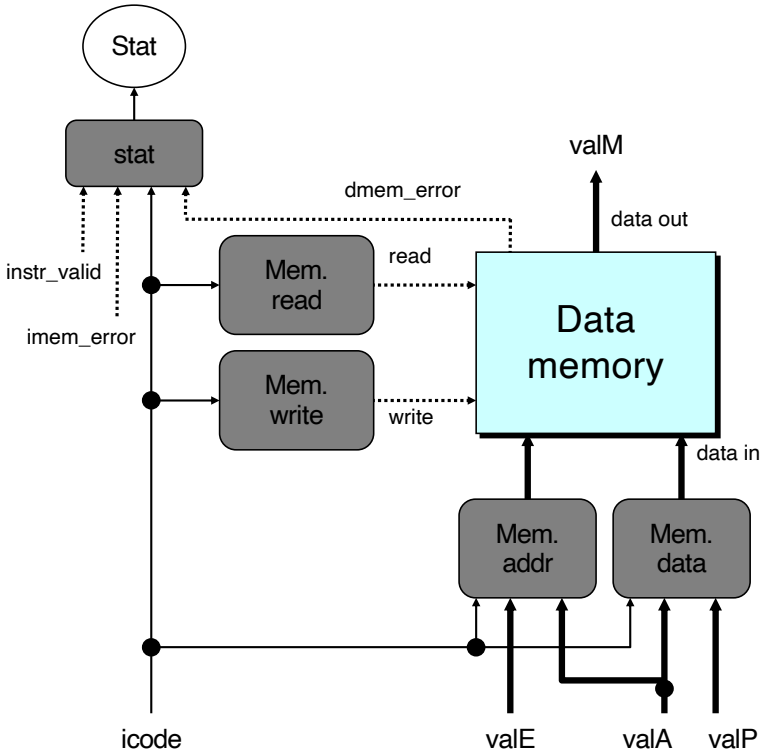



```
bool mem_read = icode in { IMRMOVQ, IPOPQ, IRET };
```

访存逻辑

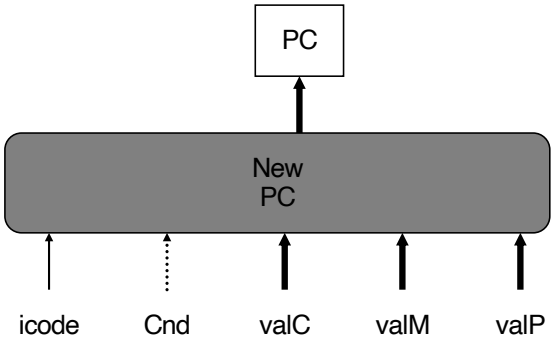
- stat
  - 1=SAOK：Normal operation
    - 继续执行
    - Otherwise：停止执行
  - 2=SHLT：Halt instruction encountered
  - 3=SADR：Bad address (either instruction or data) encountered
  - 4=SINS：Invalid instruction encountered

```
## Determine instruction status
int Stat = [
        imem_error || dmem_error : SADR;
        !instr_valid : SINS;
        icode == IHALT : SHLT;
        1 : SAOK;
];
```



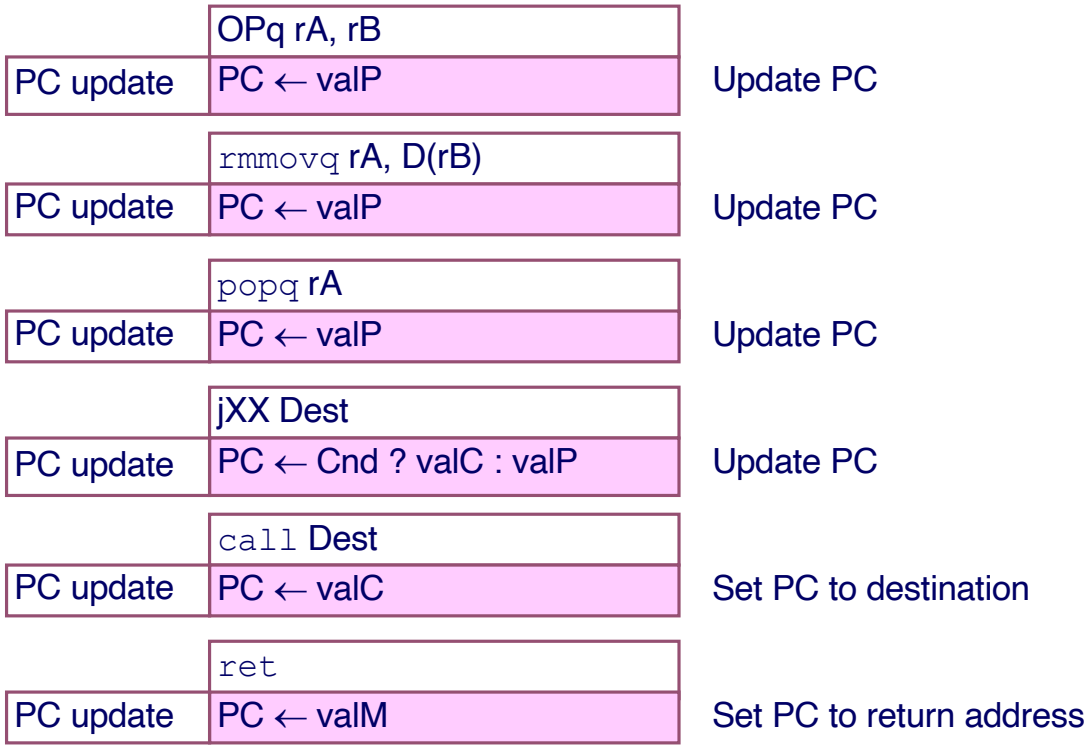
更新PC逻辑

- 根据指令代码和分支标志，从信号中选出下一个PC值



更新PC逻辑

- 根据指令代码和分支标志，从信号中选出下一个PC值

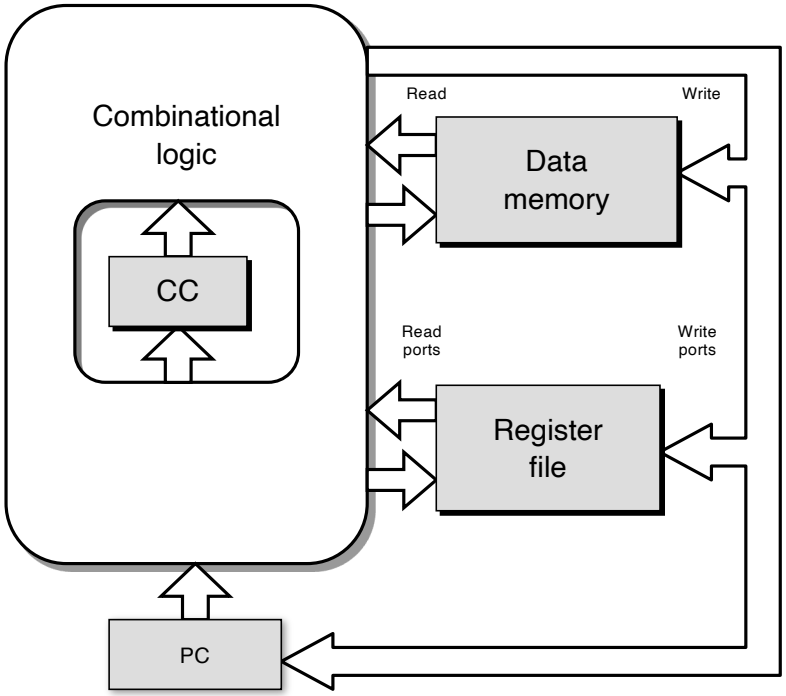



```
int new_pc = [
        icode == ICALL : valC;
        icode == IJXX && Cnd : valC;
        icode == IRET : valM;
        1 : valP;
];
```

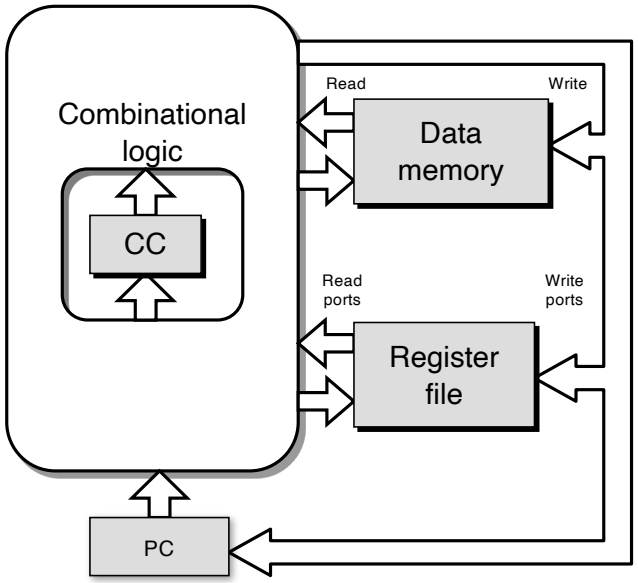
SEQ

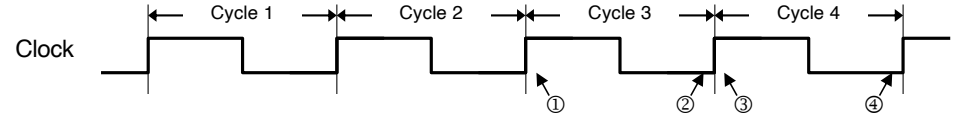
- 组合逻辑
- 程序状态
  - PC寄存器
  - 条件码寄存器
  - 数据内存
  - 寄存器文件

  时钟上升沿更新



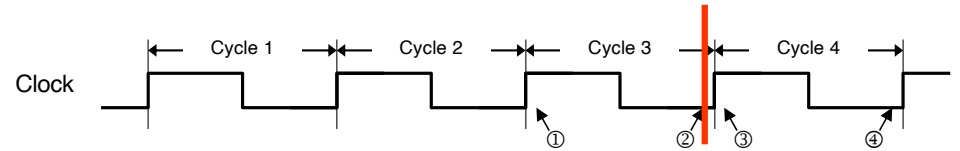
SEQ



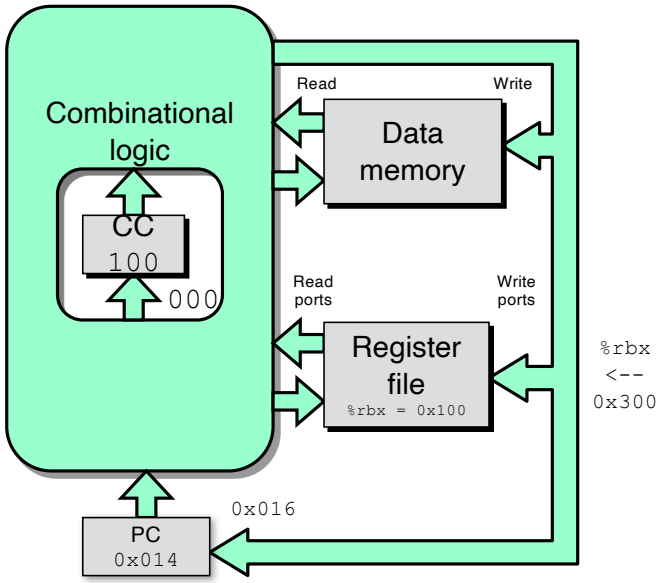


Cycle 1:	0x000:	irmovq $0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq $0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je dest	# Not taken
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300

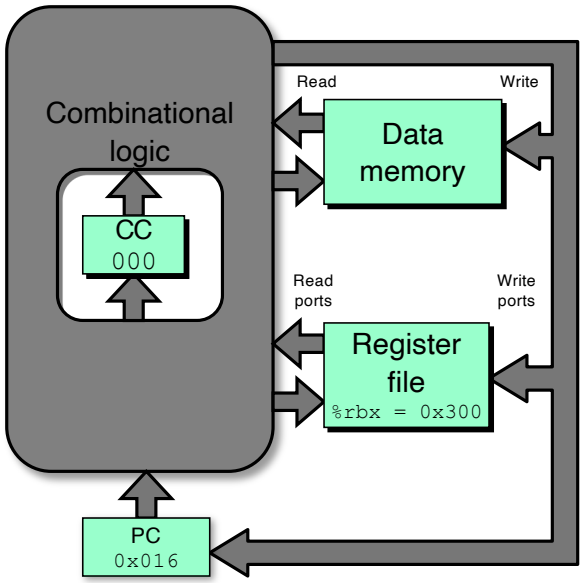
SEQ

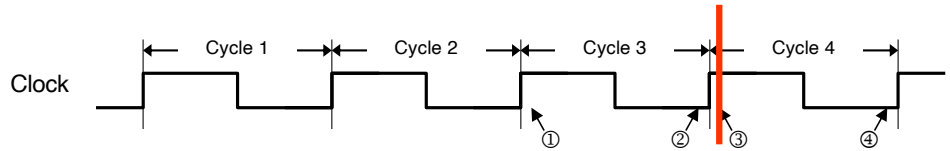


Cycle 1:	0x000:	irmovq $0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq $0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je dest	# Not taken
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300



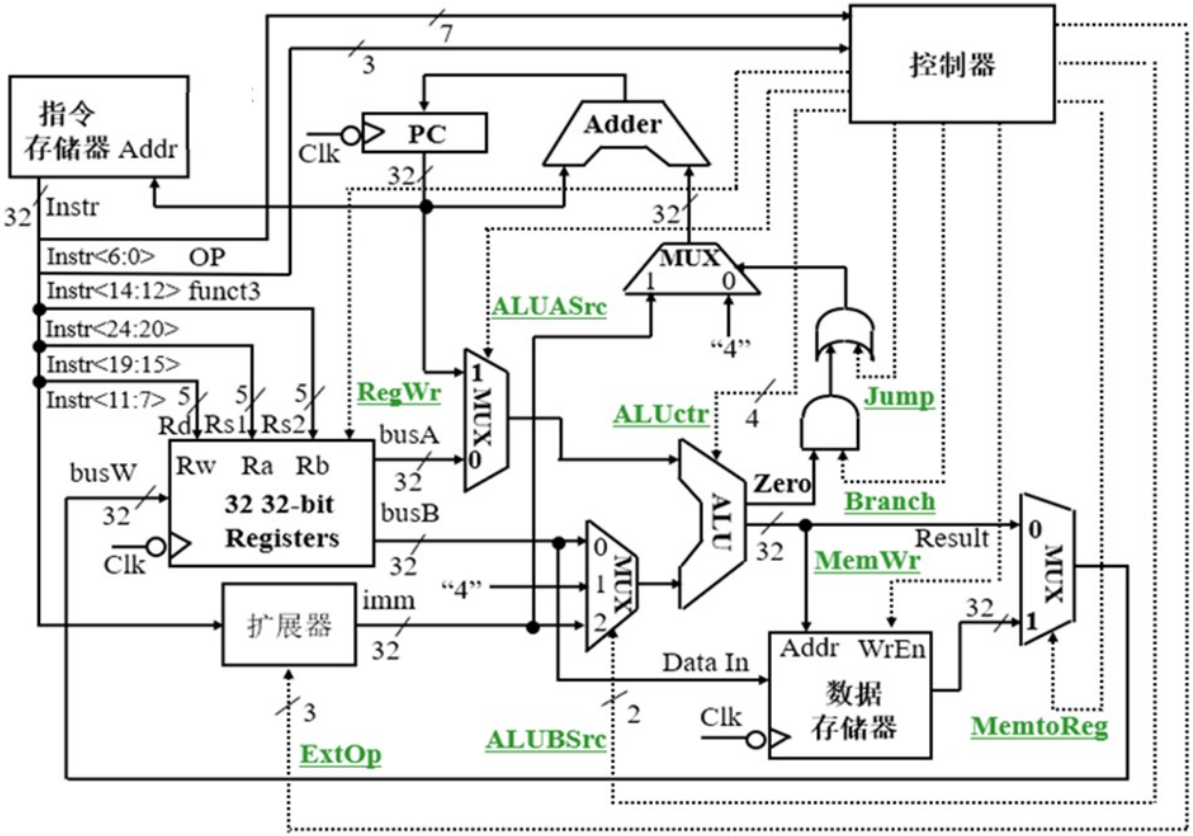
SEQ





Cycle 1:	0x000:	irmovq $0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq $0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je dest	# Not taken
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300

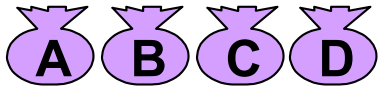
RISC-V单周期CPU设计



流水线CPU

生活中的流水线

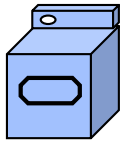
- Laundry Example
  - Ann, Brian, Cathy, Dave
    each have one load of clothes to wash, dry, and fold



  - Washer takes 30 minutes



  - Dryer takes 40 minutes

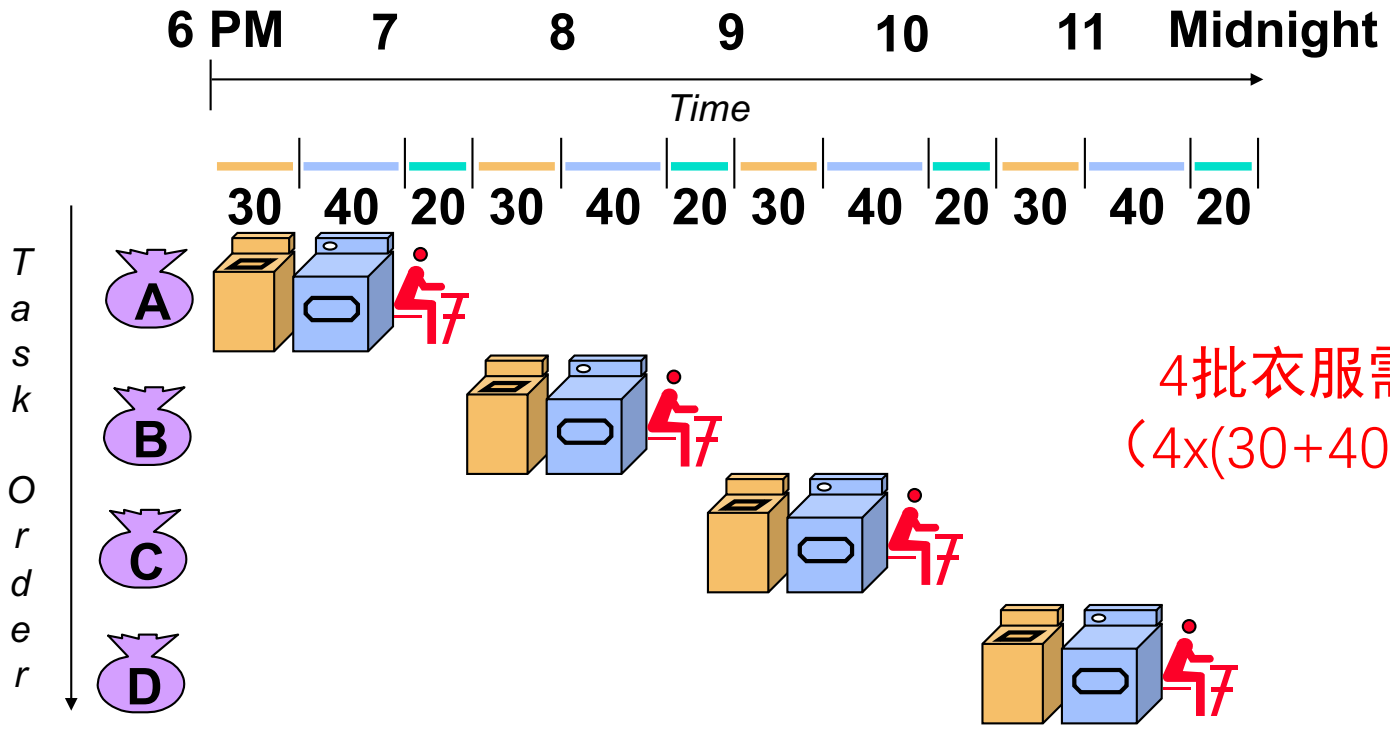


  - “Folder” takes 20 minutes



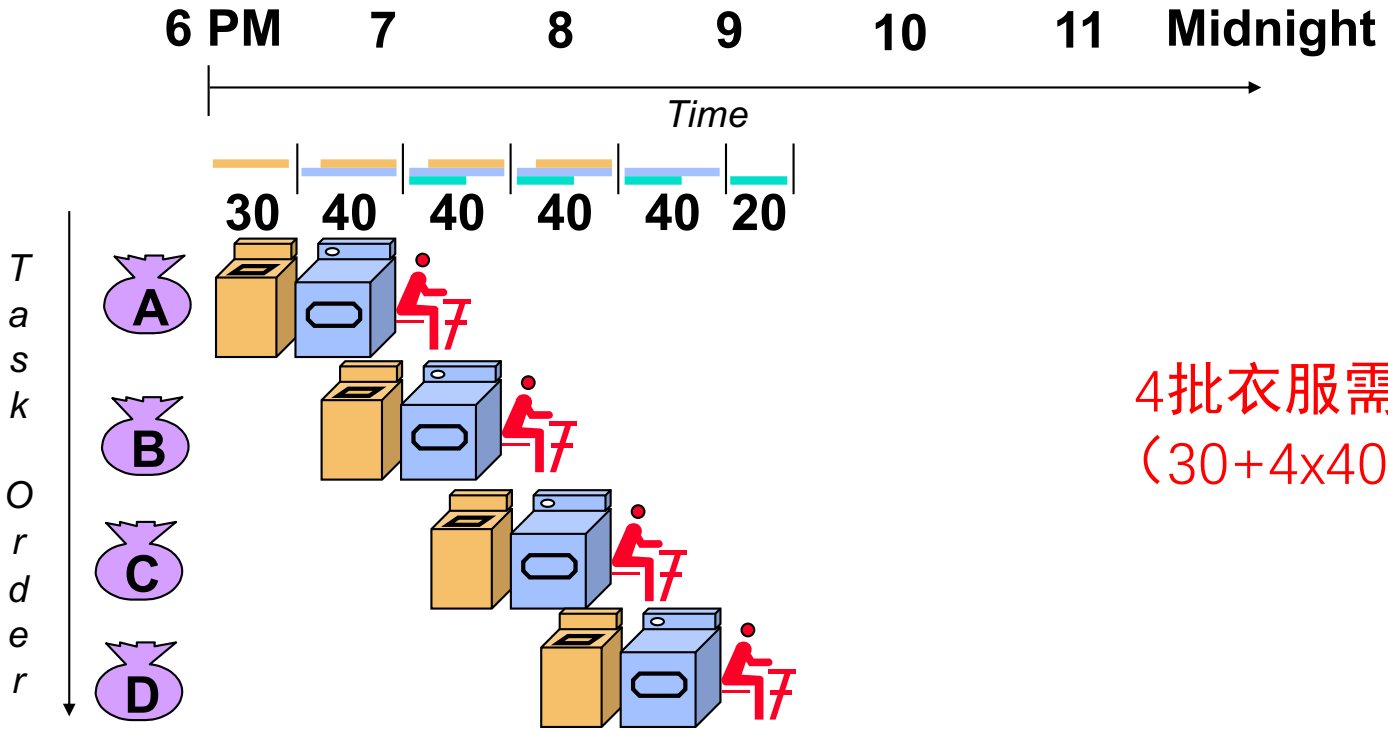
洗衣店的管理者应该如何安排？

串行洗衣（SEQ）



4批衣服需要花费6小时
（4x(30+40+20)=360分钟）

流水线洗衣



4批衣服需要花费3.5小时
（30+4x40+20=210分钟）

流水线的性能指标

- 吞吐率
  - 在单位时间内流水线所完成的任务数量或输出结果的数量。

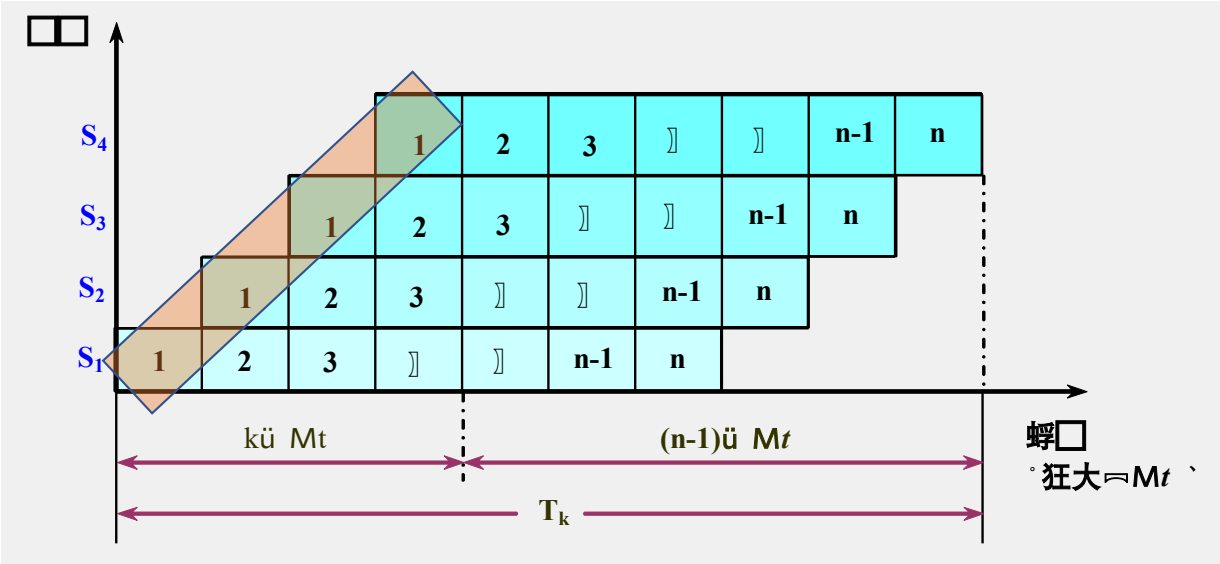
$$TP = \frac{n}{T_k}$$

  - n：任务数
  - T_k：处理完成n个任务所用的时间
  - k：每个任务包含的阶段数

流水线的性能指标

1. 各段时间均相等的流水线

- 时空图



- k段时间后
  - 1：k段做完
  - 2：做了k-1段，还剩1段
  - …
  - n：做了k-(n-1)段，还剩n-1段

流水线的性能指标

1. 各段时间均相等的流水线
   - 对于一条k段线性流水线，完成n个连续任务所需的总时间为

$$T_k = k\Delta t + (n-1)\Delta t = (k+n-1)\Delta t$$

   - 实际吞吐率

$$TP = \frac{n}{(k+n-1)\Delta t}$$

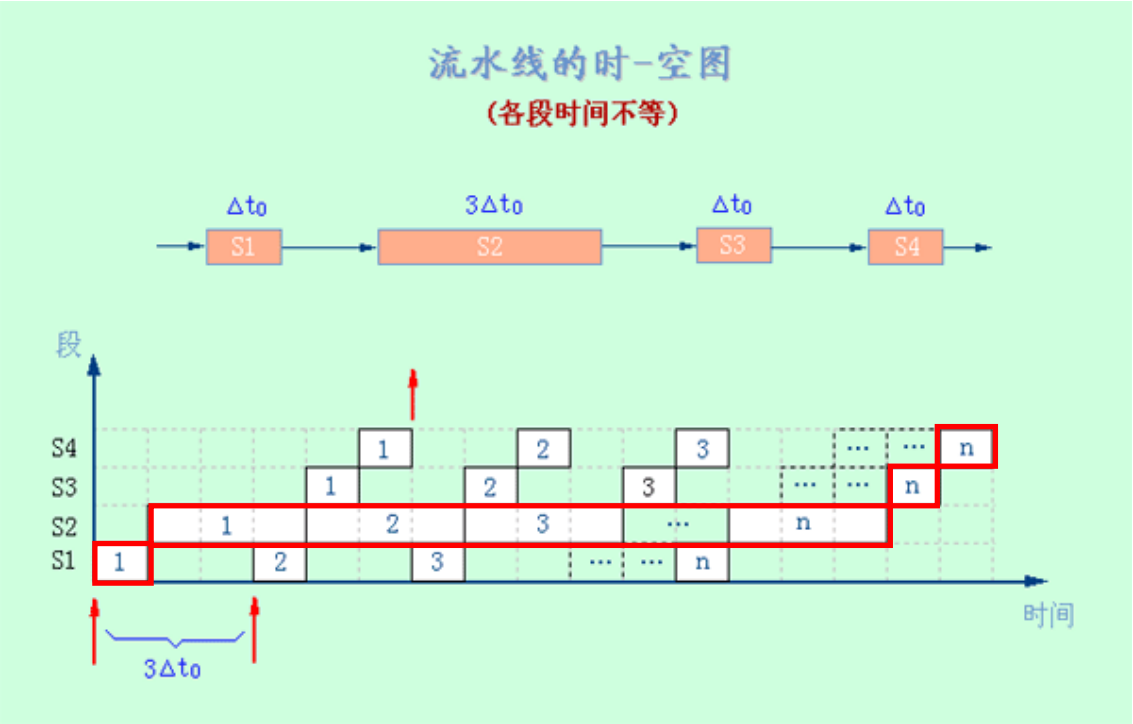
   - 最大吞吐率

$$TP_{\max} = \lim_{n\to\infty} \frac{n}{(k+n-1)\Delta t} = \frac{1}{\Delta t}$$

流水线的性能指标

2. 各段时间不完全相等的流水线

- E.g., 一条4段线性流水线
- S1，S3，S4用时Δt
- S2用时3Δt（瓶颈段）
- 时空图



流水线的性能指标

2. 各段时间不完全相等的流水线
   - Δt_i为第i段的时间，共有k个段
   - 实际吞吐率

$$TP = \frac{n}{\sum_{i=1}^{k} \Delta t_i + (n-1)\max(\Delta t_1, \Delta t_2, \cdots, \Delta t_k)}$$

   - 最大吞吐率

$$TP_{\max} = \frac{1}{\max(\Delta t_1, \Delta t_2, \cdots \Delta t_k)}$$

     - E.g., S1，S3，S4用时Δt，S2用时3Δt

$$TP_{\max} = \frac{1}{3\Delta t}$$

流水线的性能指标

- 加速比
  - 完成同样一批任务，不使用流水线所用的时间与使用流水线所用的时间之比。
    - 不使用流水线（即顺序执行）所用的时间为Ts
    - 使用流水线后所用的时间为Tk

流水线的性能指标

1. 流水线各段时间相等（都是△t）
   - 一条k段流水线完成n个连续任务
     - 所需要的时间为：

$$T_k = (k + n - 1)\Delta t$$

   - 顺序执行n个任务
     - 所需要的时间

$$T_s = nk\Delta t$$

   - 流水线的实际加速比为：

$$S = \frac{nk}{k + n - 1}$$

   - 最大加速比

$$S_{\max} = \lim_{n \to \infty} \frac{nk}{k + n - 1} = k$$

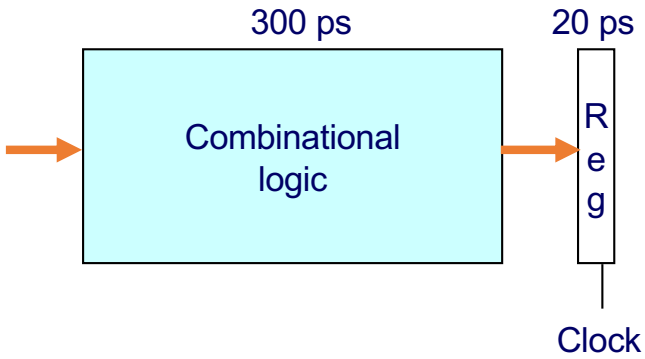
流水线的性能指标

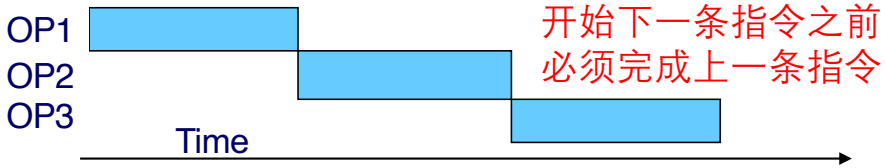
2. 流水线的各段时间不完全相等时
   - 一条k段流水线完成n个连续任务的实际加速比为

$$S=\frac{n\sum_{i=1}^{k}\Delta t_i}{\sum_{i=1}^{k}\Delta t_i+(n-1)\max(\Delta t_1,\Delta t_2,\cdots,\Delta t_k)}$$

硬件系统例子

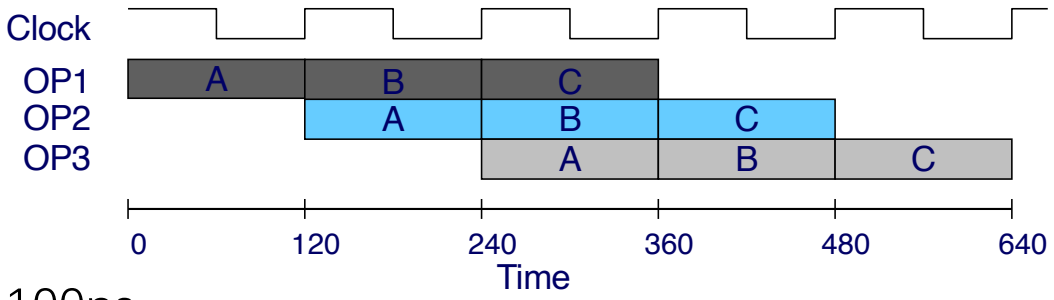
- 硬件系统
  - 一些执行计算的逻辑
    - Computation requires total of 300 picoseconds
  - 一个保存计算结果的寄存器
    - Additional 20 picoseconds to save result in register
  - Must have clock cycle of at least 320ps





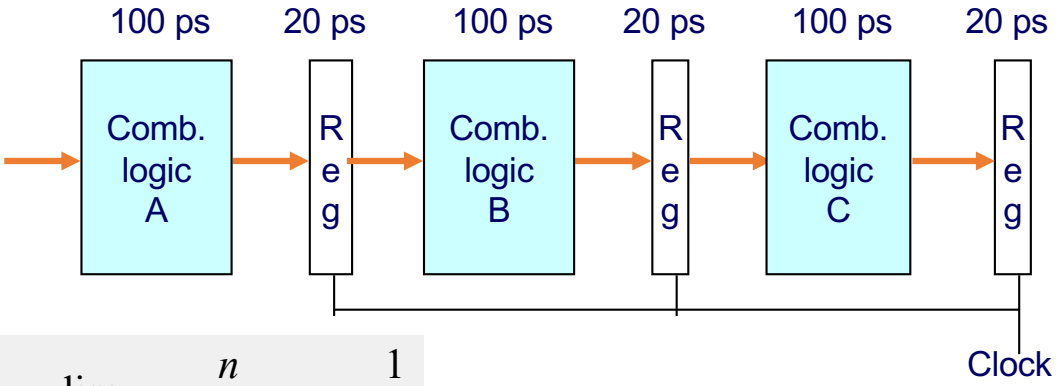
Delay延时 = 320 ps
Throughput吞吐量 = 3.12 GIPS

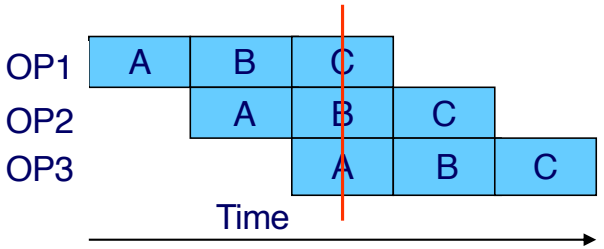
硬件系统例子



- 将系统执行的计算分成三个阶段各100ps
  - Can begin new operation as soon as previous one passes through stage A.
    - Begin new operation every 120ps (clk)
  - Overall latency increases
    - 360ps from start to finish
  - 插入两个流水线寄存器

三阶段流水线化：
- 吞吐率 3.12**X2.67**=8.33
- 单指令延迟320**+40**ps
- 增加一些硬件



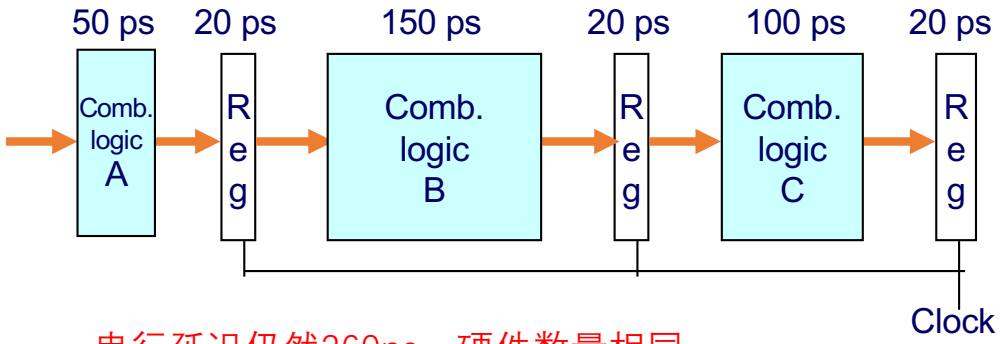


Delay = 360 ps
Throughput = 8.33 GIPS

$$TP_{\max} = \lim_{n\to\infty} \frac{n}{(k+n-1)\Delta t} = \frac{1}{\Delta t}$$

硬件系统例子

- 将系统执行的计算分成三个阶段
  - 各段时间不完全相等



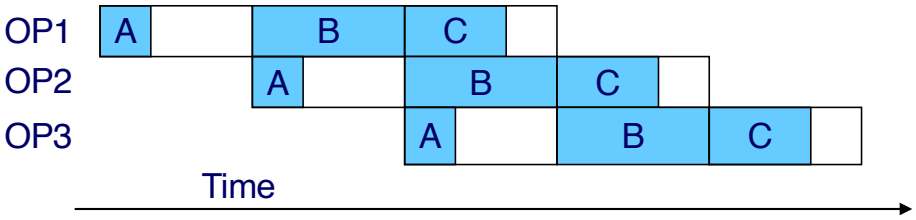
串行延迟仍然360ps，硬件数量相同

三阶段流水线化：

- 时钟周期，由最慢的阶段决定，170ps
- 延迟=170ps*3
- 吞吐率，8.33->5.88

吞吐率、时延都由瓶颈段的延迟限制（其他阶段很多时间处于空闲状态）

把计算划分成一组具有相同延迟的阶段是一个严峻的挑战



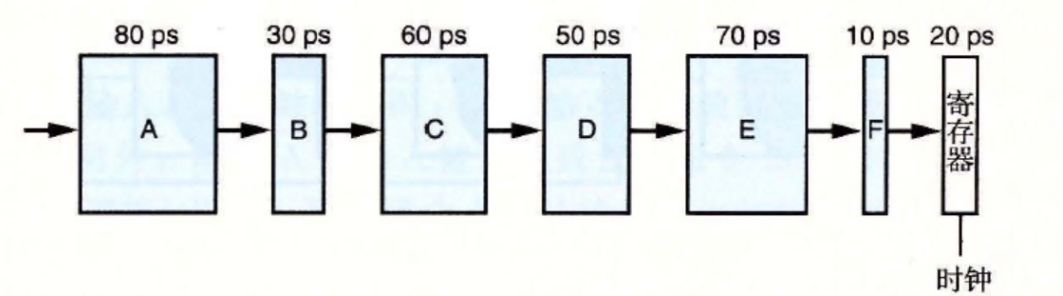
$$TP_{\max} = \frac{1}{\max(\Delta t_1, \Delta t_2, \cdots \Delta t_k)}$$

Delay = 510 ps
Throughput = 5.88 GIPS

例：假设我们分析一个硬件系统的组合逻辑，认为它可以分成6个块，依次命名为A~F，延迟分别为80 、30 、60 、50 、70 和10ps。（1ps=10^{-12}s）

在这些块之间插入流水线寄存器，得到这一硬件系统的流水线化版本。根据在哪里插入流水线寄存器，会出现不同的流水线深度（有多少个阶段）和最大吞吐率的组合。

假设每个流水线寄存器的延迟为20ps。

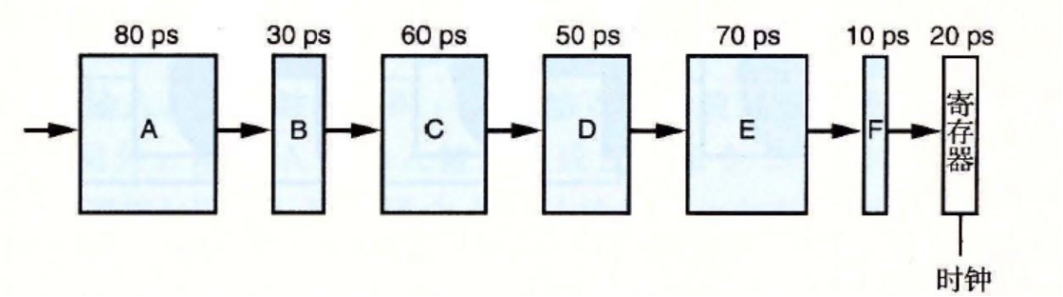


A. 只插入一个寄存器，得到一个两阶段的流水线。要使吞吐率最大化，该在哪里插入寄存器呢？吞吐率和延迟是多少？
A 、B 和C在第一阶段，D 、E 和F在第二阶段。 第一阶段的组合逻辑耗时为170ps ，大于第二阶段的130ps，所以流水线每阶段的时间（时钟周期）为170+20=190ps 。 因此延迟为380ps，吞吐率为1/190ns=5.26GIPS。

例：假设我们分析一个硬件系统的组合逻辑，认为它可以分成6个块，依次命名为A~F，延迟分别为80 、30 、60 、50 、70 和10ps。

在这些块之间插入流水线寄存器，得到这一硬件系统的流水线化版本。根据在哪里插入流水线寄存器，会出现不同的流水线深度（有多少个阶段）和最大吞吐率的组合。

假设每个流水线寄存器的延迟为20ps。



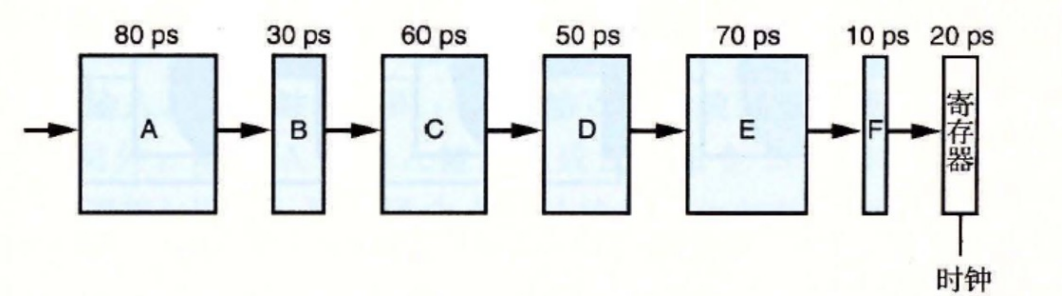
B. 要使一个三阶段的流水线的吞吐率最大化，该将两个寄存器插在哪里呢？吞吐率和延迟是多少?

对一个三阶段流水线，应该使块A和B在第一阶段，块C和D在第二阶段，块E和F在第三阶段。
三段组合逻辑分别耗时110、110、80ps，所以时钟应为110+20=130ps。
延迟为390ps。吞吐量为7. 69GIPS。

例：假设我们分析一个硬件系统的组合逻辑，认为它可以分成6个块，依次命名为A~F，延迟分别为80 、30 、60 、50 、70 和10ps。

在这些块之间插入流水线寄存器，得到这一硬件系统的流水线化版本。根据在哪里插入流水线寄存器，会出现不同的流水线深度（有多少个阶段）和最大吞吐率的组合。

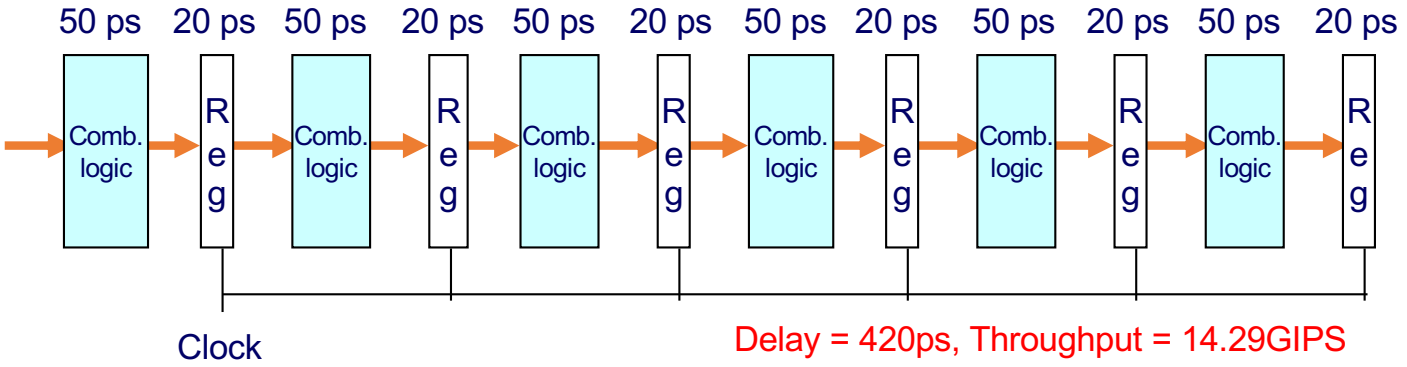
假设每个流水线寄存器的延迟为20ps。



C. 要得到一个吞吐率最大的流水线设计，至少要有几个阶段？描述这个设计及其吞吐率和延迟。

最优的设计应为五阶段流水线，除了E和F处于第五阶段以外，其他每个块是一个阶段。时钟为80+20=100ps，吞吐率为10GIPS，延迟为500ps。
变成更多阶段也不会有帮助了，因为不可能使流水线运行得比以100ps每周期更快了。

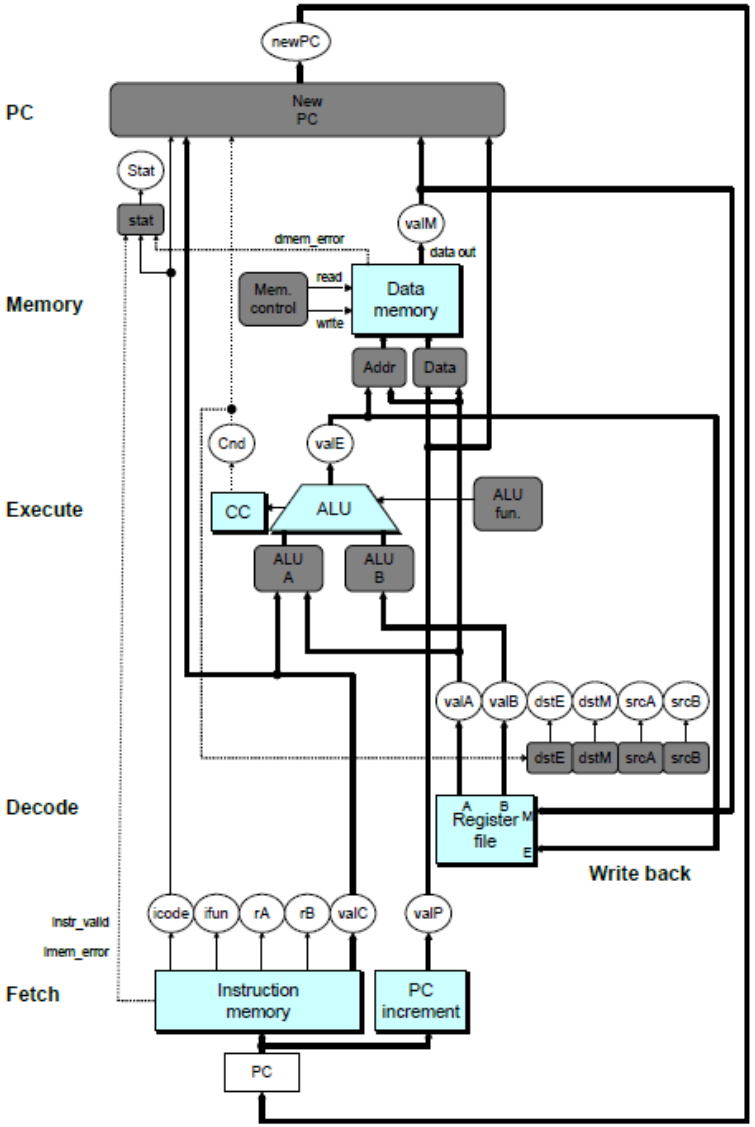
硬件系统例子



- As try to deepen pipeline, overhead of loading registers becomes more significant
  - 寄存器延时占时钟周期百分比
    - 1-stage pipeline: 6.25%=20/320
    - 3-stage pipeline: 16.67%=20/120
    - 6-stage pipeline: 28.57%=20/70
- 为了提高时钟频率，现代处理器采用了很深的15或更多阶段的流水线。

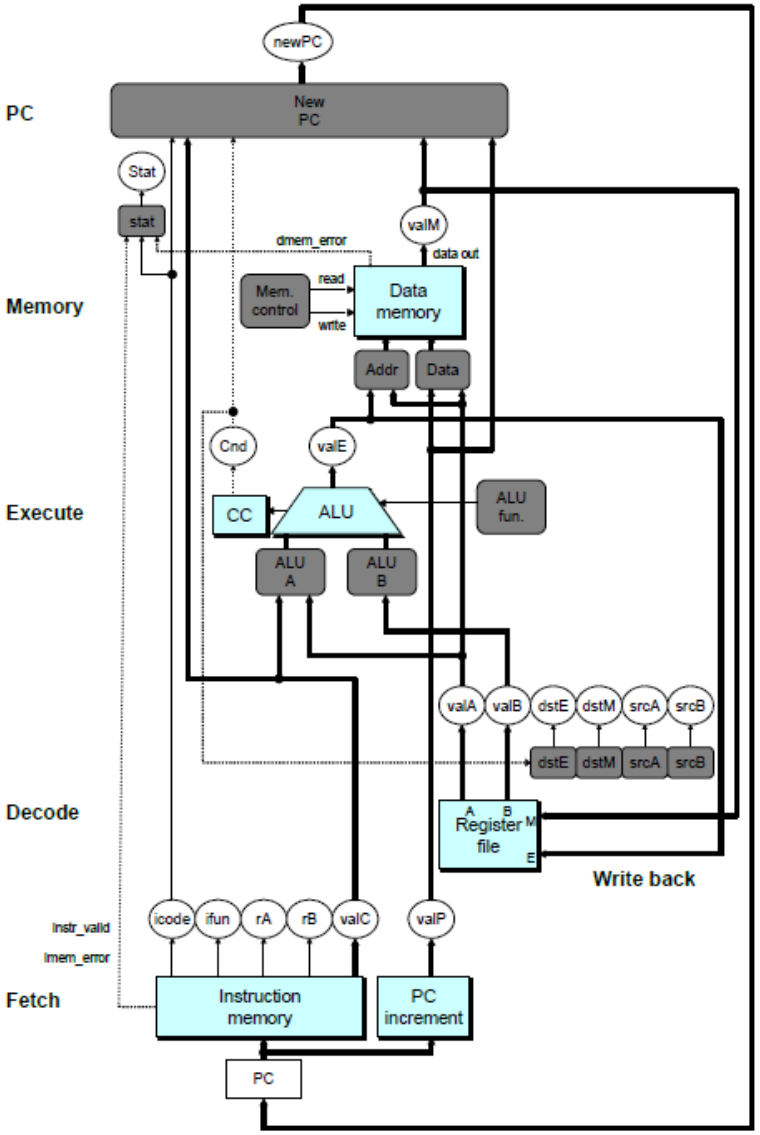
SEQ硬件

- Stages occur in sequence
- One operation in process at a time
  - 取指Fetch
  - 译码Decode
  - 执行Execute
  - 访存Memory
  - 写回Write Back
  - 更新PC



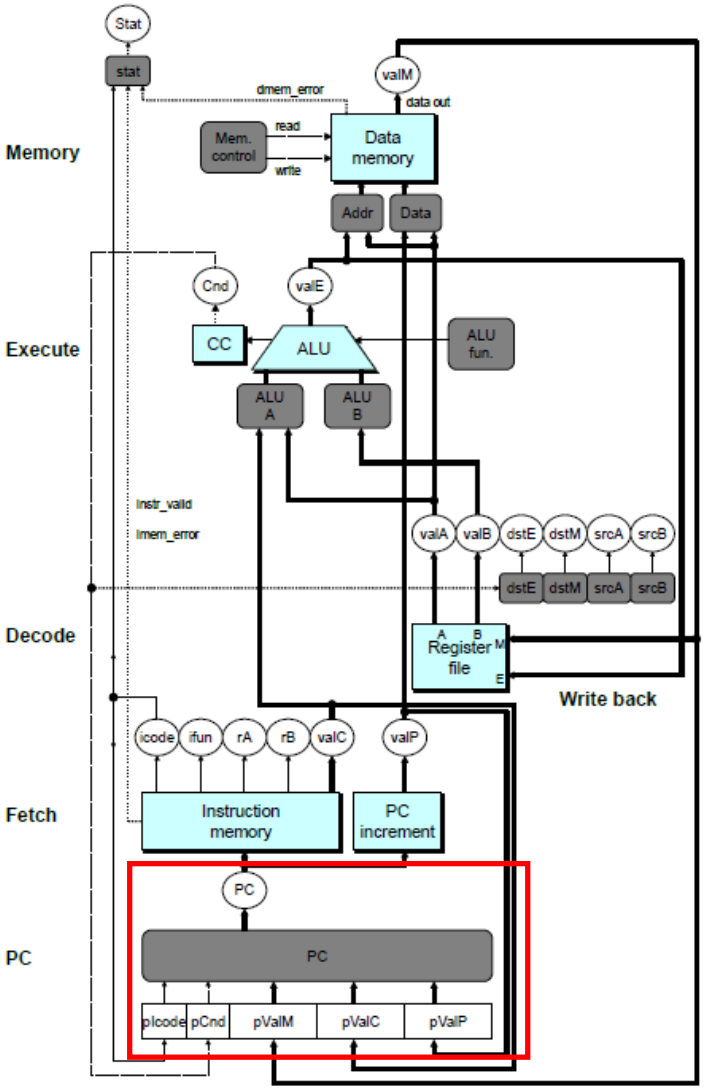
SEQ硬件

- 蓝色框
  - 预先设计好的硬件块
    - 内存、ALU
- 灰色框
  - 控制逻辑块
    - 使用HCL描述
- 白色圆
  - 线路标识
- 白色框（PC）
  - 时钟（时钟周期=指令周期，极慢）
- 粗实线（64根线）
  - 字数据传输
- 细实线（4或8根线）
  - 字节或更小数据传输
- 虚线
  - 单个位的控制值

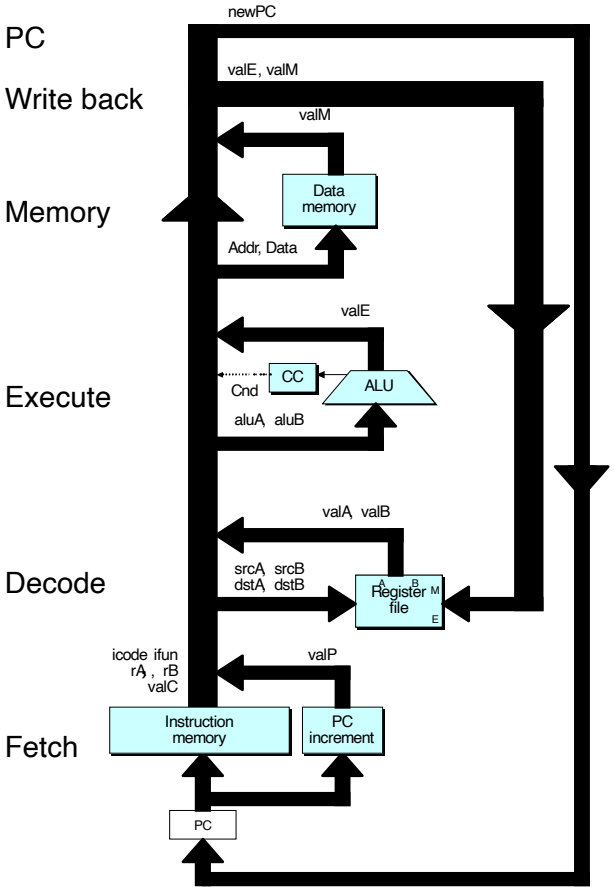


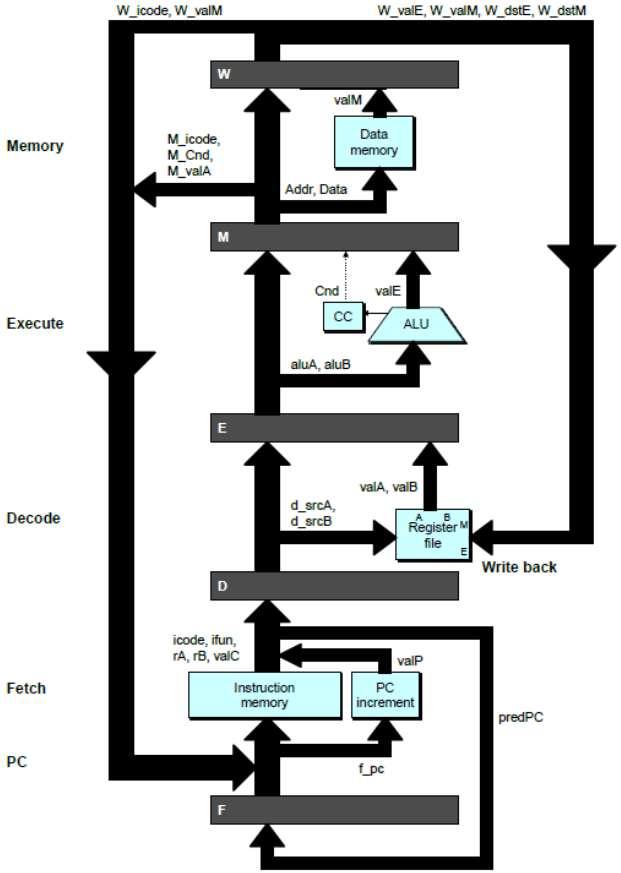
SEQ+

- SEQ
  - PC计算发生在时钟周期结束时，根据当前时钟周期内计算处的信号值来计算PC寄存器的新值
- SEQ+
  - 创建状态寄存器保存指令执行过程中计算的信号
  - 新的时钟周期开始时，计算当前的PC
  - 仍然是顺序的
- 电路重定时
  - 改变系统的状态表示，不改变逻辑行为
    - 重新分配寄存器的位置
  - 以平衡流水线各阶段的延迟



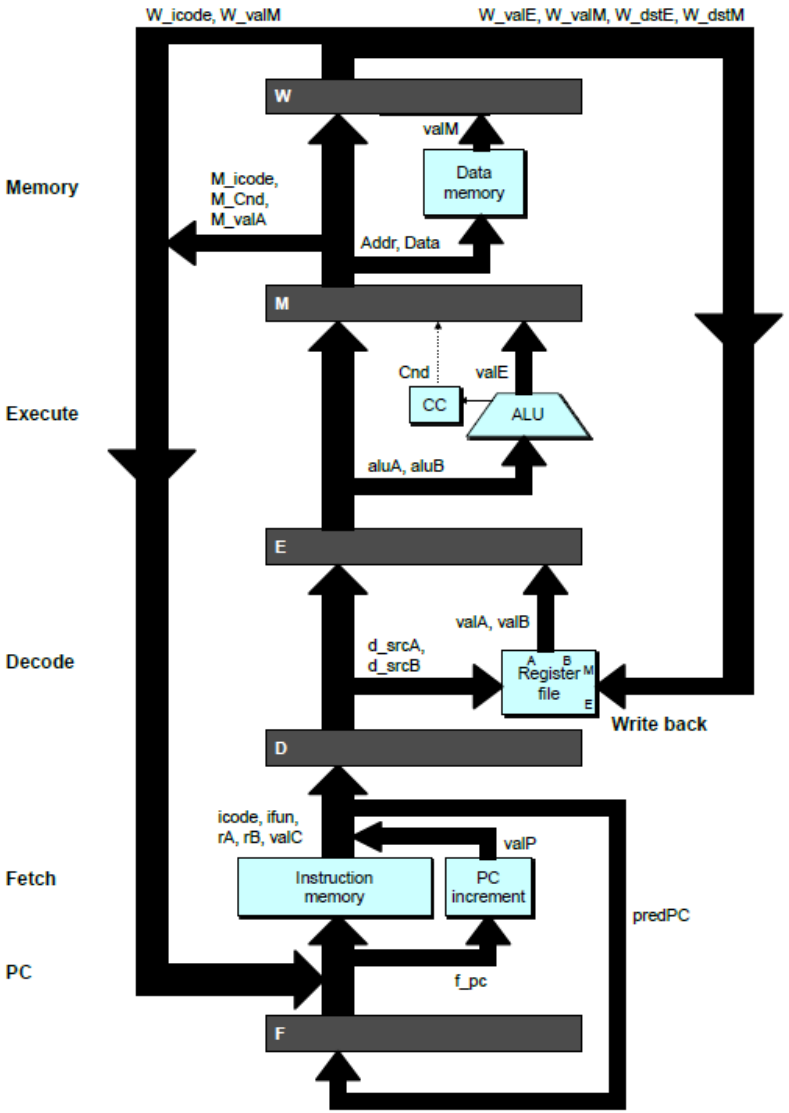
PIPE-=SEQ+ (+) 流水线寄存器





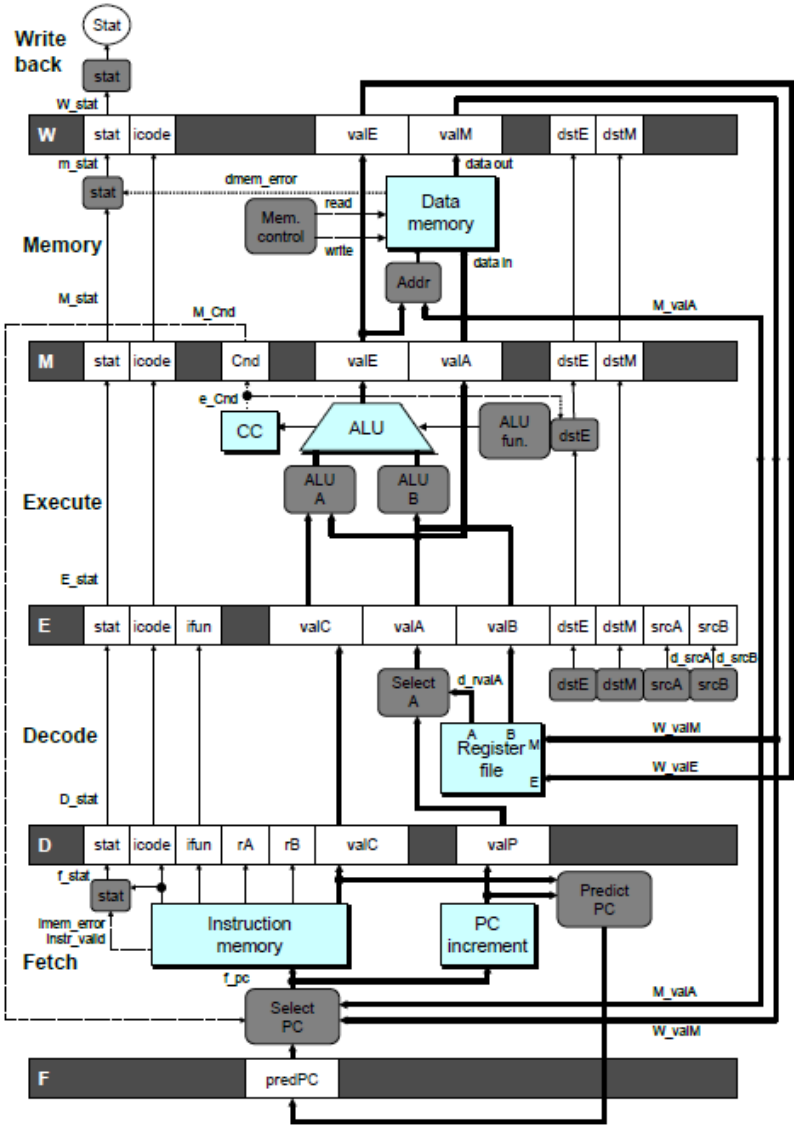
PIPE-

- 五阶段流水线
  - F：保存PC预测值
  - D：取指-D-译码，保存新取出的指令的信息
  - E：译码-E-执行，保存译码的指令和寄存器文件读出的值
  - M：执行-M-访存，保存执行指令的结果，以及条件转移的分支条件和分支目标的信息
  - W：访存-W-反馈，反馈路径将计算得到的值给寄存器文件写（写回）；完成ret指令时，向PC选择逻辑提供返回地址



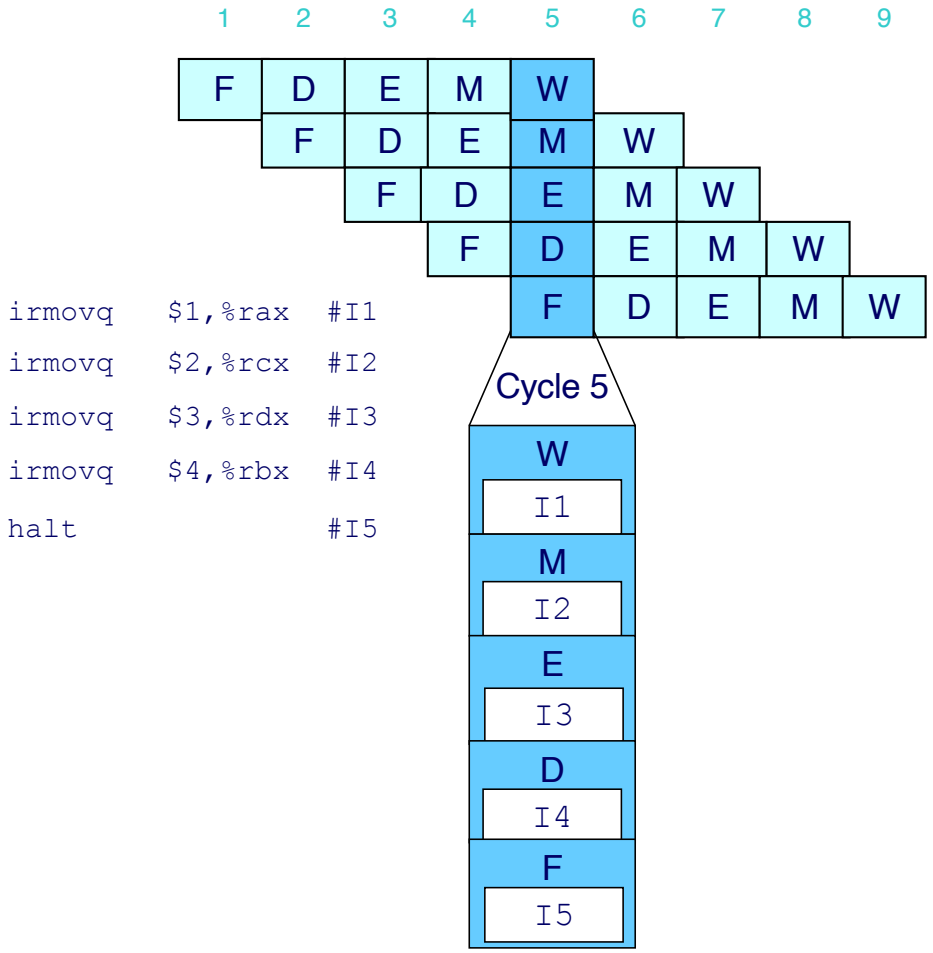
PIPE-硬件

- SEQ/SEQ+
  - 白色椭圆：线路标识（信号）
  - 灰色圆角方框：控制逻辑块（用HCL描述），从一组信号中进行选择 或 计算布尔函数
  - 没有保存在硬件中
- PIPE-
  - 白色方框：寄存器字段
  - 实际硬件



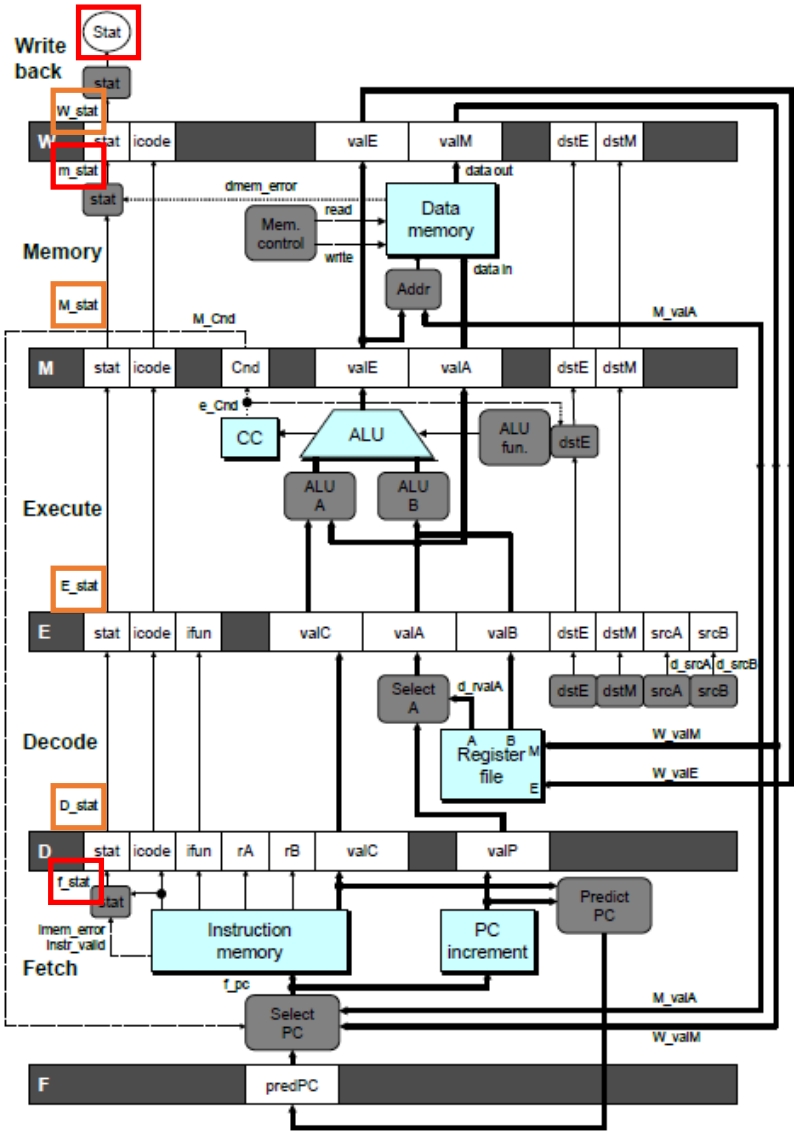
PIPE-示例

- 同一周期，恰好每个阶段各一个指令，且顺序相同



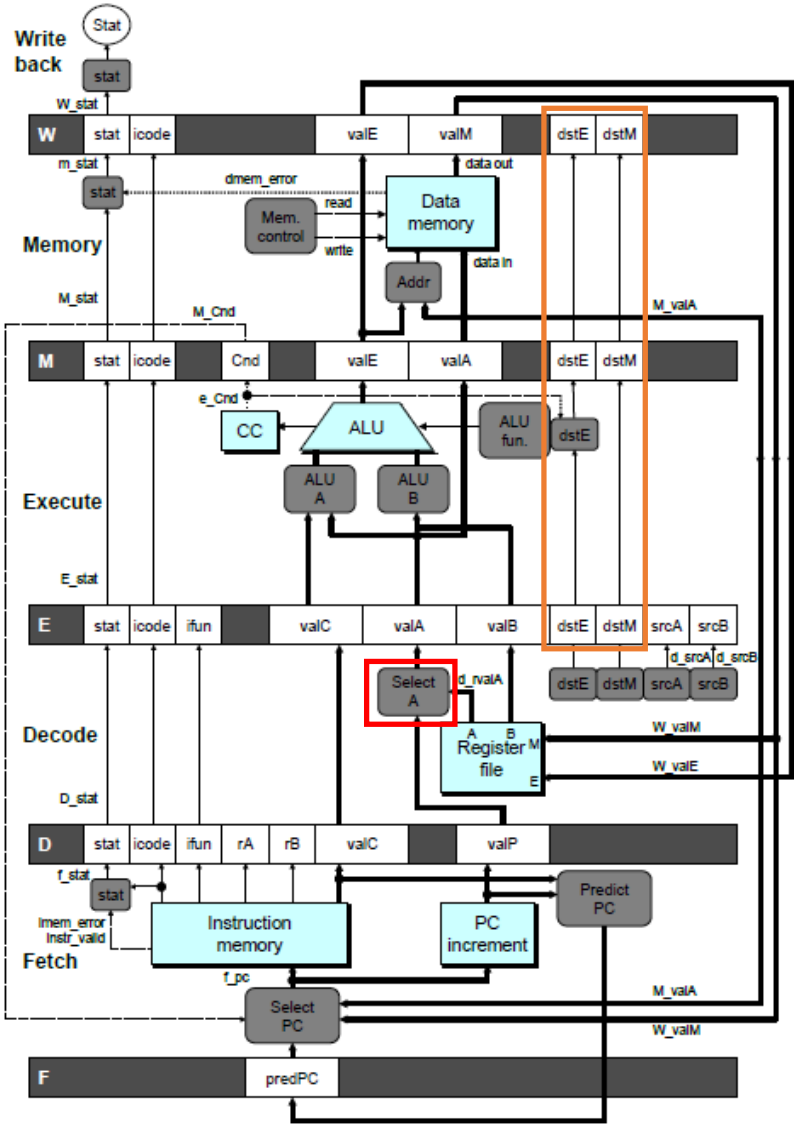
PIPE-

- SEQ/SEQ+
  - 信号值唯一：valC，srcA，…
- PIPE-
  - 信号值保存了多个版本：stat*4（保存着不同指令状态码）
  - 命名
    - 存储在流水线寄存器中的信号：大写流水线寄存器+信号名，D/E/M/W_stat
    - 控制逻辑块的输出
      - 小写阶段名+信号名：f/m_stat
      - 整个处理的状态Stat（根据流水线寄存器W中的状态值，在写回阶段的逻辑块中计算出来）



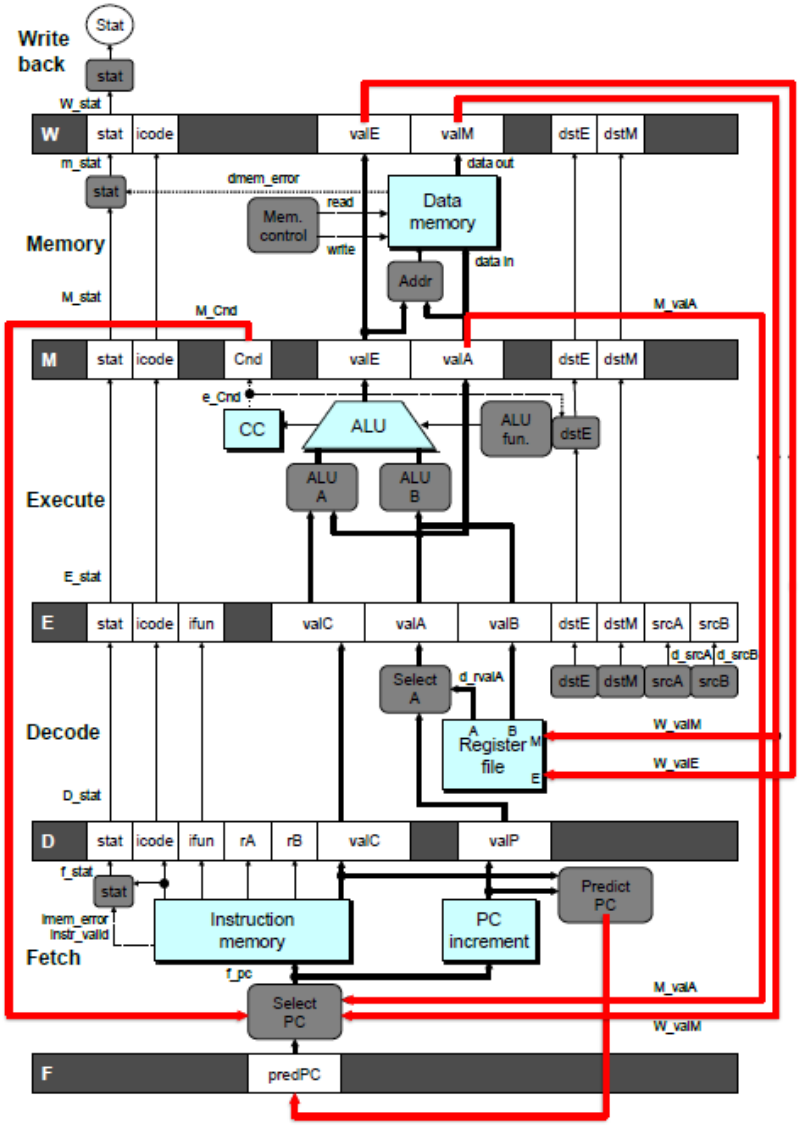
PIPE-

- 前向路径
  - dstE/M在译码阶段得到，在写回阶段使用
  - SEQ/SEQ+：直连寄存器文件写端口
  - PIPE-：在流水线中携带传过执行和访存阶段
- 信号合并
  - Select A
  - valA=valP or d_rvalA
    - D_valP：call访存需要，jxx执行需要
    - d_rvalA：call/ret都不需要
    - 合并以减少流水线寄存器的状态数



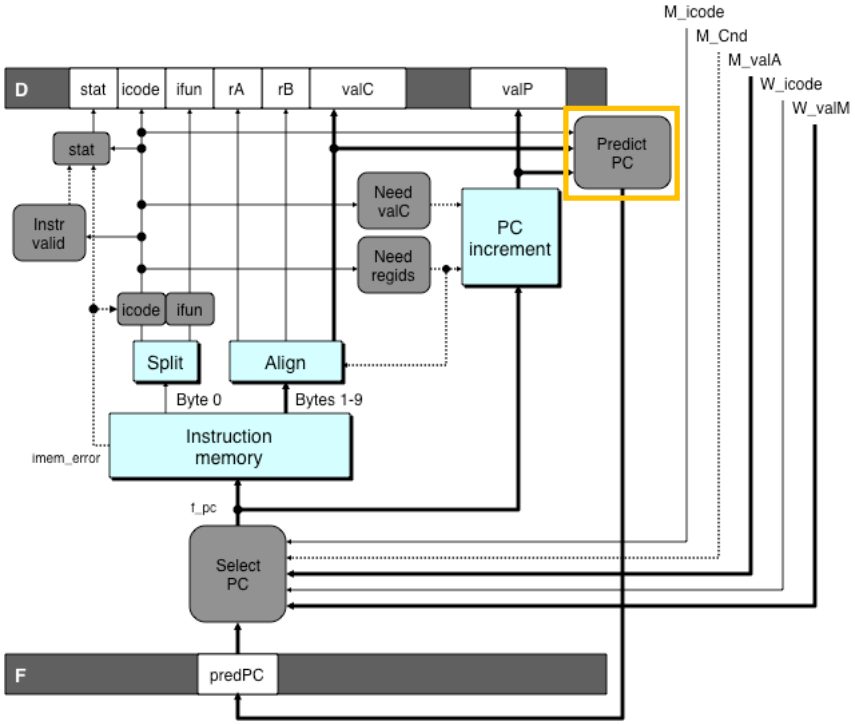
PIPE-

- 后向（反馈）路径
  - Predicted PC
    - Guess value of next PC
  - Branch information
    - Jump taken/not-taken
    - Fall-through or target address
  - Return point
    - Read from memory
  - Register updates
    - To register file write ports



预测PC

- PIPE-
  - 每个时钟周期发射一条新指令/最终完成一条指令
  - 取出当前指令后预测下一条指令位置
    - call/jmp指令：等于指令中的常数字valC
    - jXX指令：几个周期后通过该指令的执行阶段才知道
      - 分支预测
        - 选择分支valC（总是选择，60%）
        - 不选择分支valP
    - ret指令：几个周期后通过该指令的访存阶段才知道
      - 栈返回地址预测
        - call-ret成对出现，硬件保存call产生的返回地址
    - 其他指令：valP



可靠预测

不可靠预测（预测技术）

预测错误需要恢复机制（降低程序性能，编译优化避免，如用条件传送替代条件分支）

危险的反馈路径

依赖关系可能导致流水线产生计算错误（冒险）

- 数据依赖
  - 下一条指令会用到这一条指令计算出的结果

```
1    irmovq $50, %rax
2    addq %rax, %rbx
3    mrmovq 100(%rbx), %rdx
```

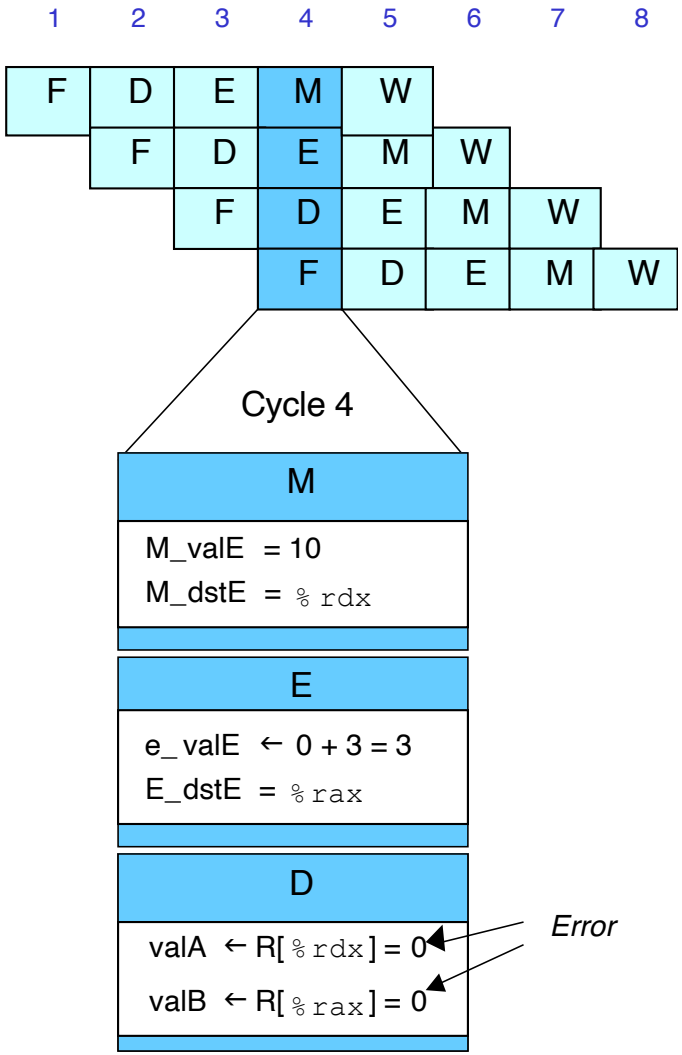
- 控制依赖
  - 一条指令要确定下一条指令的位置

```
loop:
    subq %rdx , %rbx
    jne targ
    irmovq $10 ,%rdx
    jmp loop
targ:
    halt
```

在含有反馈路径的系统中引入流水线是危险的

数据依赖+0NOP

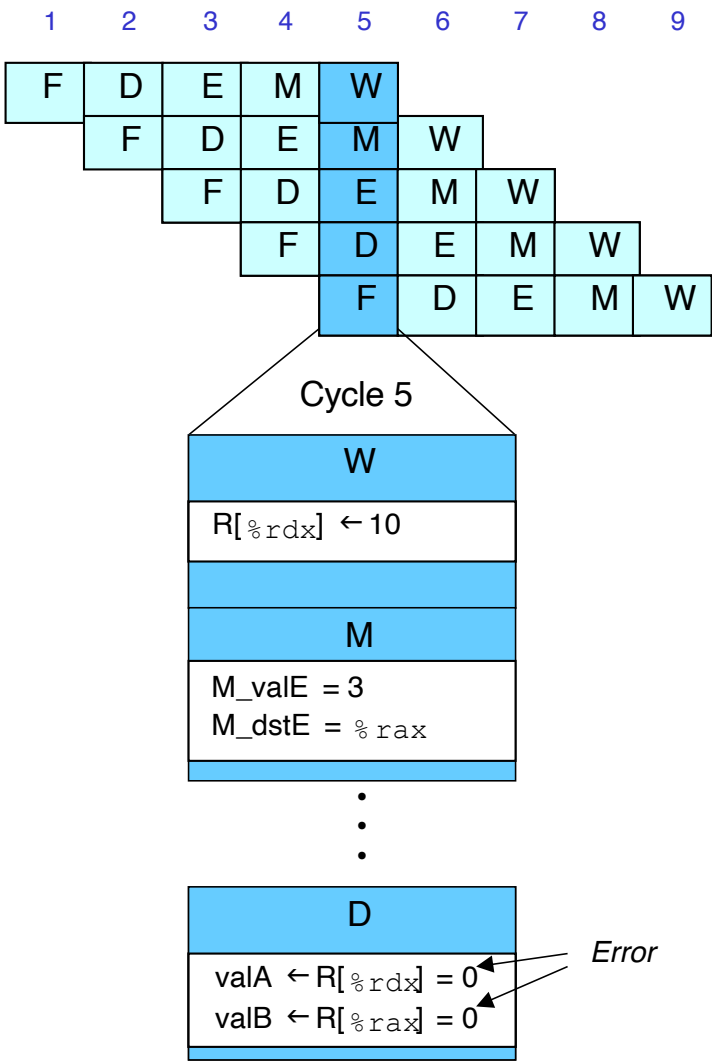
```
# demo-h0.ys
0x000:  irmovq  $10,%rdx
0x00a:  irmovq   $3,%rax
0x014:  addq  %rdx,%rax
0x016: halt
```



数据依赖+1NOP

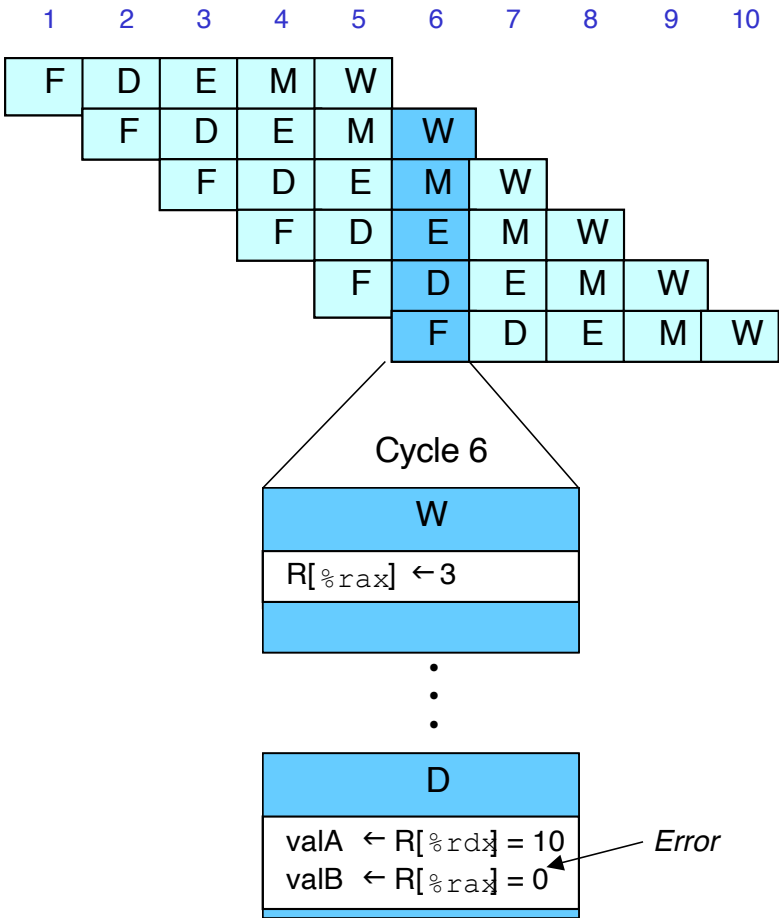
```
# demo-h1.ys

0x000: irmovq  $10,%rdx
0x00a: irmovq   $3,%rax
0x014: nop
0x015: addq %rdx,%rax
0x017: halt
```



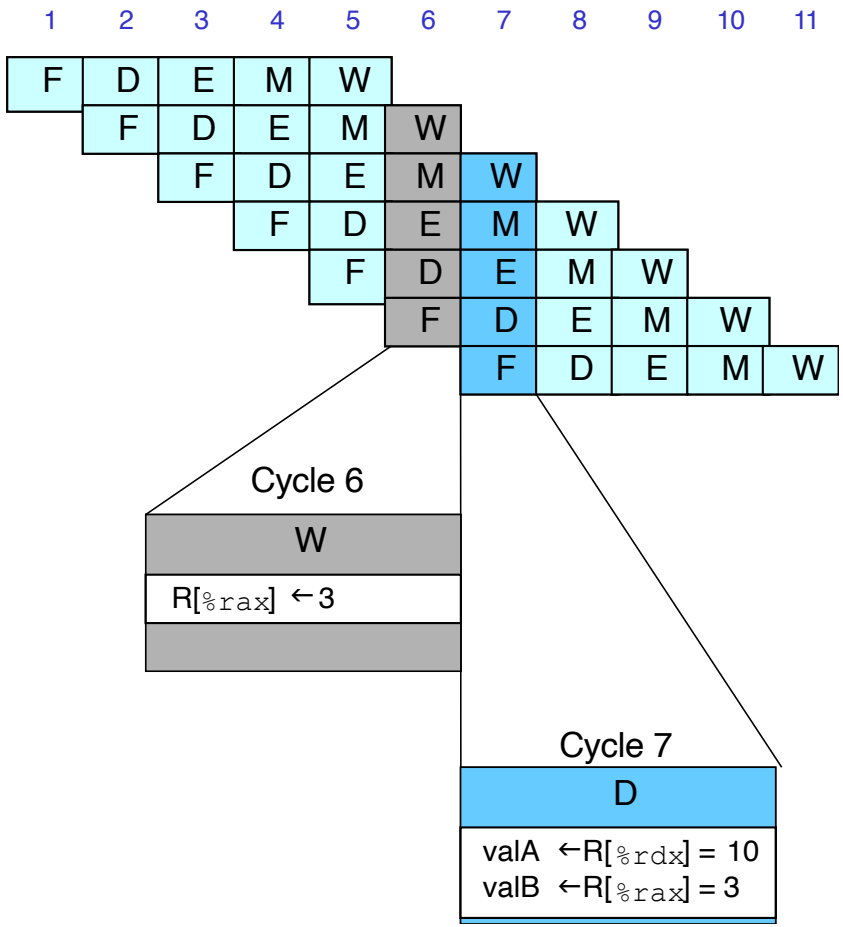
数据依赖+2NOP

```
# demo-h2.ys
0x000: irmovq  $10,%rdx
0x00a: irmovq   $3,%rax
0x014: nop
0x015: nop
0x016: addq %rdx,%rax
0x018: halt
```



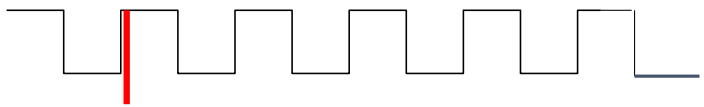
数据依赖+3NOP

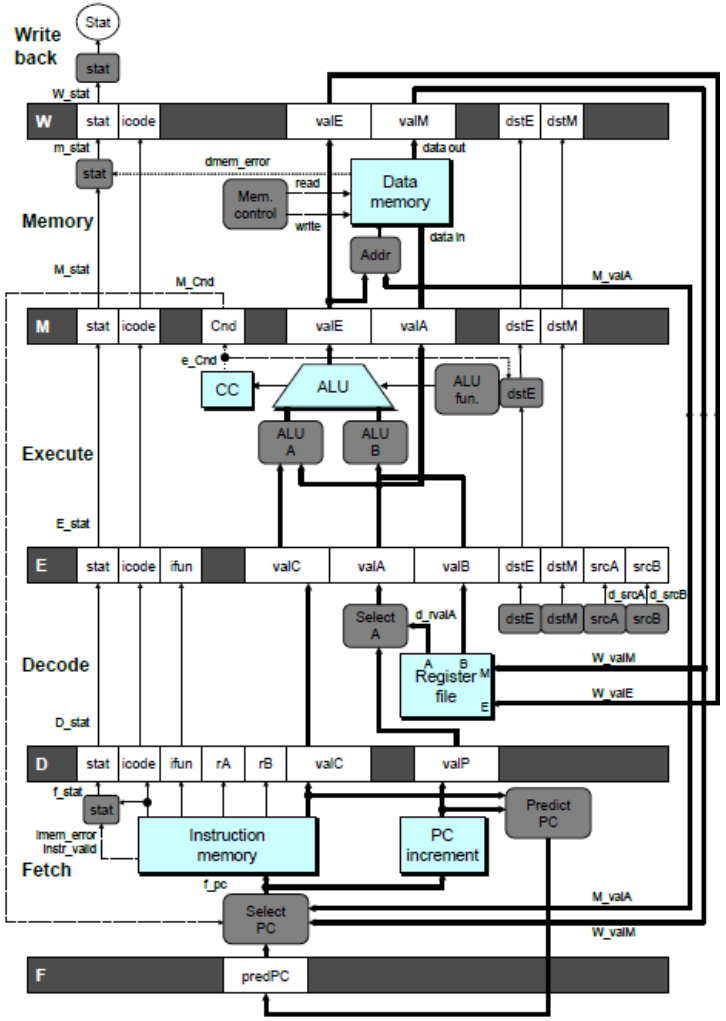
```
# demo-h3.ys
 0x000: irmovq  $10,%rdx
 0x00a: irmovq   $3,%rax
 0x014: nop
 0x015: nop
 0x016: nop
 0x017: addq %rdx,%rax
 0x019: halt
```



3NOP在数据依赖的指令之间
创造了延迟

```
irmovq  $3,   %rax
```

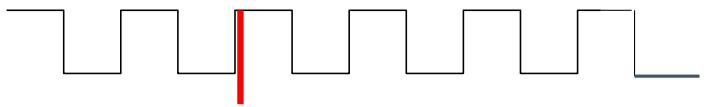


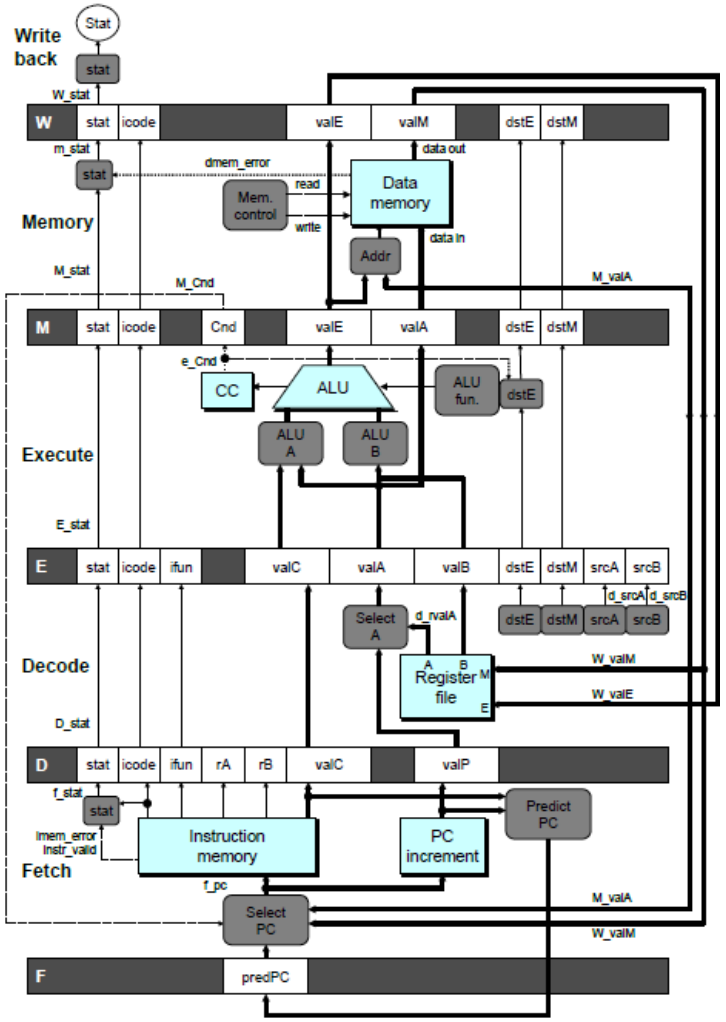


stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

predPC:10

```
irmovq  $3,   %rax
```



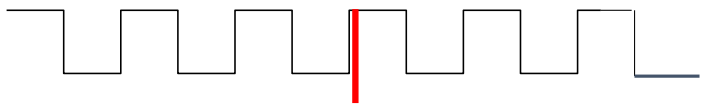


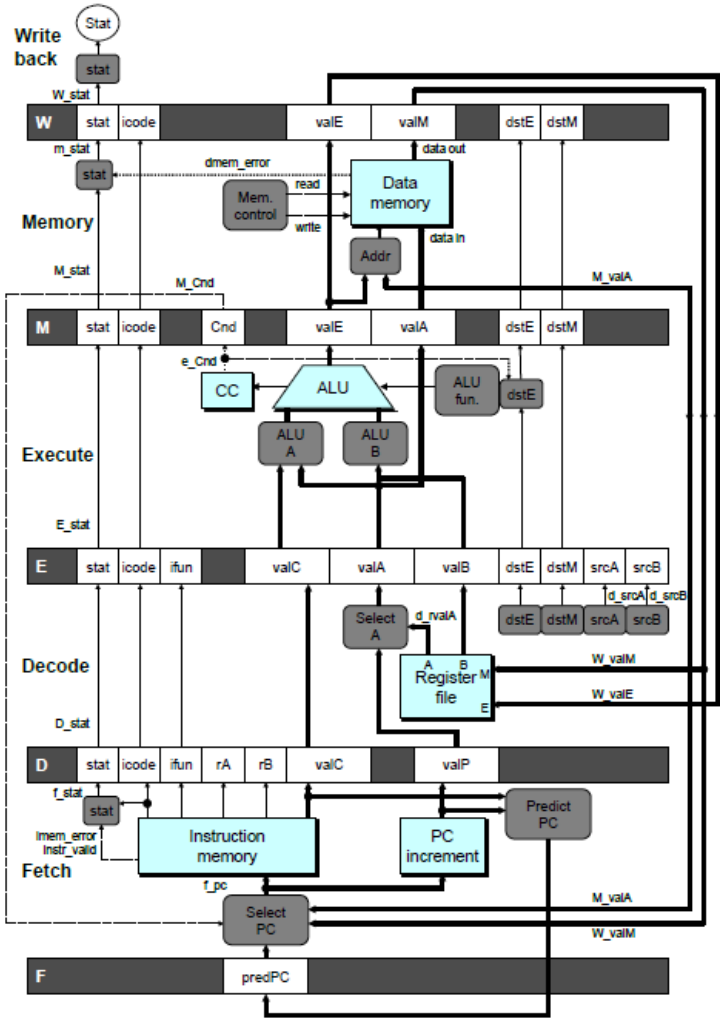
stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

predPC:10

```
irmovq  $3,   %rax
```





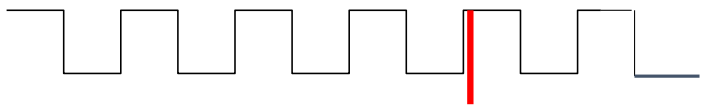
stat:AOK icode:3(irmovq) valE:3 destE:0(rax)

stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

predPC:10

```
irmovq  $3,   %rax
```



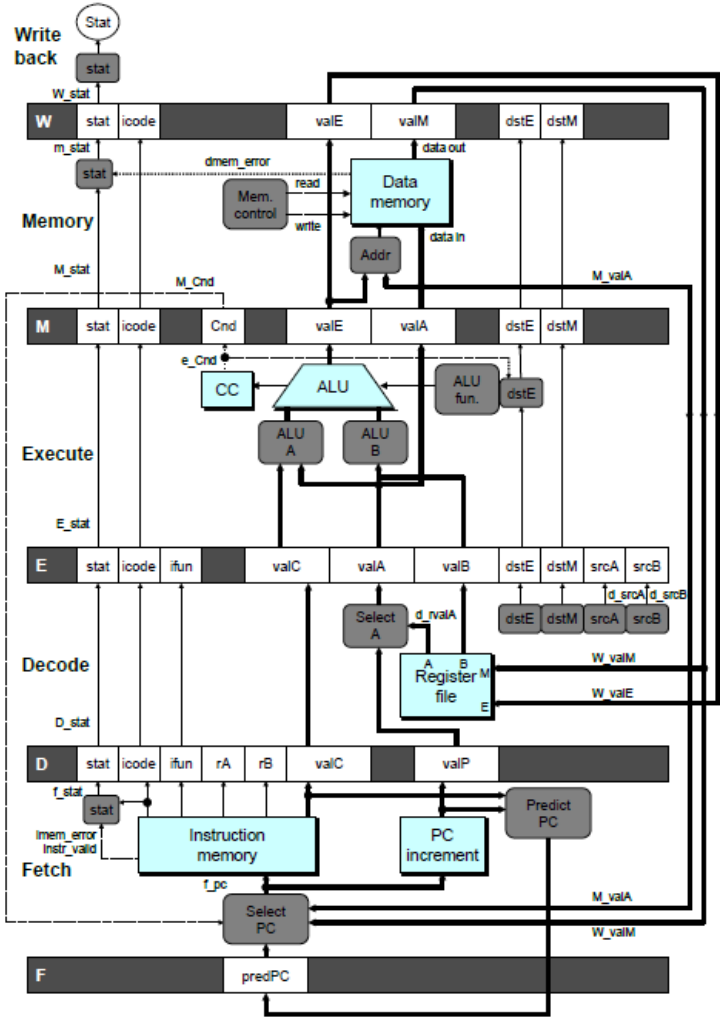
stat:AOK icode:3 valE:3 destE:0

stat:AOK icode:3(irmovq) valE:3 destE:0(rax)

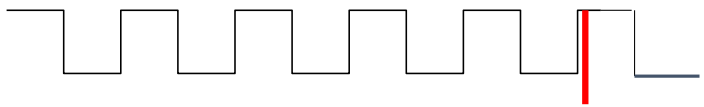
stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

predPC:10



```
irmovq   $3,    %rax
```



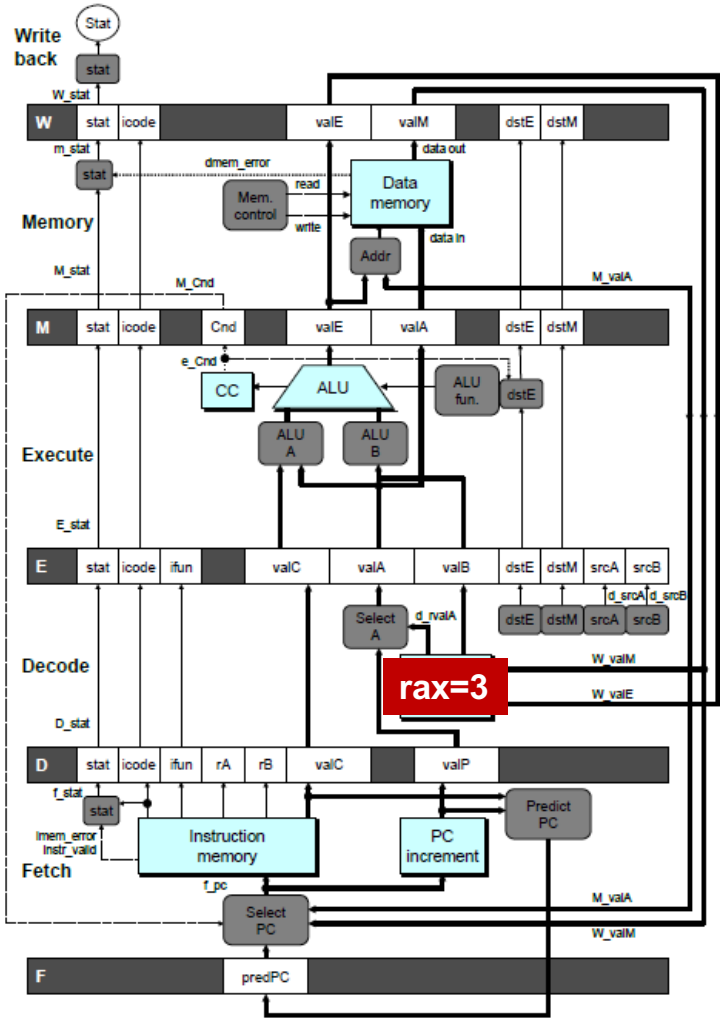
stat:AOK icode:3 valE:3 destE:0

stat:AOK icode:3(irmovq) valE:3 destE:0(rax)

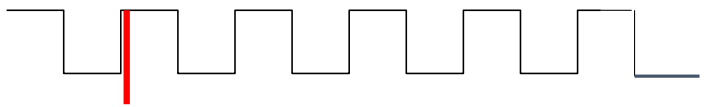
stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

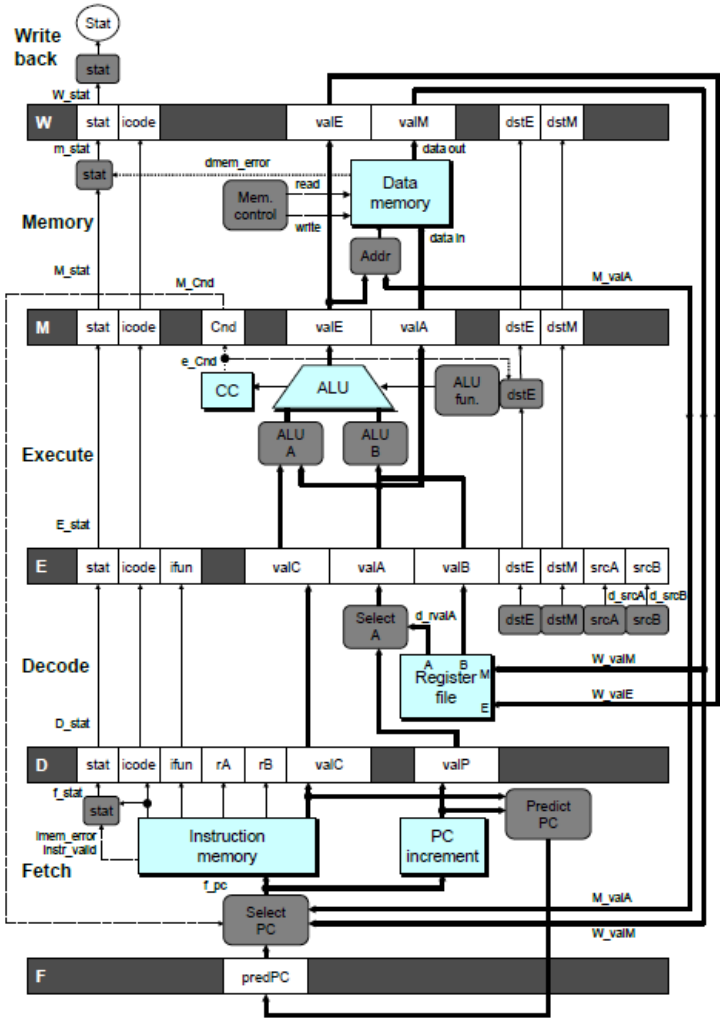
stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

predPC:10



```
irmovq   $3,    %rax
add      %rdx, %rax
```

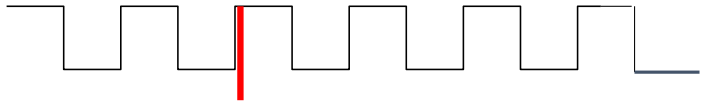


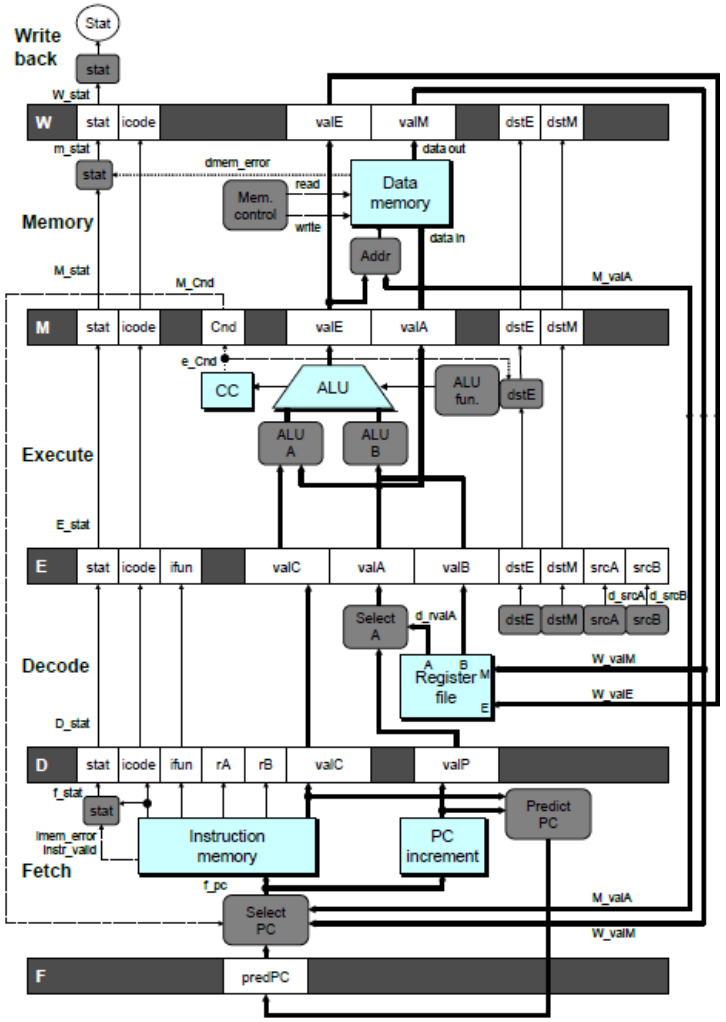


stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

predPC:10

```
irmovq   $3,   %rax
add      %rdx, %rax
```





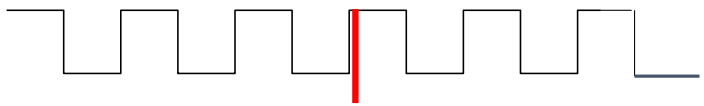
stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

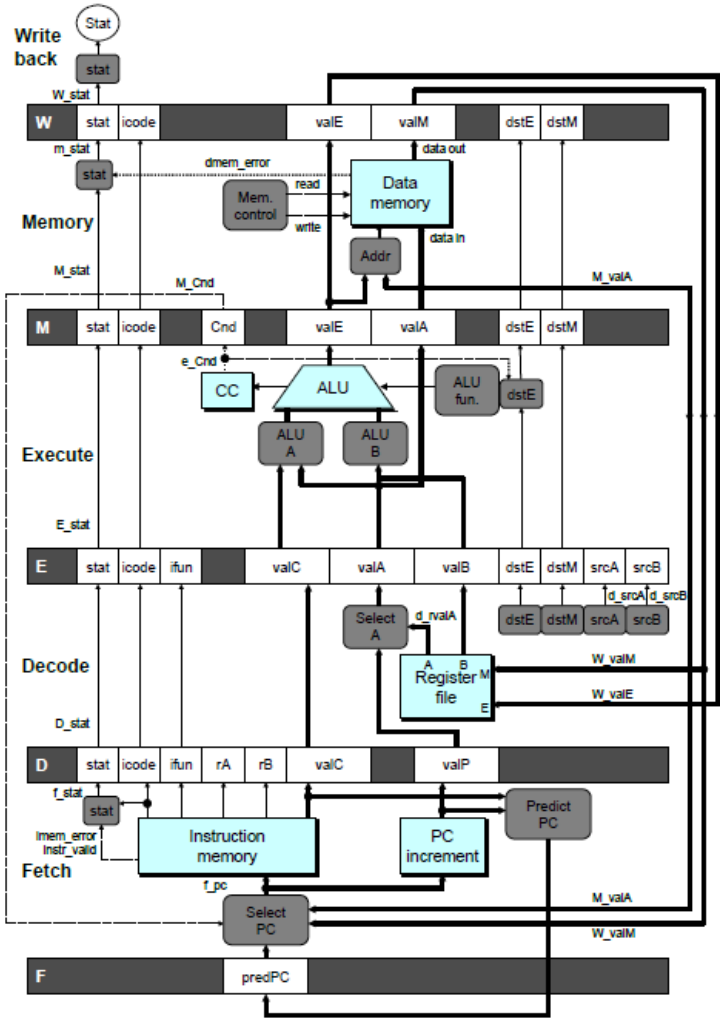
stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

stat:AOK icode:6(addq) ifun:2 ra:2(rdx) rb:0(rax) valp:12

predPC:10 predPC:12

```
irmovq  $3,   %rax
add     %rdx, %rax
```





stat:AOK icode:3(irmovq) valE:3 destE:0(rax)

stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

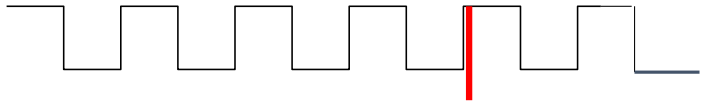
stat:AOK icode:6(addq) ifun:2, valA:0 valB:0 destE:0 srcA:2 srcB:0

stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

stat:AOK icode:6(addq) ifun:2 ra:2(rdx) rb:0(rax) valp:12

predPC:10 predPC:12

```
irmovq  $3,   %rax
add     %rdx, %rax
```



stat:AOK icode:3 valE:3 destE:0

stat:AOK icode:3(irmovq) valE:3 destE:0(rax)

stat:AOK icode:6(addq) valE:0 destE:0(rax)

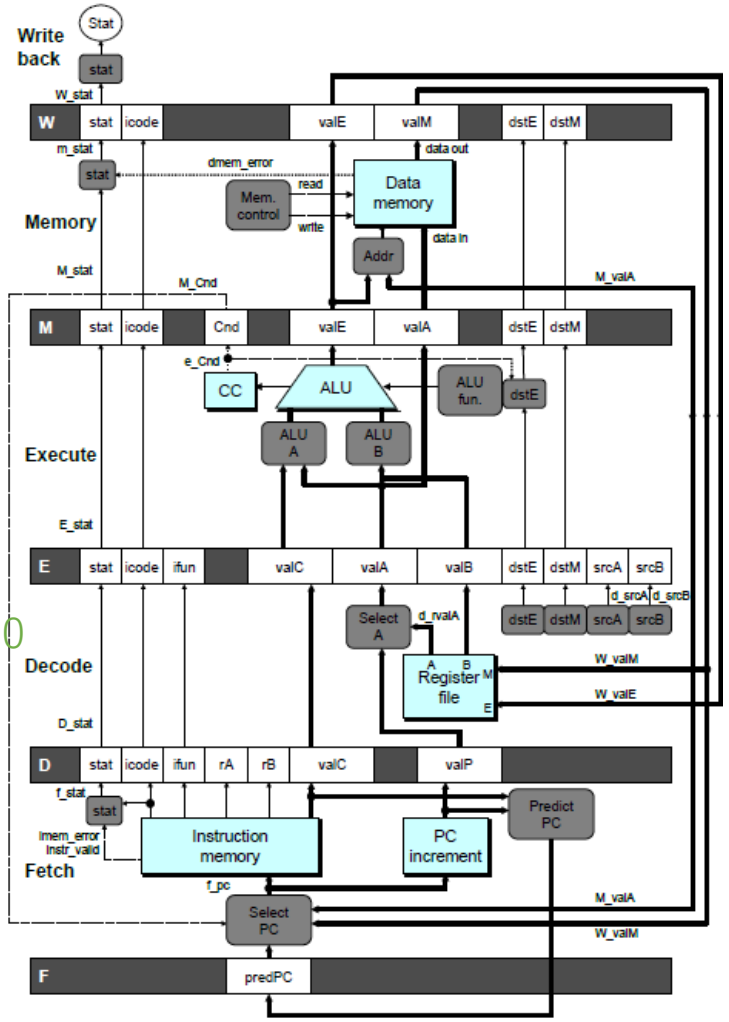
stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

stat:AOK icode:6(addq) ifun:2, valA:0 valB:0 destE:0(rax) srcA:2(rdx) srcB:0

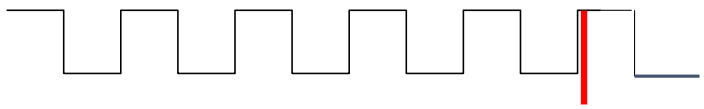
stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

stat:AOK icode:6(addq) ifun:2 ra:2(rdx) rb:0(rax) valp:12

predPC:10 predPC:12



```
irmovq   $3,    %rax
add      %rdx, %rax
```



stat:AOK icode:3 valE:3 destE:0

stat:AOK icode:6(addq) valE:0 destE:0

stat:AOK icode:3(irmovq) valE:3 destE:0(rax)

stat:AOK icode:6(addq) valE:0 destE:0(rax)

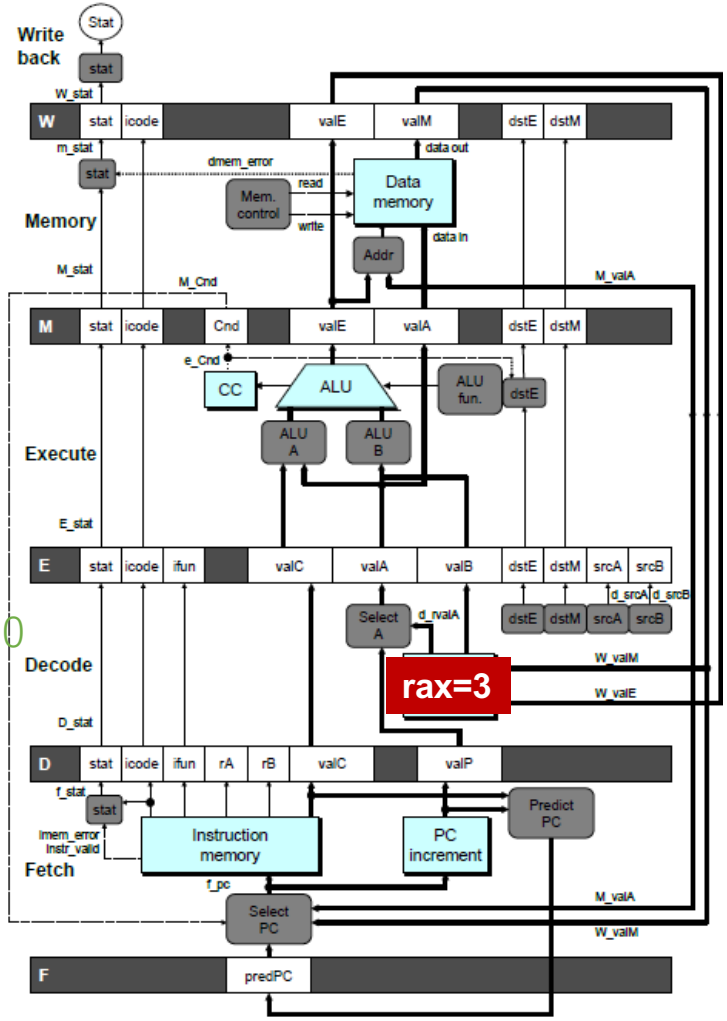
stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

stat:AOK icode:6(addq) ifun:2, valA:0 valB:0 destE:0(rax) srcA:2(rdx) srcB:0

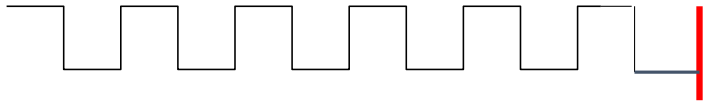
stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

stat:AOK icode:6(addq) ifun:2 ra:2(rdx) rb:0(rax) valp:12

predPC:10 predPC:12



```
irmovq  $3,   %rax
add     %rdx, %rax
```



stat:AOK icode:3 valE:3 destE:0

stat:AOK icode:6(addq) valE:0 destE:0

stat:AOK icode:3(irmovq) valE:3 destE:0(rax)

stat:AOK icode:6(addq) valE:0 destE:0(rax)

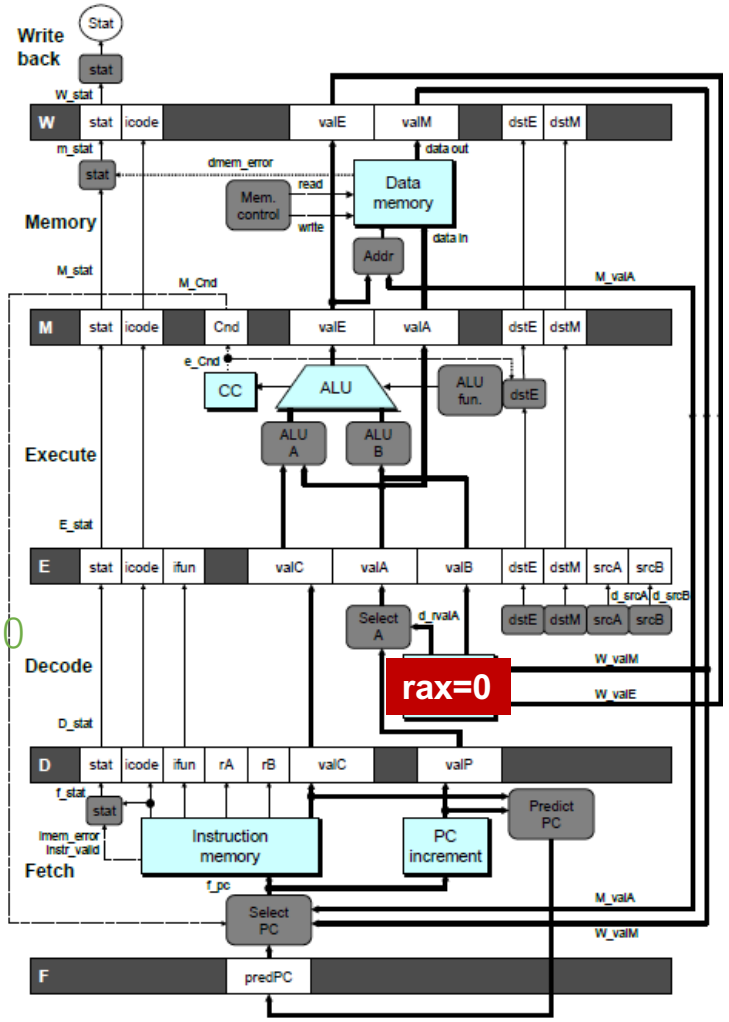
stat:AOK icode:3(irmovq) ifun:0 valc:3 destE:0(rax)

stat:AOK icode:6(addq) ifun:2, valA:0 valB:0 destE:0(rax) srcA:2(rdx) srcB:0

stat:AOK icode:3(irmovq) ifun:0 ra:0xf rb:0(rax) valc:3 valp:10

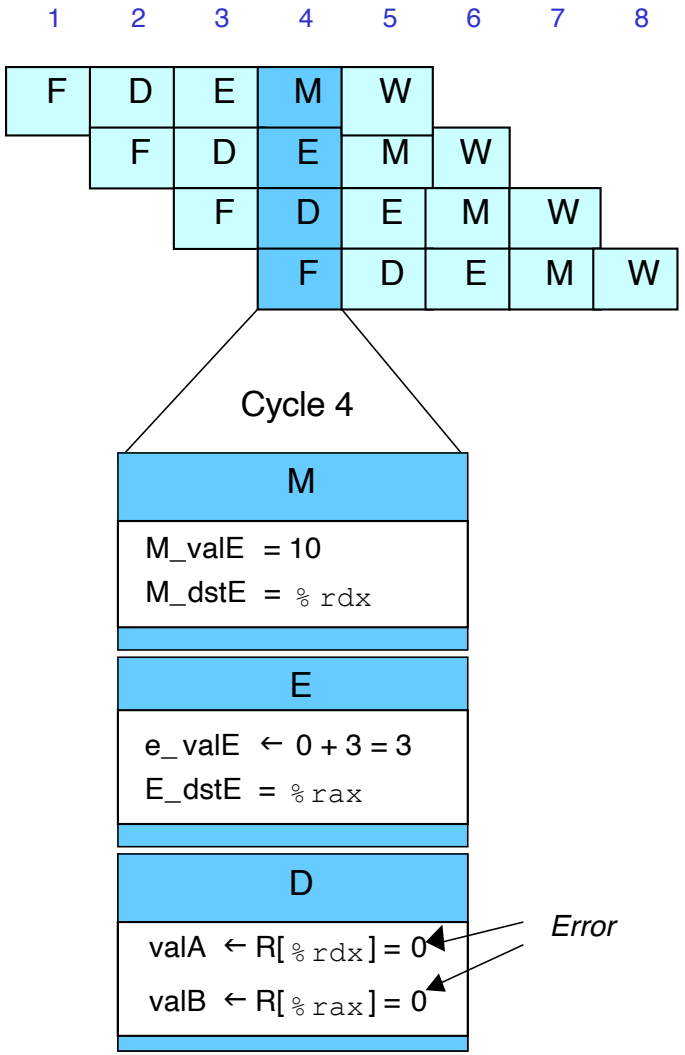
stat:AOK icode:6(addq) ifun:2 ra:2(rdx) rb:0(rax) valp:12

predPC:10 predPC:12



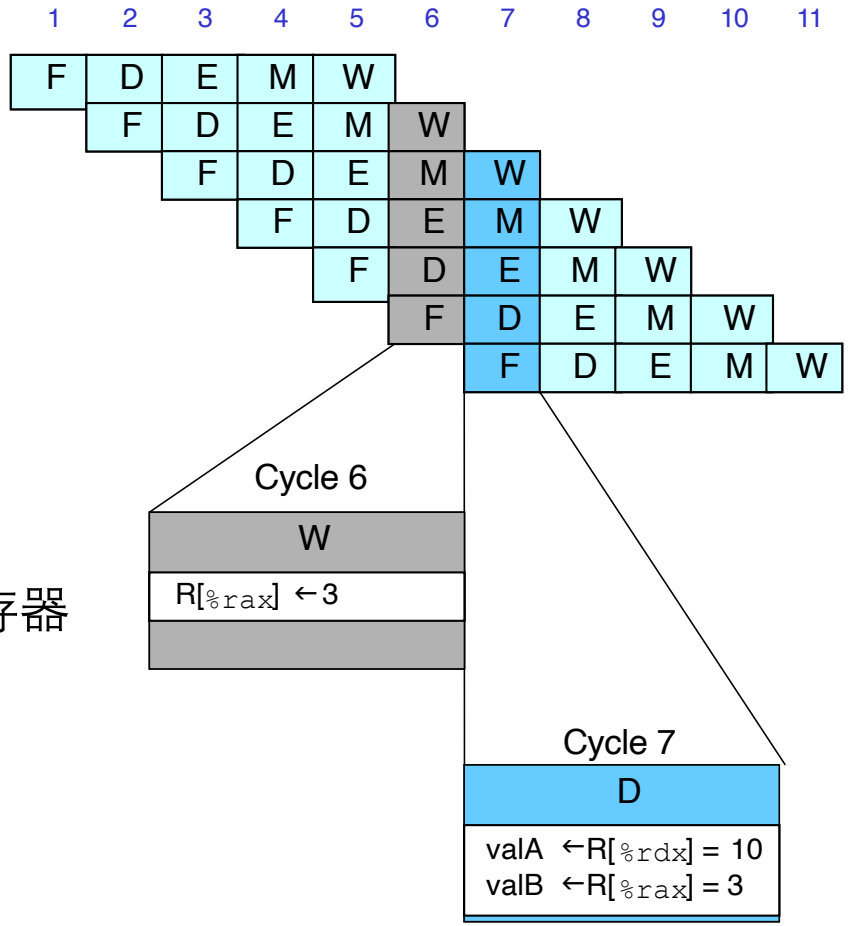
数据依赖+0NOP

```
# demo-h0.ys
0x000:  irmovq  $10,%rdx
0x00a:  irmovq   $3,%rax
0x014:  addq  %rdx,%rax
0x016: halt
```



数据冒险

```
# demo-h3.ys
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: nop
0x015: nop
0x016: nop
0x017: addq %rdx,%rax
0x019: halt
```



- 流水线化的处理器
  - 在译码阶段从寄存器文件读操作数
  - 三个周期后，写回阶段才将结果写到寄存器文件

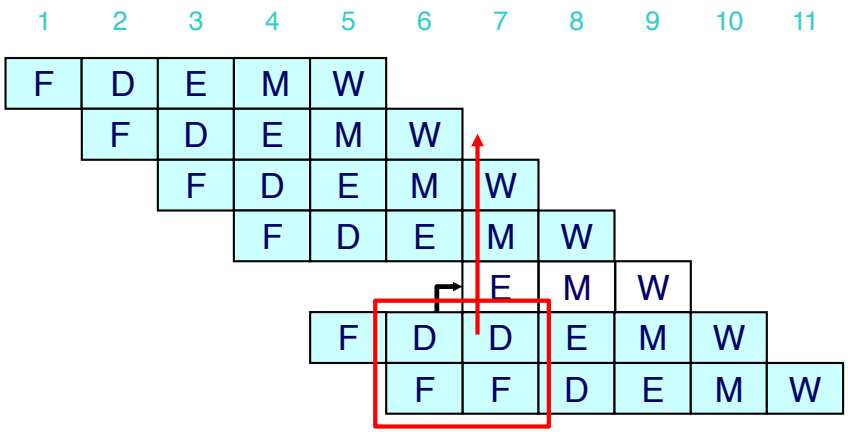
使用暂停（stalling）避免数据冒险

- 让一条指令停在译码阶段，直到产生其源操作数的指令通过写回阶段
  - 下一条指令停在取指阶段
  - 实际阻塞了一组指令
  - 前序指令继续通过

使用暂停（stalling）避免数据冒险

- 2nop为例
- C6
  - 对addq译码，暂停控制逻辑 发现数据冒险（写回%rax未进行）
  - 在执行阶段中插入气泡（动态插入的nop，不改变状态）
- C7
  - 重复对addq译码

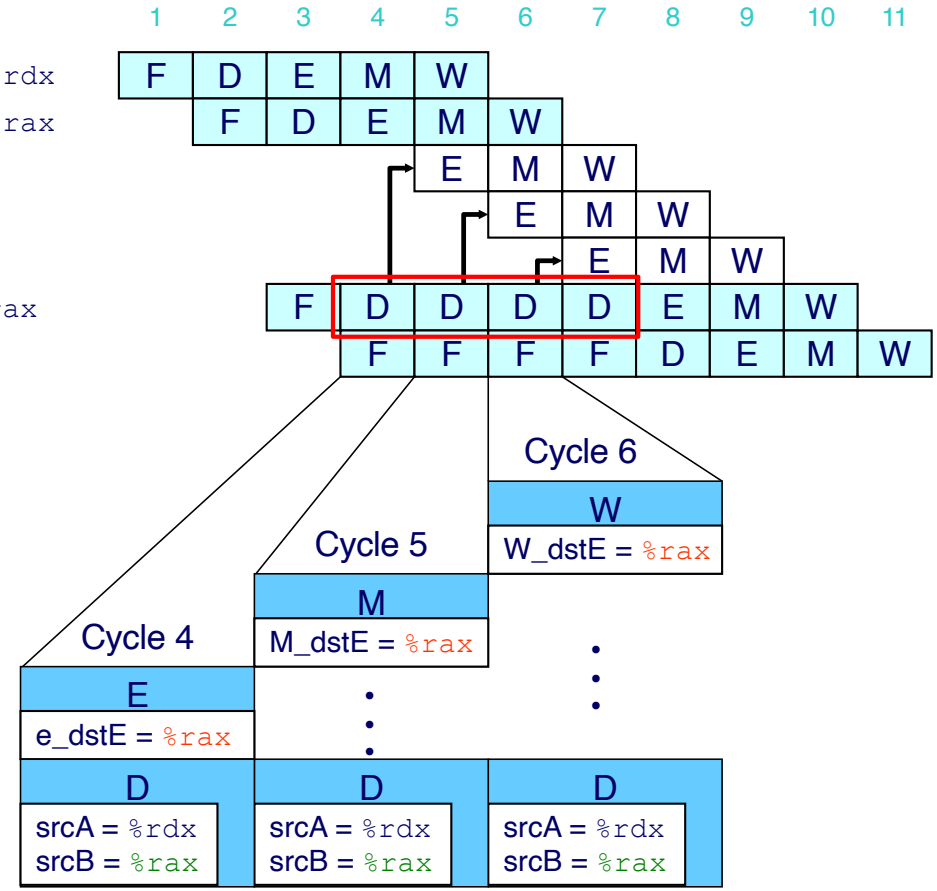
```
# demo-h2.ys
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: nop
0x015: nop
       bubble
0x016: addq %rdx,%rax
0x018: halt
```



使用暂停（stalling）避免数据冒险

- 0nop为例
- C4：译码发现冒险，插入一个气泡
- C5：译码发现冒险，插入一个气泡
- C6：译码发现冒险，插入一个气泡
- stalling*3

```
# demo-h0.ys
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
       bubble
       bubble
       bubble
0x014: addq %rdx,%rax
0x016: halt
```

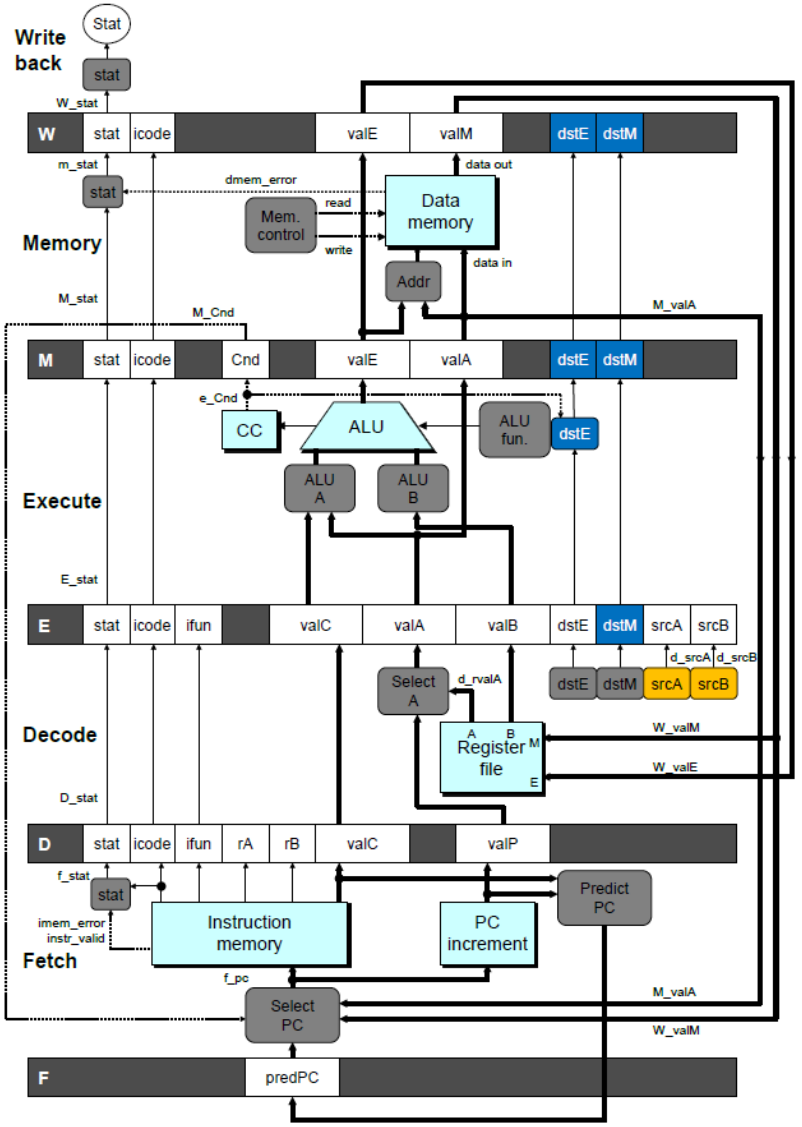


Cycle 8

Write Back	*bubble*
Memory	*bubble*
Execute	`0x014: addq %rdx,%rax`
Decode	`0x016: halt`
Fetch	

暂停条件

- Source Registers
  - srcA and srcB of current instruction in decode stage
- Destination Registers
  - dstE and dstM fields
  - Instructions in execute, memory, and write-back stages
- Special Case
  - Don't stall for register ID 15 (0xF), indicating absence of register operand
  - Or failed cond. move



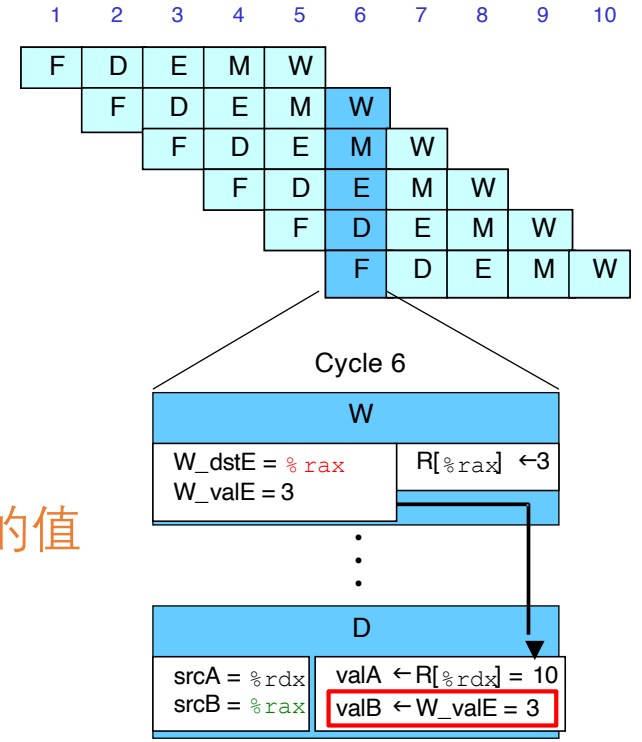
使用数据转发避免数据冒险

- 暂停的弊端：严重降低吞吐量，性能差
- 转发
  - 与其暂停直到写完成，不如将要写的值传到流水线寄存器E作为源操作数
  - 旁路：需要增加额外数据连接和控制逻辑

使用数据转发避免数据冒险

- 2nop为例
- 译码阶段发现
  - %rax是操作数valB的源寄存器
  - 写端口E有对%rax未进行的写

  ✓将给端口E的数据字（W_valE）作为操作数valB的值




```
# demo-h2.ys
    0x000:  irmovq    $10,% rdx
    0x00a:  irmovq     $3,% rax
    0x014:  nop
    0x015:  nop
    0x016:  addq   % rdx ,% rax
    0x018:  halt
```

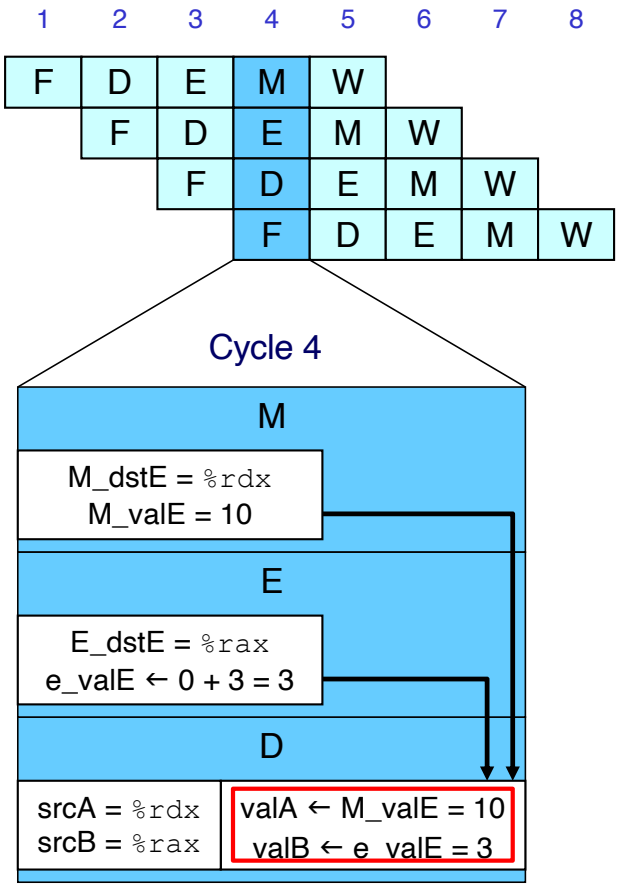
使用数据转发避免数据冒险

- 0nop为例
- C4：译码阶段发现
  - 访存阶段有对%rdx未进行的写
  - 执行阶段在计算%rax的新值

✓valA ← M_valE

✓valB ← e_valE

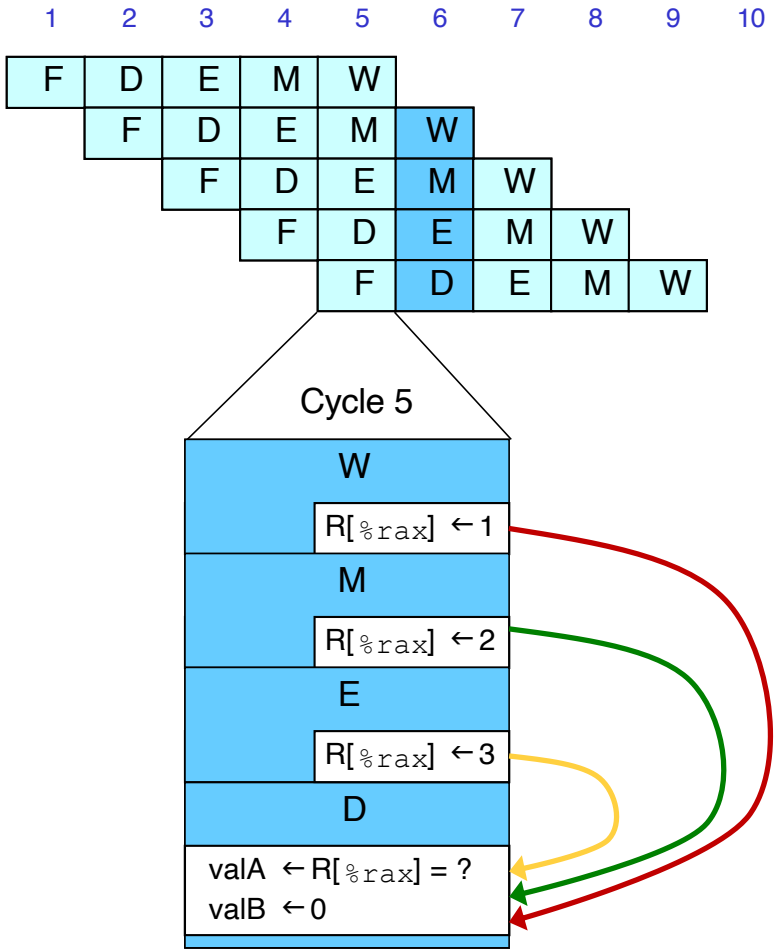
```
# demo-h0.ys
0x000: irmovq $10,%rdx
0x00a: irmovq  $3,%rax
0x014: addq %rdx,%rax
0x016: halt
```



使用数据转发避免数据冒险

- 转发优先级
  - 流水线阶段越早，其中匹配值的优先级越高
    - E>M>W

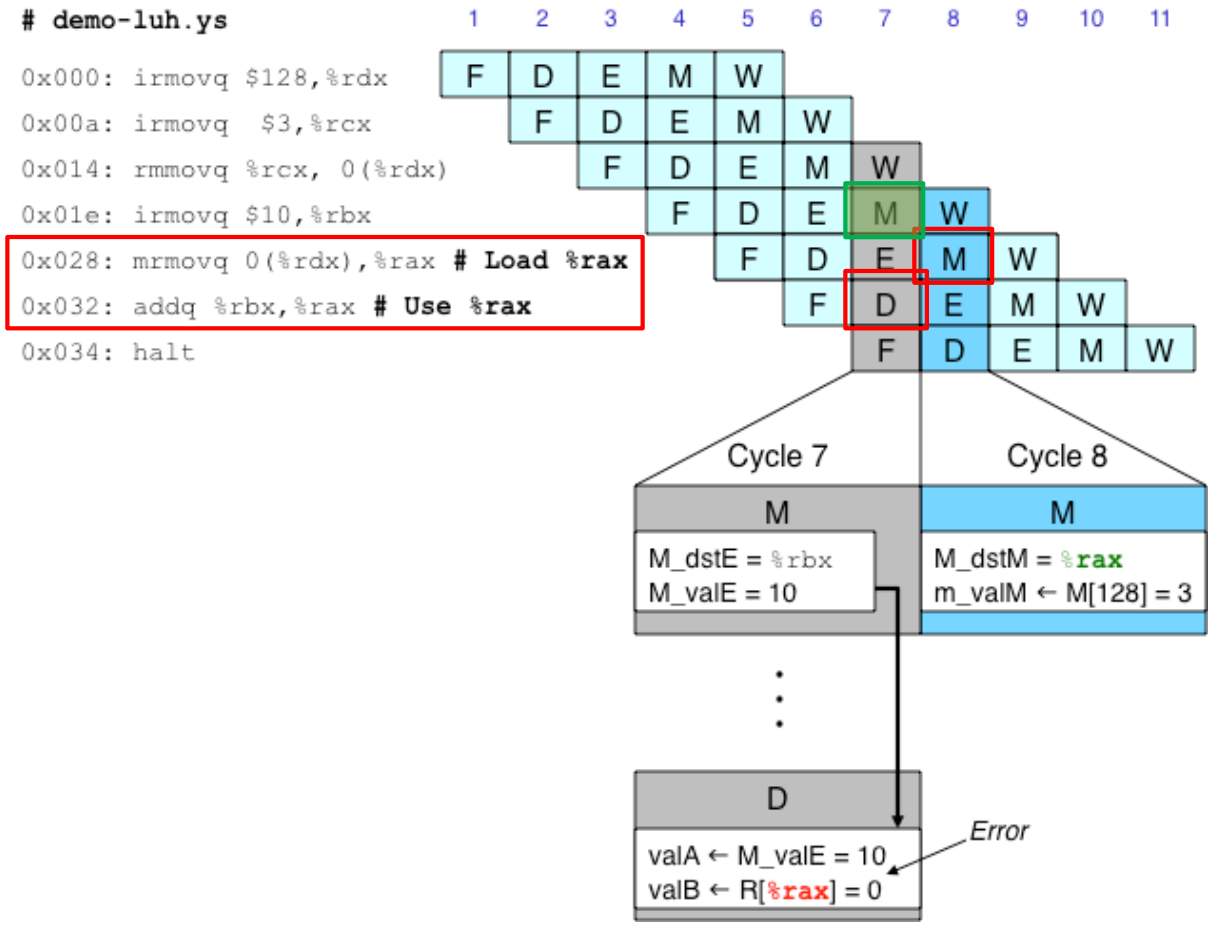
```
# demo-priority.ys
0x000: irmovq $1, %rax
0x00a: irmovq $2, %rax
0x014: irmovq $3, %rax
0x01e: rrmovq %rax, %rdx
0x020: halt
```



使用数据转发避免数据冒险

- 转发的弊端：加载/使用数据冒险
  - addq使用%rax
  - mrmovq加载%rax
  - 转发valB ← m_valM

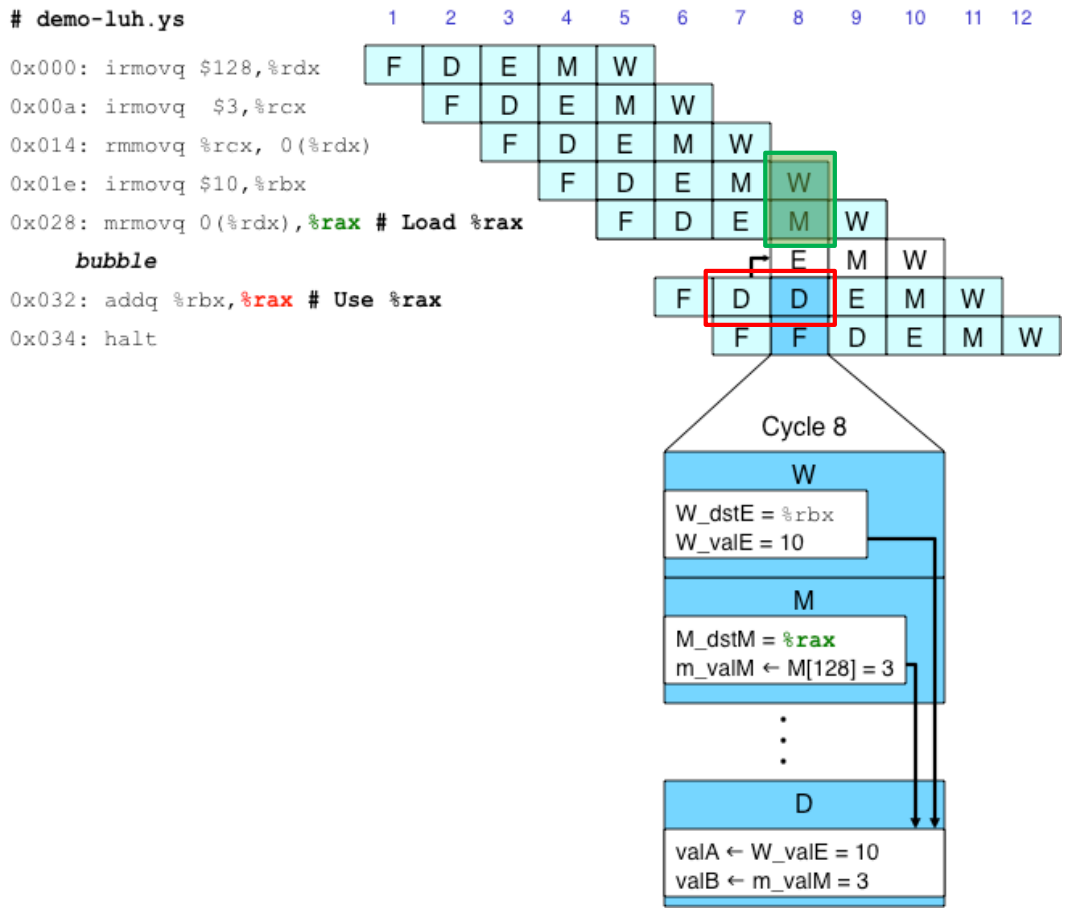
  - addq使用%rax发生在C7
  - mrmovq读内存发生在C8
  - 转发不可能将值送回过去（C7 ← C8）



暂停+转发避免（加载/使用）数据冒险

- C7控制逻辑发现
  - mrmovq执行阶段
  - addq译码阶段
    - 需要从内存读出的结果

  ✓译码阶段的指令（addq）的执行阶段插入一个气泡
- 加载互锁=转发（多数情况）+暂停（少数情况）
  - 实现几乎每个时钟发射一条新指令
  - 解决数据冒险同时保持高吞吐量



控制冒险

- 预测PC：取出当前指令后预测下一条指令位置
  - call/jmp指令：等于指令中的常数字valC
  - **jXX指令**：几个周期后通过该指令的执行阶段才知道
    - 分支预测
      - 选择分支valC（总是选择，60%）
      - 不选择分支valP
  - **ret指令**：几个周期后通过该指令的访存阶段才知道
    - 栈返回地址预测
      - call-ret成对出现，硬件保存call产生的返回地址
  - 其他指令：valP

控制冒险：ret

- 几个周期后通过该指令的访存阶段才知道

```
0x000:     irmovq Stack,%rsp  # Intialize stack pointer
0x00a:     call p             # Procedure call
0x013:     irmovq $5,%rsi     # Return point
0x01d:     halt

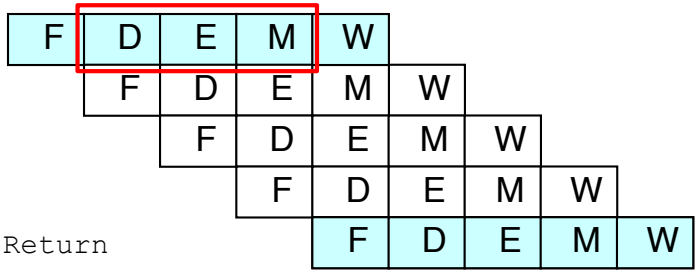
0x020: .pos 0x20
0x020: p:  irmovq $-1,%rdi    # procedure
0x02a:     ret
0x02b:     irmovq $1,%rax     # Should not be executed
0x035:     irmovq $2,%rcx     # Should not be executed
0x03f:     irmovq $3,%rdx     # Should not be executed
0x049:     irmovq $4,%rbx     # Should not be executed

0x100: .pos 0x100
0x100: Stack:                 # Stack: Stack pointer
```

控制冒险： ret

- 几个周期后通过该指令的访存阶段才知道
- 对ret的译码、执行、访存插入气泡（或不断重复取出irmov $1，对其D插入气泡，直到ret访存结束）
- 写回阶段，PC选择逻辑得到返回地址

```
# demo-retb
0x026:     ret
           bubble
           bubble
           bubble
0x013:     irmovq $5,%rsi # Return
```



控制冒险：jXX

- 最简单的预测PC：选择分支
- 几个周期后通过该指令的执行阶段才知道预测正确与否

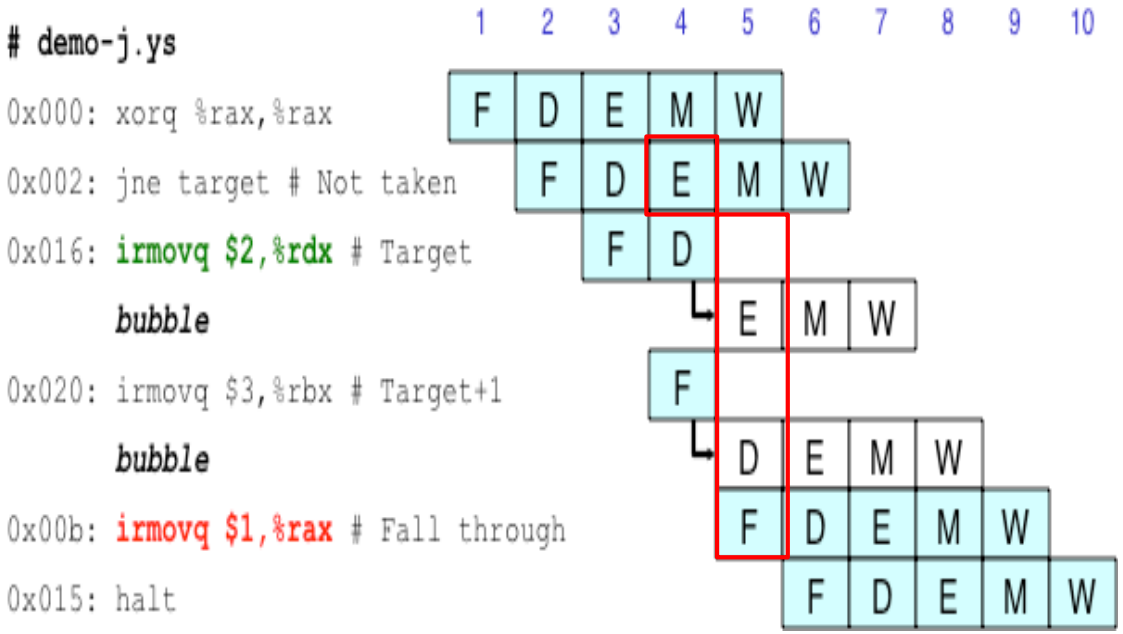
```
0x000:     xorq %rax,%rax
0x002:     jne  t                # Not taken
0x00b:     irmovq $1, %rax       # Fall through
0x015:     halt
0x016: t:  irmovq $2, %rdx       # Target
0x020:     irmovq $3, %rcx       # Target+1
0x02a:     halt
```

控制冒险：jXX

- 最简单的预测PC：选择分支
- 几个周期后通过该指令的执行阶段才知道预测正确与否
- C3
  - F 选择分支t取指
- C4
  - F 选择分支取t+1取指
  - E 分支逻辑发现预测错（但取错的两条指令还没有进入E，没有改变可见的状态）
- C5
  - t指令插入气泡
  - t+1指令插入气泡
  - 取不选择分支的指令

指令清除：把错误路径上的指令在流水线寄存器中的控制信号全部清零，使其变成“空操作”（等效气泡），不产生任何架构状态改变。

```
0x000:     xorq %rax,%rax
0x002:     jne  t                # Not taken
0x00b:     irmovq $1, %rax       # Fall through
0x015:     halt
0x016: t:  irmovq $2, %rdx       # Target
0x020:     irmovq $3, %rcx       # Target+1
0x02a:     halt
```

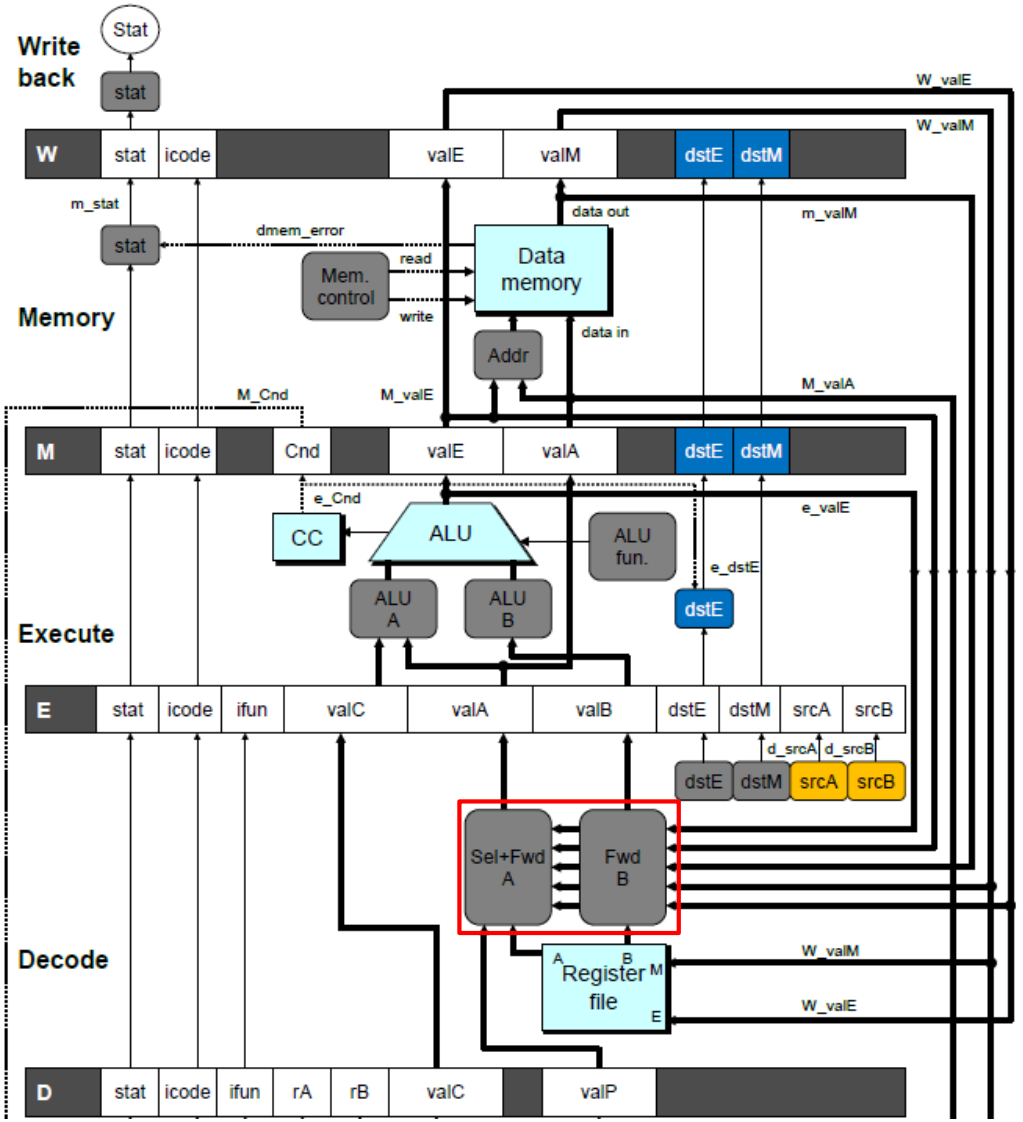


SEQ → SEQ+ → PIPE- → PIPE

- SEQ+ = SEQ + 预测PC前置（电路重定时）
- PIPE- = SEQ+ + 流水线寄存器（FDEMW五阶段流水线化）
- PIPE = PIPE- + 冒险控制（转发，暂停）

PIPE硬件

- 扩展PIPE-支持转发处理数据冒险
- 反馈路径
  - 5个转发源
    - e_valE（ALU产生的值）
    - m_valM（刚从内存读出的值）
    - M_valE（目标为写端口E的值）
    - W_valM（端口M未进行的写）
    - W_valE（端口E未进行的写）
  - 2个转发目的
    - valA
    - valB
    - 控制逻辑



流水线控制逻辑（用HCL描述）

SEQ

```
int srcA = [
  icode in { IRRMOVQ, IRMMOVQ, IOPQ,
IPUSHQ  } : rA;
  icode in { IPOPQ, IRET } : RRSP;
  1 : RNONE; # Don't need register
];
```

PIPE

```
int d_srcA = [
  icode in { IRRMOVQ, IRMMOVQ, IOPQ,
IPUSHQ  } : D_rA;
  icode in { IPOPQ, IRET } : RRSP;
  1 : RNONE; # Don't need register
];
```

- D_*：源值，信号来自流水线寄存器D
- d_*：结果值，表明在译码阶段产生

流水线控制逻辑

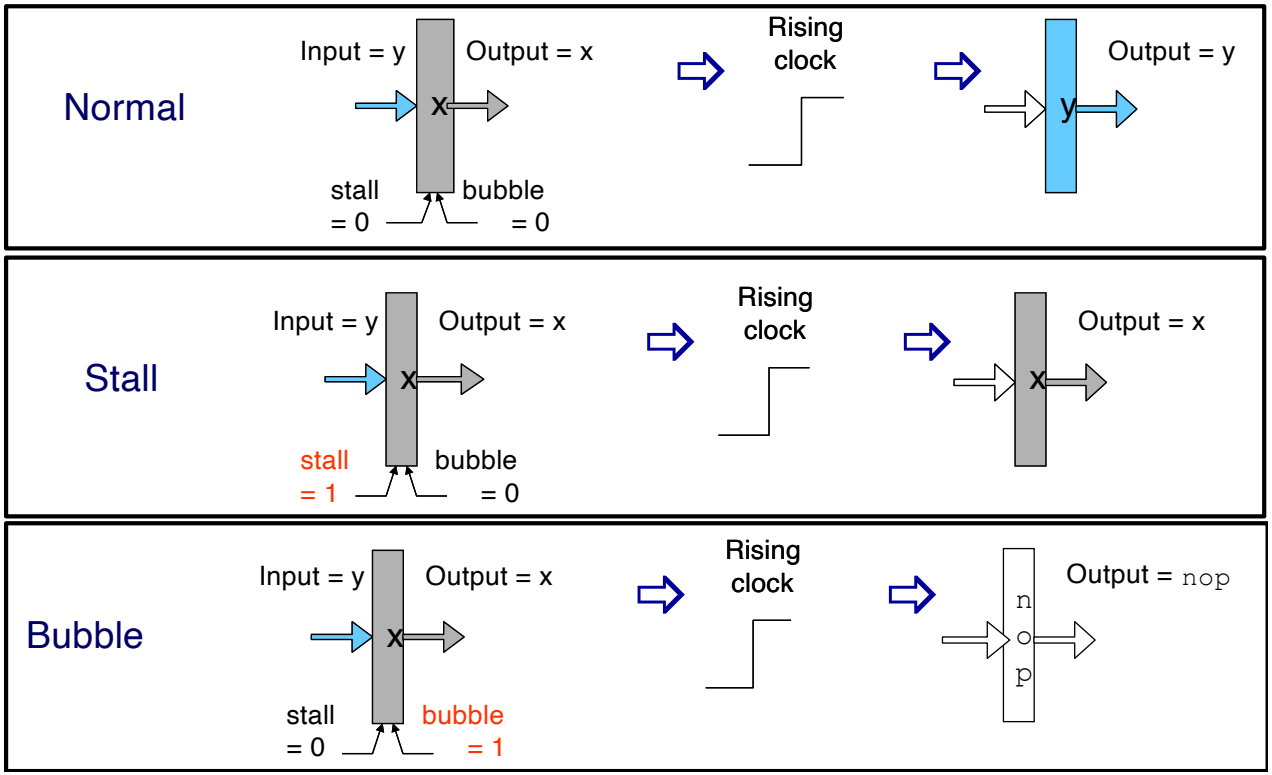
- 预测PC

```
int f_pc = [
  W_icode == IRET : W_valM; # ret指令访存结束进入写回，读返回地址
  M_icode == IJXX && !M_Cnd : M_valA; # 执行结束根据条件码发现预测错误分支，读valP
  1 : F_predPC; # 预测PC
];

int f_predPC = [ # F_predPC中的PC预测值
  f_icode in {IJXX, ICALL} : f_valC;
  1 : f_valP;
];
```

PIPE硬件

- 扩展基本时钟寄存器
  - 流水线寄存器的两个控制输入
    - 暂停
      - 保持旧值
    - 气泡
      - 清空内容（注入空操作）



流水线控制逻辑

	流水线寄存器				
	F	D	E	M	W
加载/使用冒险	暂停	暂停	气泡	正常	正常
处理ret	暂停	气泡	正常	正常	正常
预测错误分支	正常	气泡	气泡	正常	正常

```
bool F_stall = # 流水线寄存器F暂停条件
  # 记载/使用冒险
  E_icode in {IMRMOVQ, IPOPQ} &&
  E_dstE in {d_srcA, d_srcB} ||
  # 处理ret
  IRET in {D_icode, E_icode, M_icode};
```

单选题 1分

设置

在以下数据冒险的叙述中，正确的是：

1.数据冒险是指后面指令用到的数据还未来得及由前面的指令产生

2.在发生数据冒险的指令之间插入空操作指令能避免数据冒险

3. 采用转发（旁路）技术可以解决一部分数据冒险问题

4. 通过编译器调整指令顺序可解决部分数据冒险问题

A 1、2和4

B 1，2和3

C 1，3和4

D 全部

提交

单选题 1分

设置

在以下控制冒险的叙述中，正确的是：

1.条件转移指令执行时可能发生控制冒险

2.在分支指令后加入若干空操作指令能避免控制冒险

3. 采用转发（旁路）技术可以解决一部分控制冒险问题

4. 通过编译器调整指令顺序可解决部分控制冒险问题

A 1、2和4

B 1，2和3

C 1，2，3和4

D 全部

提交

控制器

根据产生操作控制信号的方式不同，
控制器可分为硬布线控制器和微程序控制器

硬布线控制器

- 硬布线控制器（硬连接控制器、组合逻辑控制器）
- 控制信号由组合逻辑电路产生
- 适合于简单或规整的RISC指令系统
- 优点：速度取决于电路延迟、速度快
- 缺点：它是一个多输入/多输出的巨大逻辑网络。对于复杂指令系统来说，逻辑线路固定、结构庞杂、实现困难；修改、维护不易；灵活性差。

简化控制器设计的一个方法：微程序设计

微程序控制器

- 英国剑桥大学的M·V·Wilkes于1951年在曼彻斯特大学计算机会议上首先提出微程序的概念和原理。1964年，IBM公司在IBM 360系列机上成功地采用了微程序设计技术。
- 微程序控制器的基本思想
  - 每条指令的执行过程对应一个微程序
  - 每个微程序由若干条微指令构成
  - 所有微程序放在ROM（Control Storage，控制存储器）中
  - 执行某条指令时，取出CS中对应微程序中的各条微指令执行，产生指令运行所需的操作控制信号，使相应部件执行规定的操作
- 微程序设计的特点
  - 具有规整性、可维性和灵活性，但速度慢
  - 通常应用于CISC计算机

微程序控制器

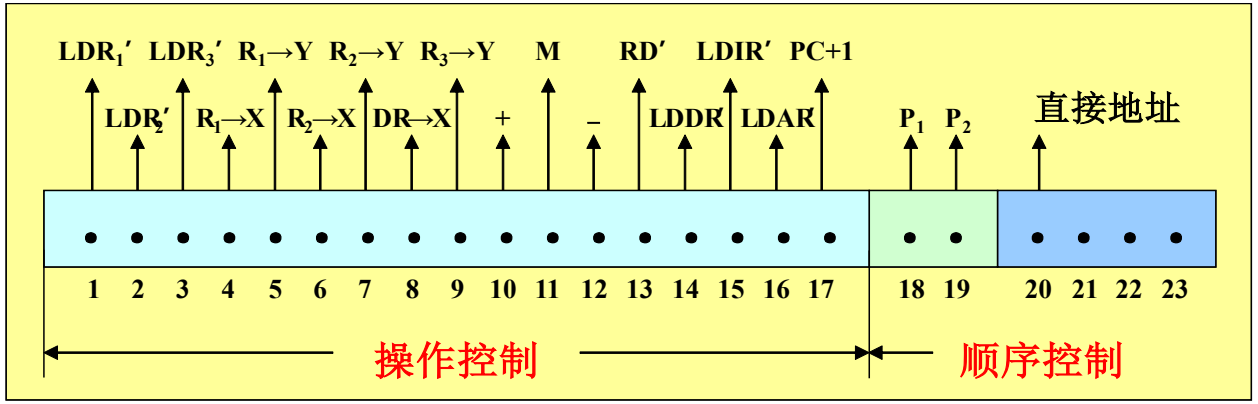
- 微程序：一系列微指令的有序集合，一段微程序对应一条机器指令。
- 微指令：机器的一个CPU周期中，一组实现一定操作功能的微命令的组合。
  - 是微命令的组合，存储在控制器中的控制存储器中
  - 一条微指令至少包含两部分信息：
    - 操作控制字段，又称微操作码字段，用以产生某一步操作所需的各个微操作控制信号。
    - 顺序控制字段，又称微地址码字段，用以控制产生下一条要执行的微指令地址。

- 微命令：控制部件通过控制线向执行部件发出的各种控制命令，是构成控制序列的最小单位。
  - 微命令是控制计算机各部件完成某个基本微操作的命令。
- 微操作：执行部件接受微命令后所进行的操作，是执行部件中最基本的操作。
- 微命令和微操作是一一对应的。 微命令是微操作的控制信号，微操作是微命令的操作过程。

微程序控制器

- 由于数据通路的结构关系，微操作可分为相容性和相斥性两种。
  - 相容性微操作：能够同时或在同一个CPU周期内并行执行的微操作。
  - 相斥性微操作：是指不能同时或不能在同一个CPU周期内并行执行的微操作。

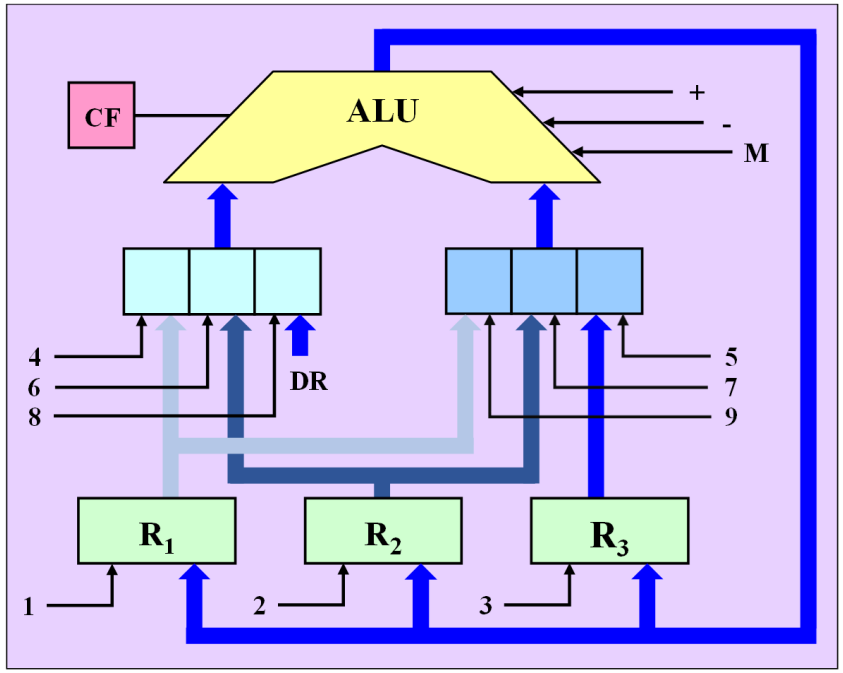
微程序控制器



- 微指令字长23位
  - 操作控制字段17位
    - 每位表示一个微命令
      - 1：发出微命令
      - 0：不发出微命令
  - 顺序控制6位
    - 判别测试标志2位
      - 00：不进行测试，直接按20-23位取下一条微指令
      - 01/10/11：根据P1/P2测试，修改20-23的若干位后再取下一条微指令
    - 下一条微指令地址4位

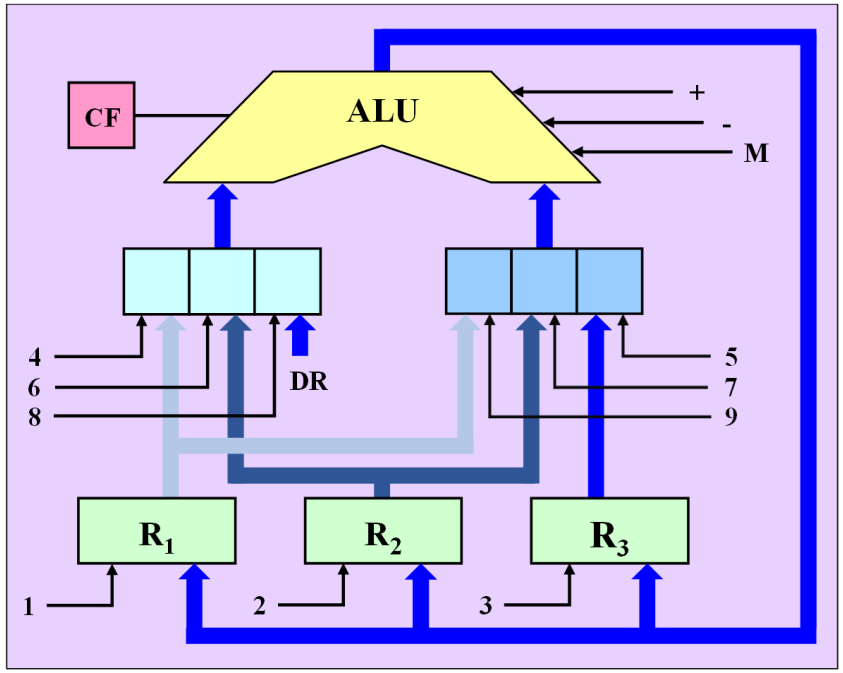
微程序控制器

- 图中R1~R3的内容都可以通过多路开关从ALU的X或Y输入端送至ALU，而ALU的输出可以送往任何一个寄存器或同时送往R1~R3。
- 多路开关的每个控制门仅是一个常闭的开关，其一个输入端代表来自寄存器的信息，而另一个输入端则作为操作控制端。
- 一旦两个输入端都有输入信号时，才产生一个输出信号，从而在控制线能起作用的一个时间宽度中控制信息在部件中流动。



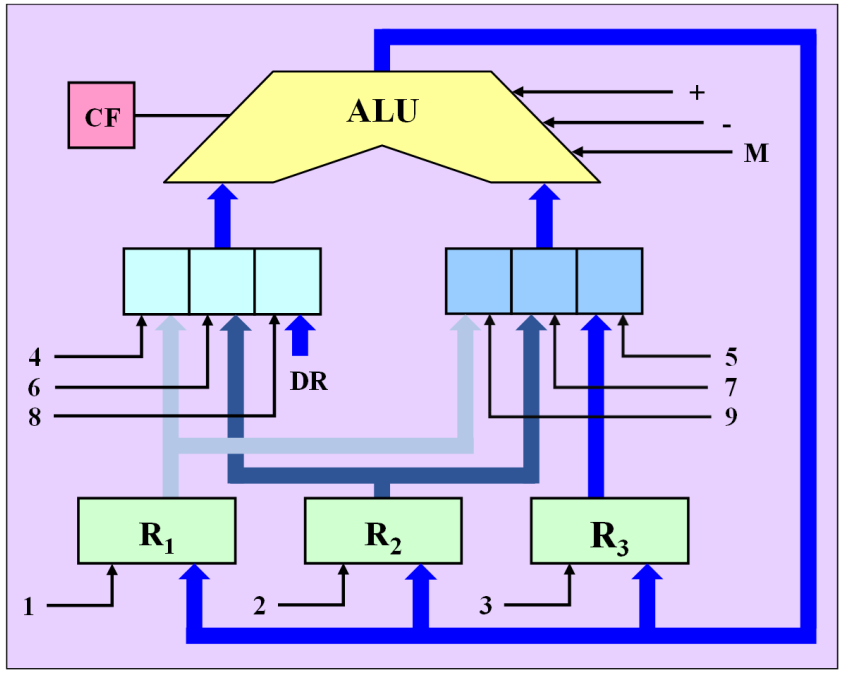
微程序控制器

- 图中每个开关门由控制器中相应的微命令来控制。
- 例如，开关门4由控制器中编号为4的微命令控制。三个寄存器R1、R2和R3的时钟输入端1、2、3也需要加以控制，在ALU运算完毕而输出公共总线上电平稳定时，将结果打入到某一寄存器。
- ALU有+、-和M（传送）三种操作，CF为进位标志。



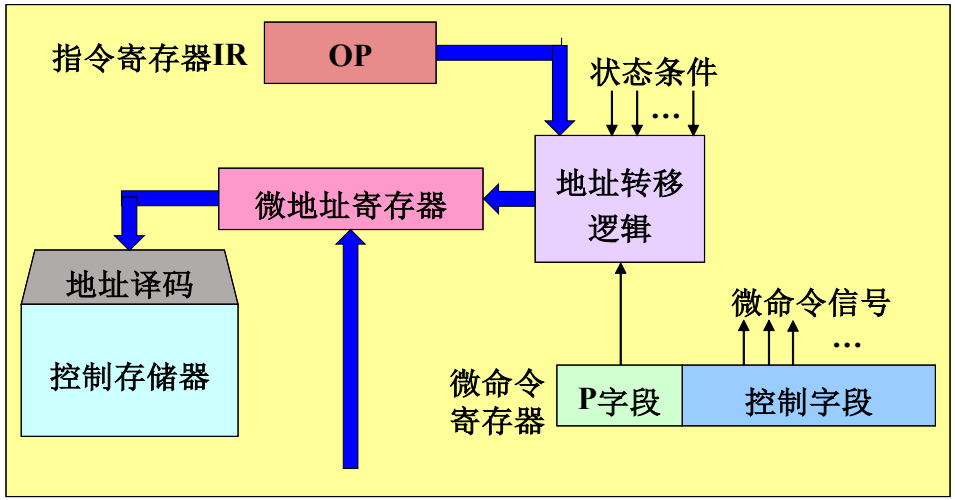
微程序控制器

- ALU的操作+、-和M在同一个CPU周期中只能选择一个，不能并行，即 相斥的；同样，4、6、8三个微操作是相斥的；5、7、9三个微操作也是相斥的。
- 操作1、2、3是可以同时进行的，因此是相容性微操作。
- 此外，ALU的X输入端微操作4、6、8中的一个分别与Y输入端的5、7、9的任意一个也是相容的。



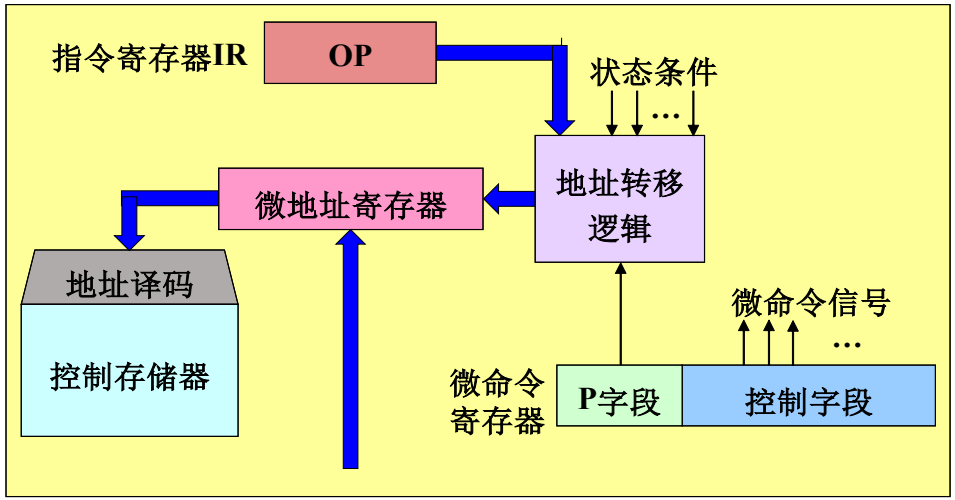
微程序控制器

- 微程序控制器主要由三大部分组成
  - 控制存储器
  - 微指令寄存器，又包括
    - 微地址寄存器
    - 微命令寄存器
  - 地址转移逻辑



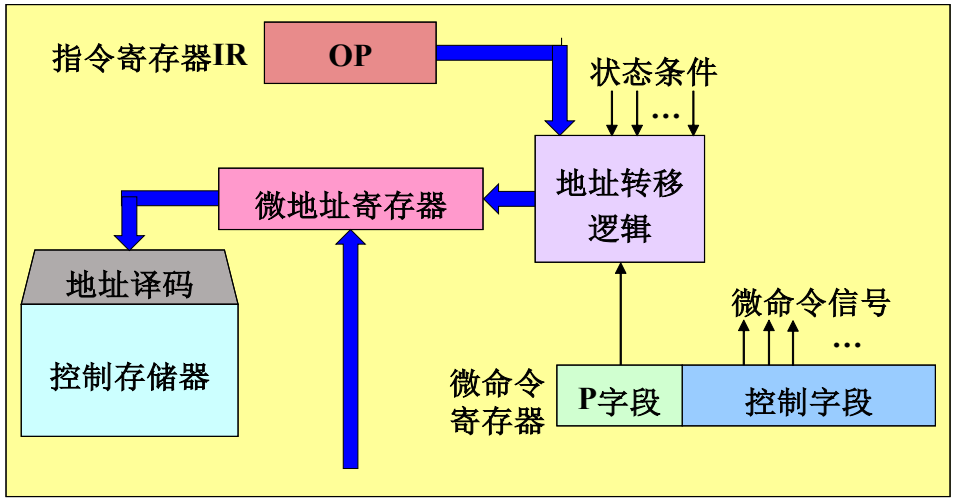
微程序控制器

- 控制存储器
  - 微程序控制器的核心部件，用来存放实现全部指令系统的微程序，是一种只读型存储器。其性能（包括容量、速度、可靠性等）与计算机的性能密切相关。
  - 工作周期就是微指令周期，即读出一条微指令并执行微指令的时间总和。



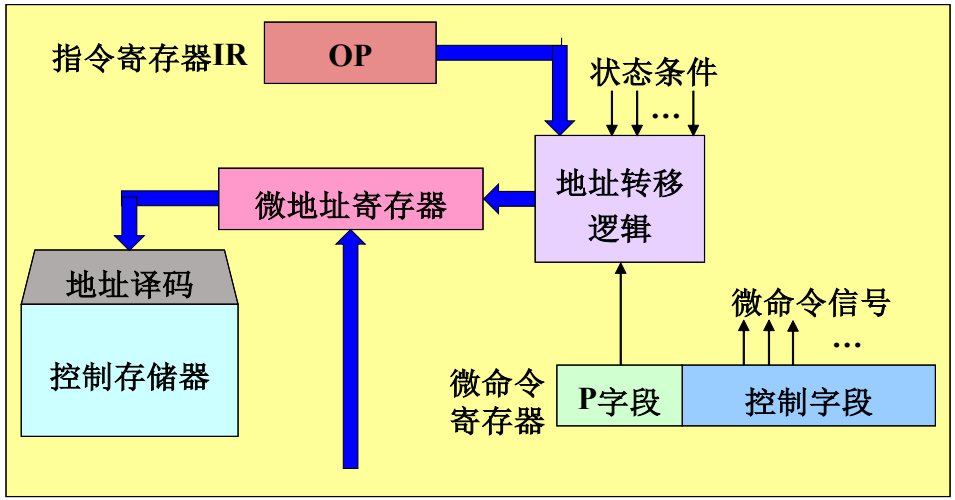
微程序控制器

- 微指令寄存器
  - 用来存放从控制存储器取出的正在执行的微指令，其位数同微指令字长相等，分为微地址寄存器和微命令寄存器。
  - 微地址寄存器决定将要访问的下一条微指令的地址。
  - 微命令寄存器保存一条微指令的操作控制字段和判别测试字段的信息。



微程序控制器

- 地址转移逻辑
  - 为下一步从控制存储器中读取微指令作准备。
  - 微程序顺序执行时，下一条微指令的地址直接由微地址寄存器给出。
  - 微程序出现分支时，通过判别测试字段P和执行部件的“状态条件”反馈信息，修改微地址寄存器的内容，并按修改的内容去读下一条微指令。

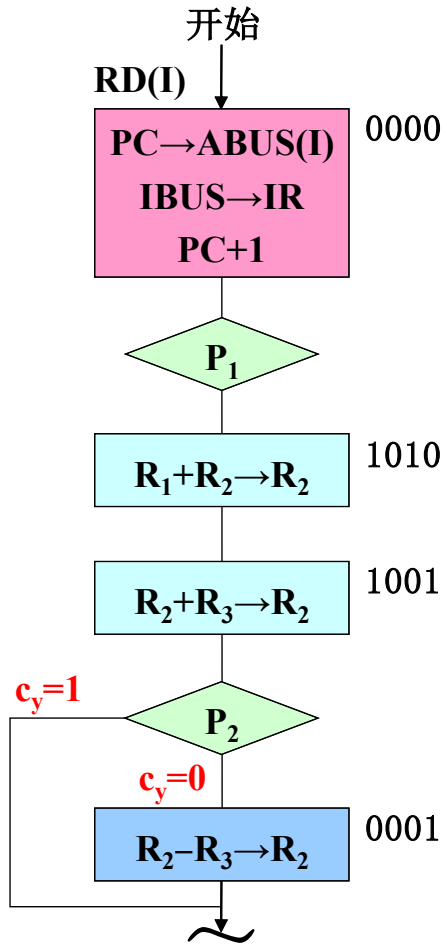


微程序控制器

- 工作过程
  1. 执行取指令的公共操作
     - 取指令的公共操作通常由一段取指微程序来完成，在机器开始运行时，自动将取指微程序的入口微地址送微地址寄存器，并从控制存储器中读出相应的微指令送入微指令寄存器。
     - 微指令的操作控制字段产生相关微命令，来控制实现取机器指令的公共操作。
     - 取指微程序的入口地址一般为控制存储器的0号单元，当取指微程序执行完后，从主存中取出的机器指令就已存入IR中了。
  2. 由机器指令的操作码字段通过微地址形成部件产生出该机器指令所对应的微程序的入口地址，并送入微地址寄存器。
  3. 从控制存储器中逐条取出对应的微指令并执行，每条微指令都能自动产生下一条微指令的地址。
  4. 一条机器指令对应的微程序的最后一条微指令执行完毕后，其下一条微指令地址又回到取指微程序的入口地址，从而继续第1步，以完成取下条机器指令的公共操作。
  - 以上是一条机器指令的执行过程，如此周而复始，直到整个程序的所有机器指令执行完毕。

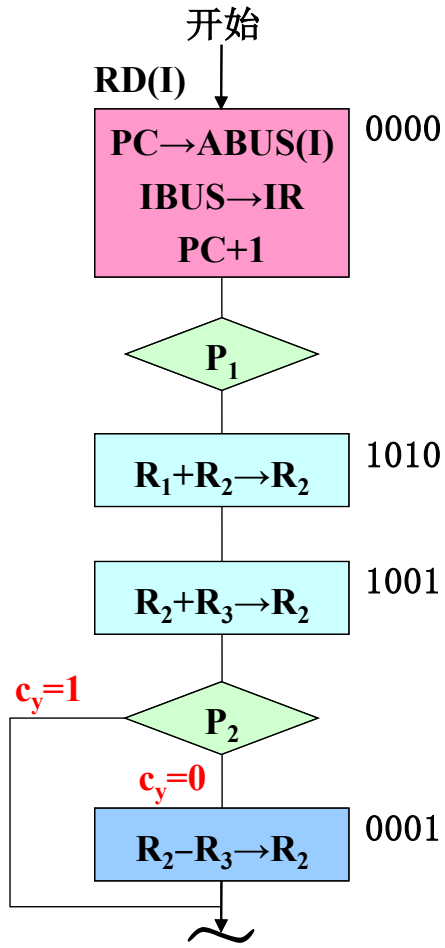
微程序控制器

- 一条机器指令对应一个微程序，微程序的总和实现整个指令系统。
- 举例：BCD码的十进制加法。
  - 52+40=0101 0010+0100 0000=1001 0010=92
  - 52+61=0101 0010+0110 0001
    =1011 0011(B3)+0110(6) 0000=1 0001 0011=113
  - 如果和大于9，加6实现进位修正
- 假定指令存放在指存中，数据a、b及常数6存放在R1、R2、R3中，完成十进制加法的微程序流程如图。



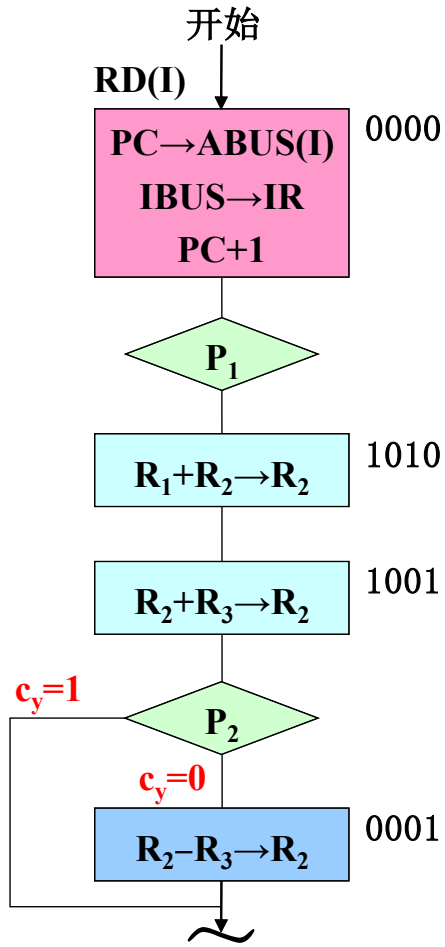
微程序控制器

- 执行周期先进行a+b+6运算，然后判断结果有无进位
  - 进位标志c_y=1，不减6
  - 进位标志c_y=0，减去6
  - 得到正确结果
- 十进制加法微程序流程图由四条微指令组成，每一条微指令用一个长方框表示，微指令的地址用数字标识在长方框的右上角。
- 菱形符号代表判别测试，其动作依附于上一条微指令。



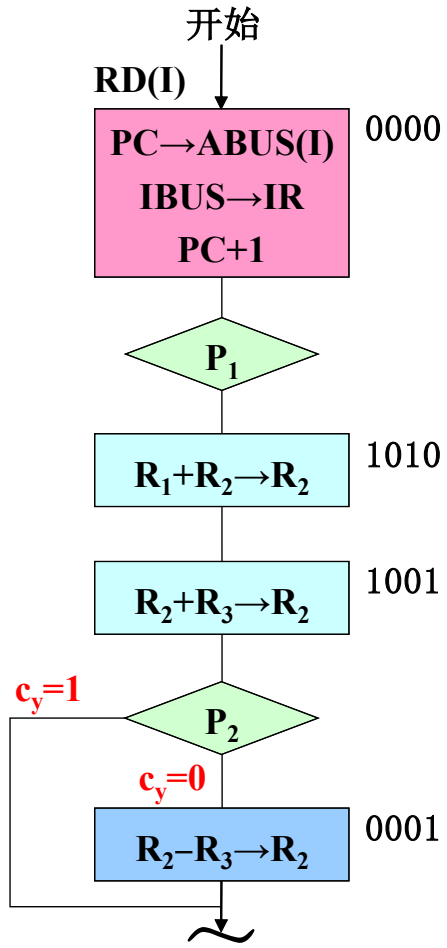
微程序控制器

- 第一条微指令：取指微指令
  - 专门用来取机器指令的微指令，完成三项操作。
    - 从内存中取出一条机器指令，并将指令放到指令寄存器IR中。
    - 程序计数器PC+1，做好取下一条机器指令的准备。
    - 对机器指令的操作码用P1进行判别测试，然后修改微地址寄存器内容，给出下一条微指令的地址。
- 第二条微指令：完成a+b运算



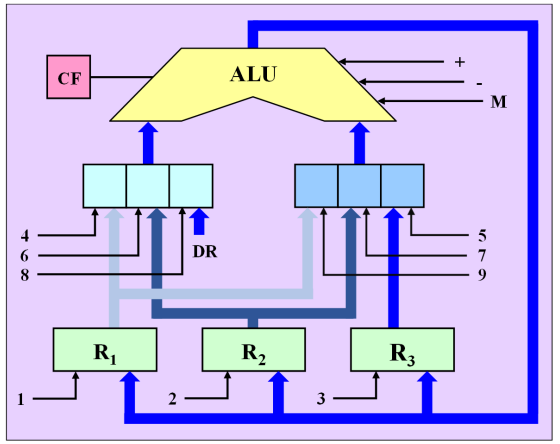
微程序控制器

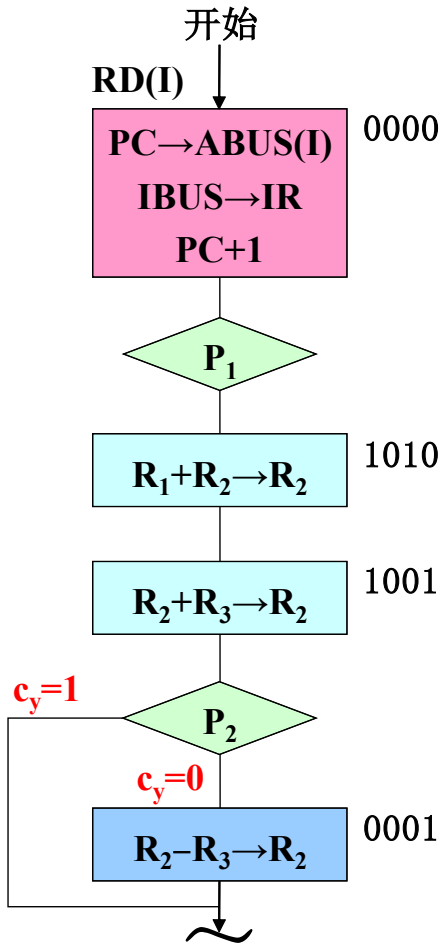
- 第三条微指令：完成a+b+6运算，进行判别测试P2
  - 根据测试结果，微程序或者转向公共操作，或者转向第四条微指令。
  - 当微程序转向公共操作（~）时，如果没有外围设备请求服务，则转向取下一条机器指令。
  - 第三条微指令和第四条微指令的下一个微地址均指向第一条微指令，即“取指”微指令。



微程序控制器

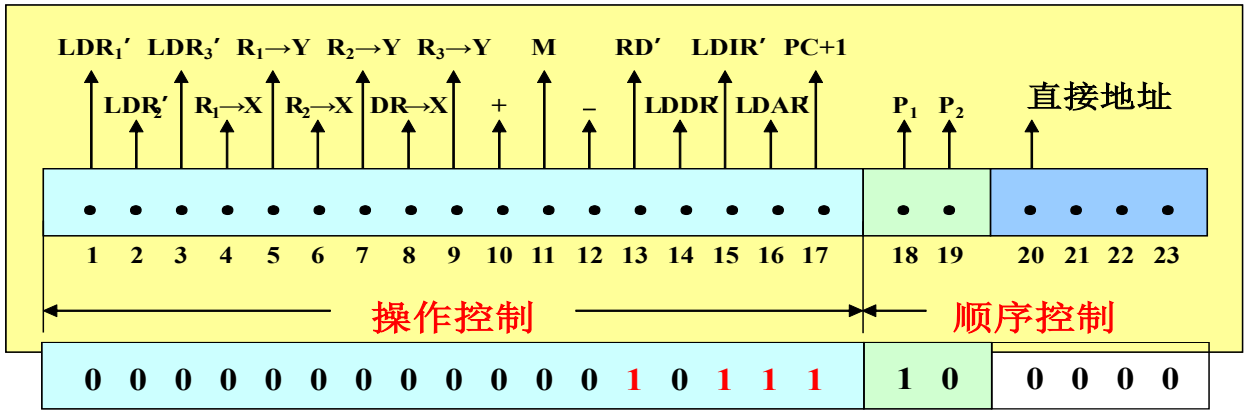
- 假设已经按微指令流程图编好了微程序，并已事先存放到控制存储器中，以下图中的运算器作为执行部件。
- 机器启动时，只要给出控制存储器的首地址，就可以调出所需要的微程序。
- 第一条微指令的地址为0000，经地址译码，控制存储器选中所对应的“取指”微指令，并将其读到微指令寄存器中。





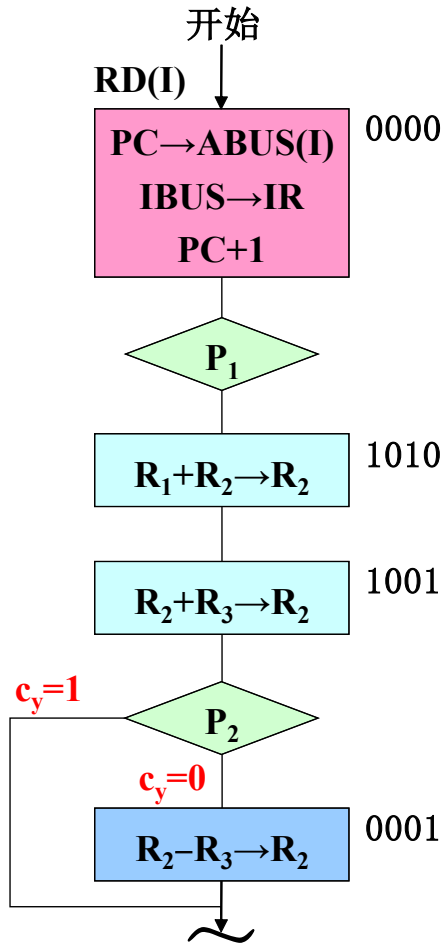
微程序控制器

- 第一条微指令：取指微指令



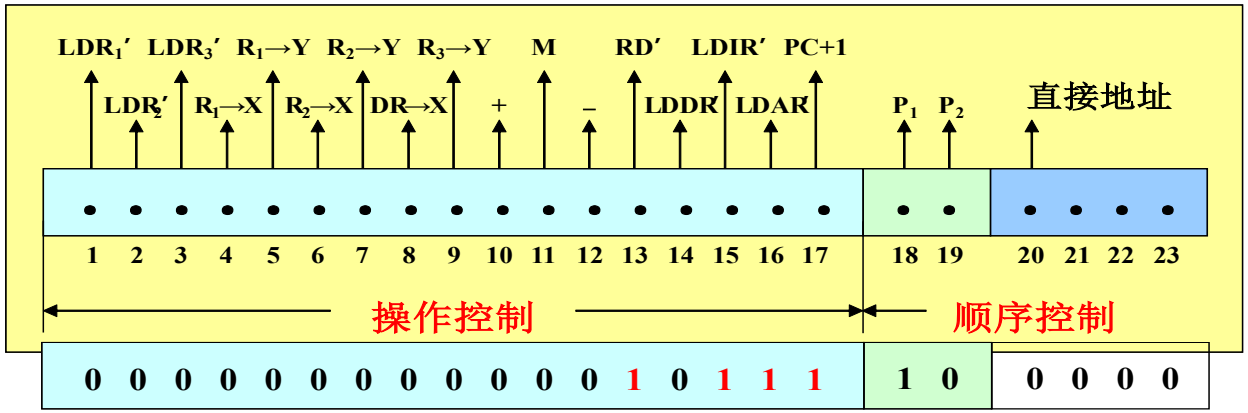
操作控制字段有4个微命令：

- 第13位：指存读命令RD(I)，从指存单元取出“十进制加法”指令放到指令总线IBUS上；
- 第16位：LDAR‘，PC→ABUS(I)，PC内容送到指存地址总线ABUS(I)；
- 第15位：LDIR’，将IBUS上的“十进制加法”指令打入到指令寄存器IR；
- 第17位：PC+1，使程序计数器加1，做好取下一条机器指令的准备。



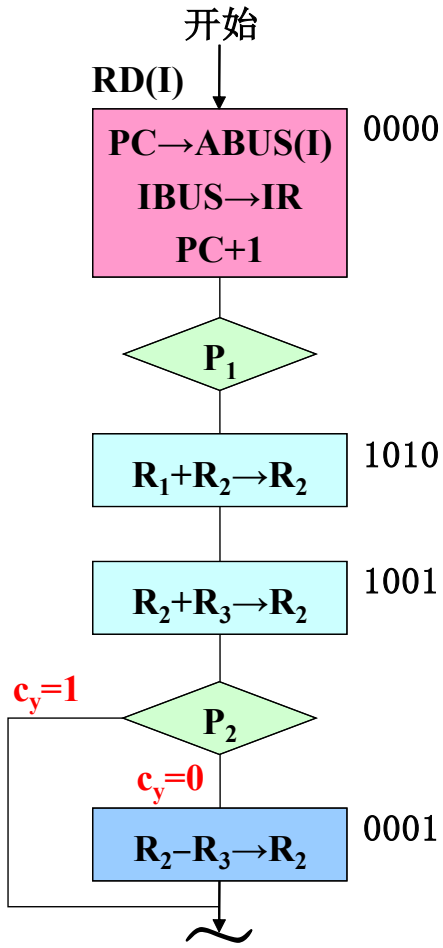
微程序控制器

- 第一条微指令：取指微指令

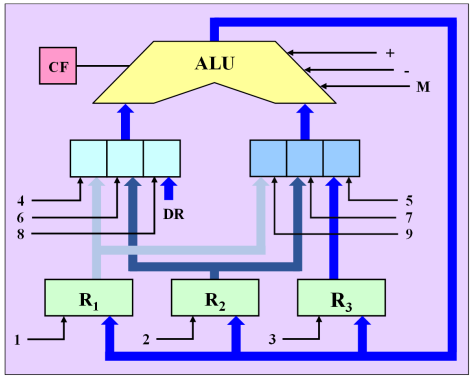


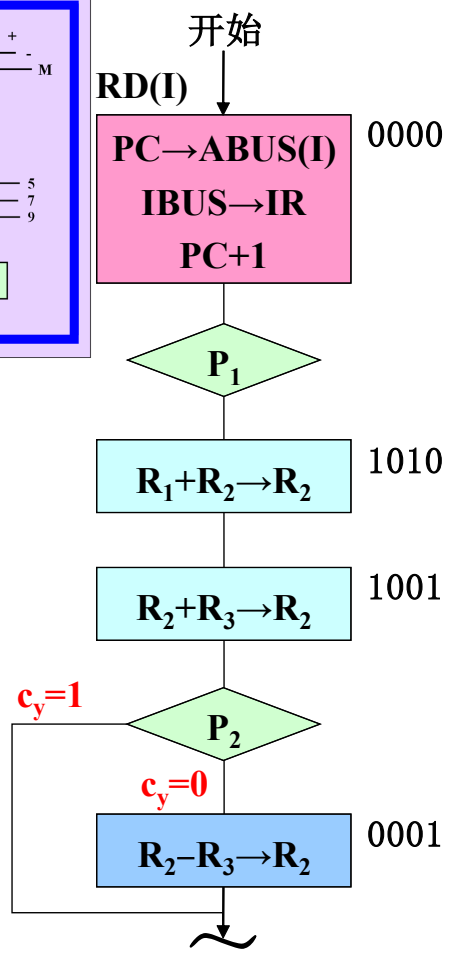
顺序控制字段

- 指明下一条微指令的地址是0000
- 但由于判别字段第18位为1，表明是P1测试，其测试条件是指令寄存器的操作码字段，即用OP字段作为形成下一条微指令的地址，假定OP字段为1010。

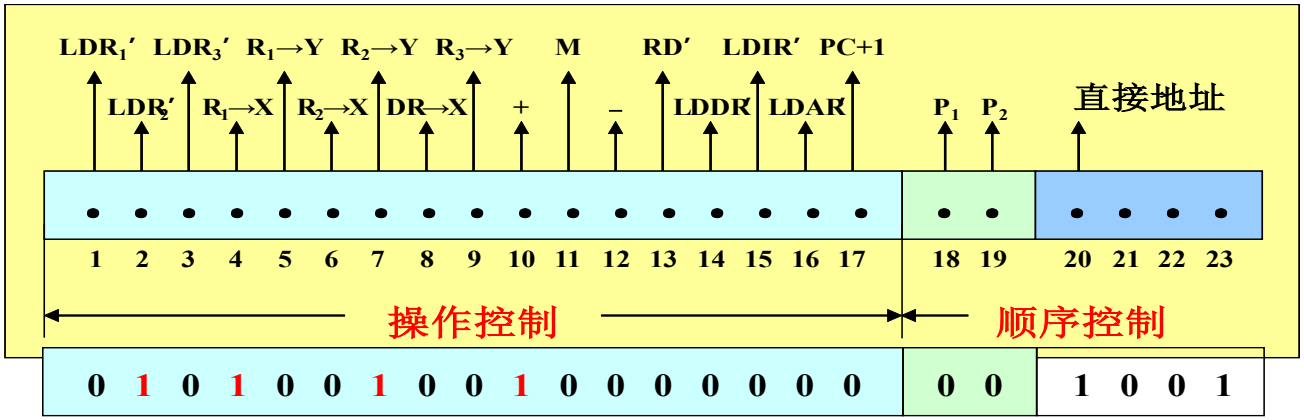


微程序控制器



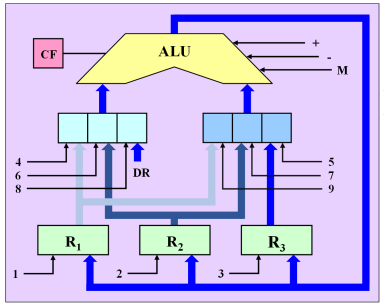


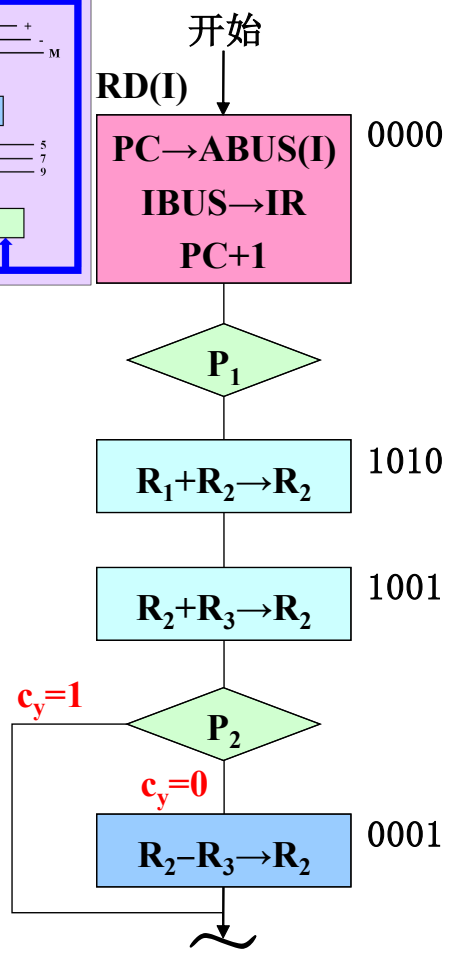
- 第二条微指令：完成a+b运算



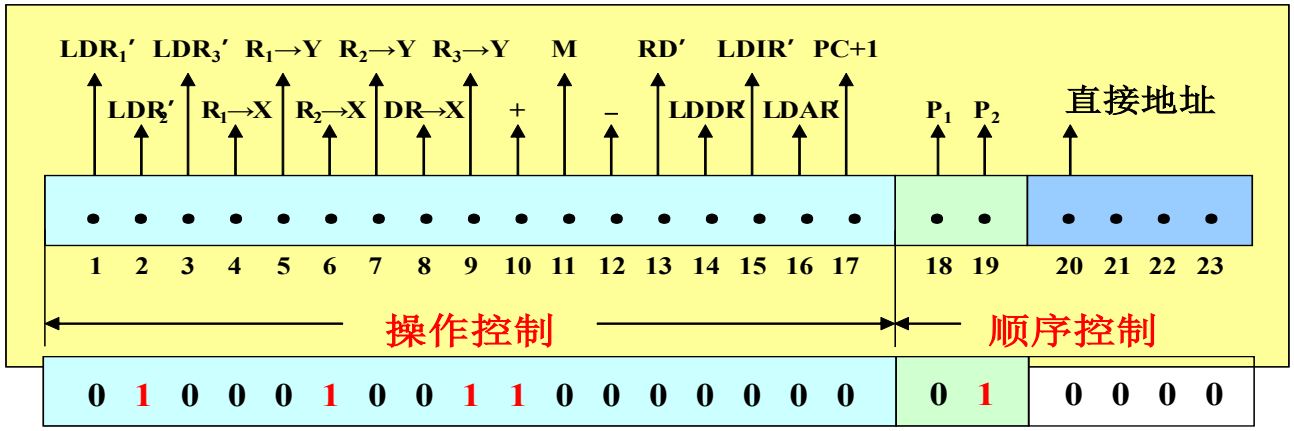
- 按照1010这个微地址读出第二条微指令
- 操作控制字段发出4个微命令
  - R1→X（4），R2→Y（7），+（10），LDR2'（2），运算器完成R1+R2→R2操作。
- 顺序控制字段中，判别测试字段P为0，因此下一条微指令的地址为1001

微程序控制器



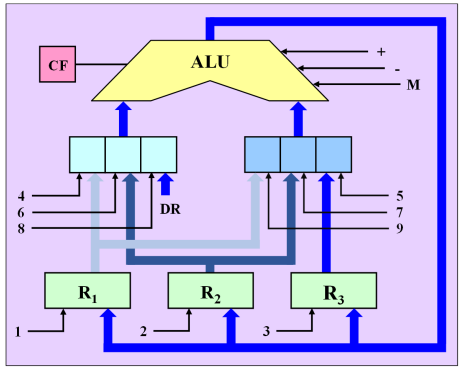


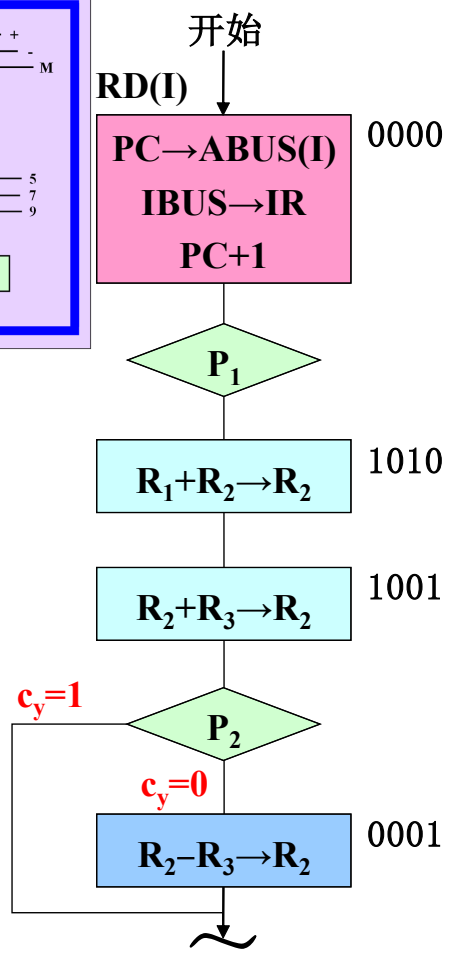
- 第三条微指令：完成a+b+6运算，进行判别测试P2



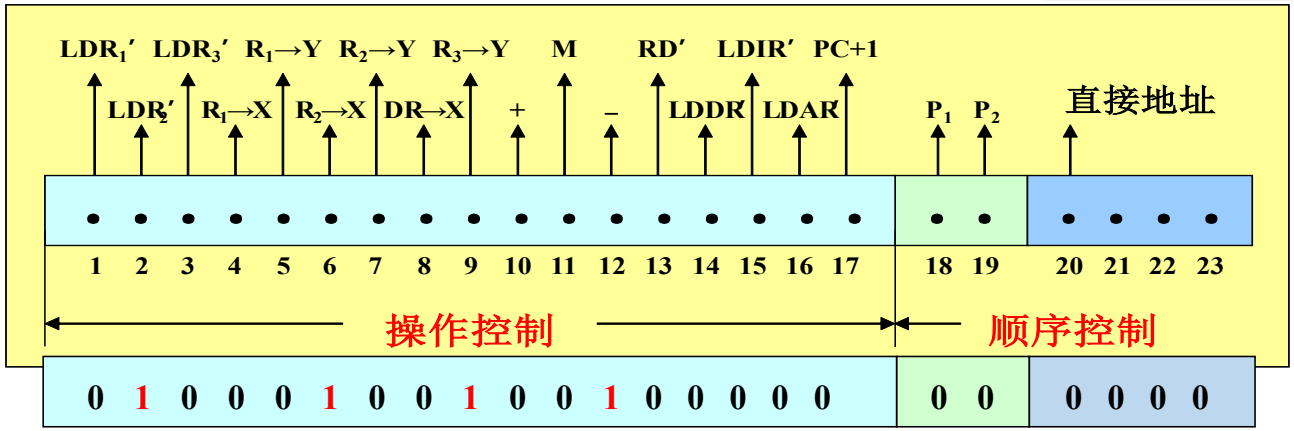
- 按照1001这个微地址读出第三条微指令
- 操作控制字段发出4个微命令
  - R2→X（6），R3→Y（9），+（10），LDR2’（2），运算器完成R2+R3→R2操作
- 顺序控制字段中，判别测试字段P2为1（19），表明进行P2测试，测试的“状态条件”为进位标志Cy，假定用Cy的状态来修改微地址寄存器的最后一位：Cy=0时，下一条微指令的地址为0001；Cy=1时，下一条微指令的地址为0000。
  - 假定Cy=0，则要执行的下一条微指令地址为0001。

微程序控制器





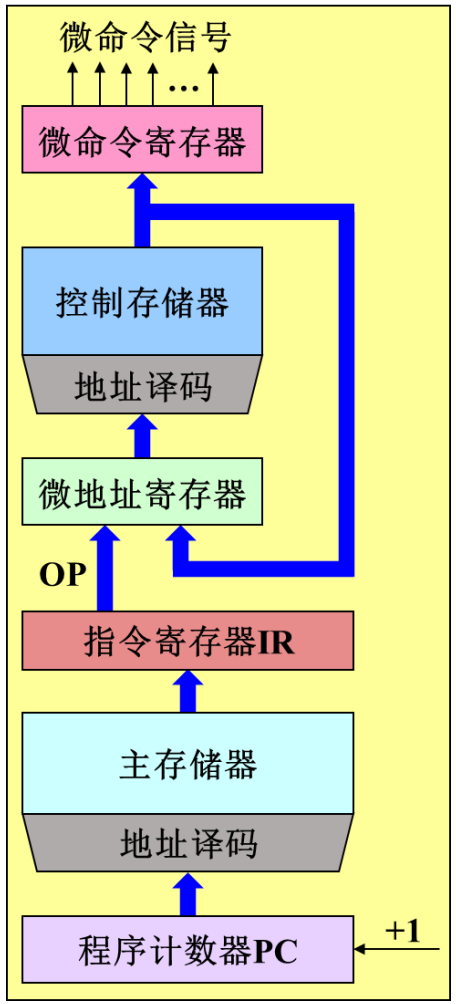
- 第四条微指令：完成a+b+6-6运算



- 按照0001这个微地址读出第四条微指令。
- 操作控制字段发出4个微命令
  - R2→X（6），R3→Y（9），-（12），LDR2’（2），运算器完成R2-R3→R2操作。
- 顺序控制字段中，判别测试字段P为0，因此下一条微指令的地址为0000，按该地址取出的微指令是“取指”微指令。
  - 如果第三条微指令进行测试时Cy=1，则微地址仍保持为0000，将不执行第四条微指令而直接由第三条微指令转向公共操作“取指”。

微程序控制器

- 机器指令与微指令的关系
  - 一条机器指令对应一个微程序
  - 一条机器指令所完成的操作划分成若干条微指令来完成，由微指令进行解释和执行
- 任何一条机器指令的取指操作是相同的，因此将取指操作统一编写成一个微程序
  - 若指令系统有n种机器指令，则控制存储器至少有n+1个微程序（1个公共的取指微程序）



微程序控制器

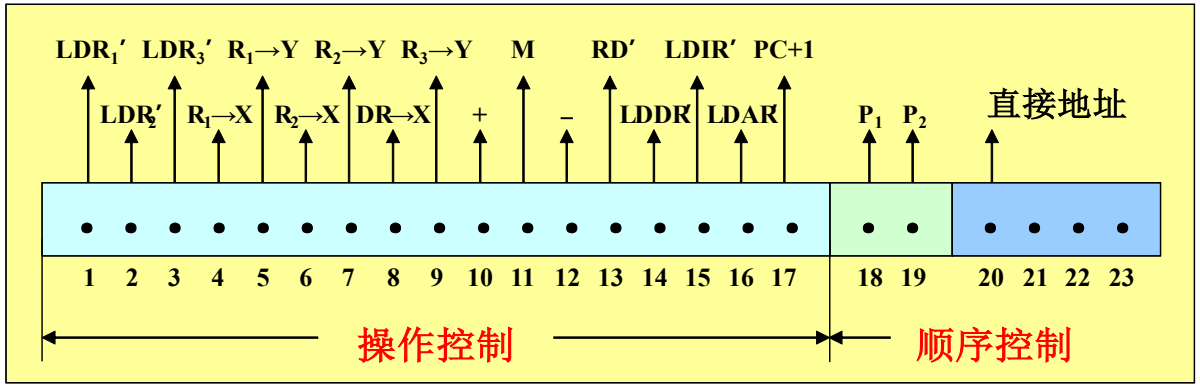
- 微指令格式
  - 微指令的执行方法是决定微指令格式的主要因素，可以分成两类
    - 水平型微指令
      - 一次能定义并并行执行多个并行操作的微命令
    - 垂直型微指令
      - 类似于机器指令的结构，有操作码，一次只能执行1或2个微命令

微程序控制器

- 水平型微指令 VS 垂直型微指令
  - 水平型微指令并行操作能力强，效率高，灵活性强；垂直型微指令一般不强调并行控制。
  - 水平型微指令执行一条指令的时间短；垂直型微指令执行时间长。
  - 水平型微指令的指令字较长而微程序较短；垂直型微指令则相反。
  - 水平型微指令用户难以掌握；而垂直型微指令与指令比较相似，相对来说，比较容易掌握。

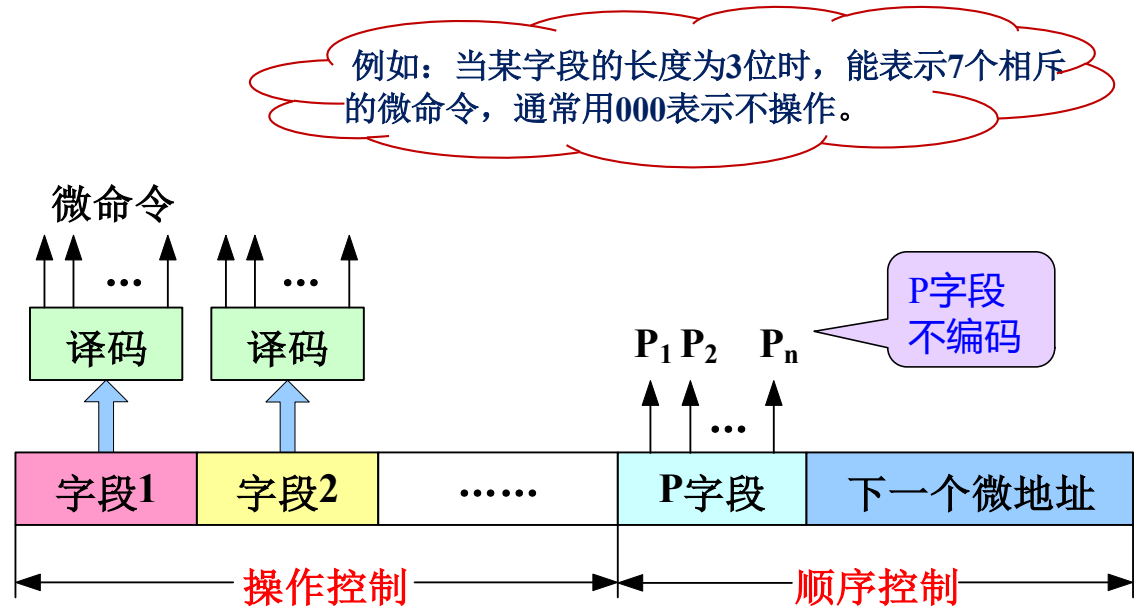
微程序控制器

- 水平型微指令的微命令编码
  - 对微指令中的操作控制字段采用的表示方法
  1. 直接表示法
     - 控制字段中的每一位表示一个微命令
     - 优点：简单直观，输出直接用于控制
     - 缺点：控制字段长，如三、四百位
     - 控制存储器容量要大



微程序控制器

- 水平型微指令的微命令编码
  - 对微指令中的操作控制字段采用的表示方法
  - 2. 编码表示法（较普遍）
    - a. 把相斥性微命令编为一组
    - b. 对微命令进行编码，留出一个代码表示本段不发出微命令；
    - c. 增设微命令译码器
    - 优点：可以避免互斥，使指令字大大缩短
    - 缺点：增加了译码电路，使微程序的执行速度减慢



微程序控制器

- 水平型微指令的微命令编码
  - 对微指令中的操作控制字段采用的表示方法

  3. 混合表示法
     - 把直接表示法与字段编码法混合使用，以便能综合考虑微指令字长、灵活性、执行速度等方面的要求
     - 一个字段的某些编码不能独立地定义某些微命令，而需要与其他字段的编码来联合定义

微程序控制器

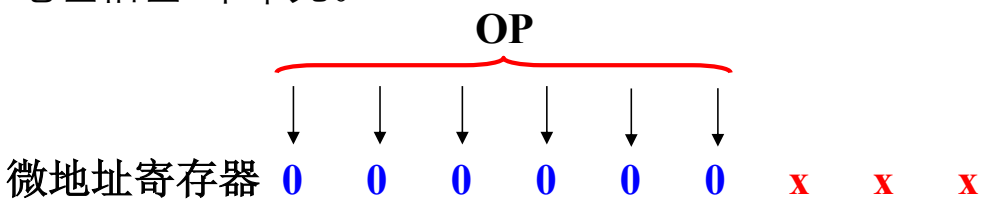
例1：某机采用微程序控制器设计，已知每条机器指令的执行过程均可分解成8条微指令组成的微程序，该机指令系统采用6位定长操作码格式，请问控制存储器至少应能容纳多少条微指令？如何确定机器指令操作码与该指令微程序的入口地址的对应关系，请给出具体方案。

解:

每条机器指令可以分解成8条微指令，其中包含一条公共取指微指令。机器指令系统采用6位定长操作码，总共允许有2^6种不同的机器指令，控制存储器可容纳的微指令条数至少为64×7+1=449。

如果不考虑公共取指微指令，则控制存储器的容量为64×8=512个单元。

微地址寄存器为9位，其中高6位为机器指令的操作码，它与任意的低3位拼接即可形成微程序的入口地址，相邻两条机器指令的微程序入口地址相差8个单元。

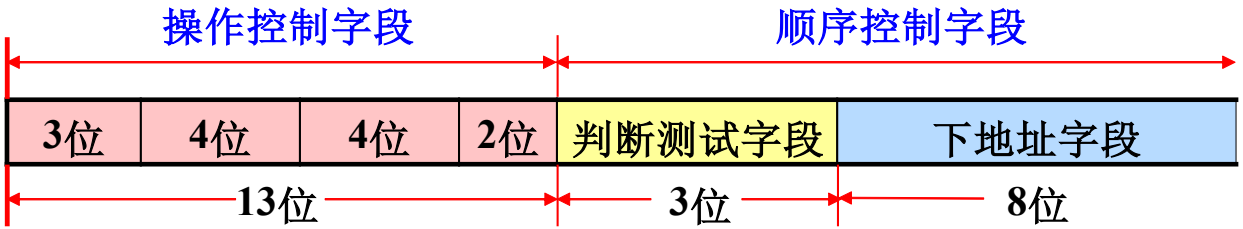


微程序控制器

例2：某机采用微程序控制方式，微指令字长24位，采用水平型编码控制的微指令格式。共有微命令30个，构成4个互斥类，各包含5个、8个、14个和3个微命令，外部条件共3个且直接控制。问：1）控制存储器的容量应为多少？2）设计微指令的具体格式。

解：

水平型 编码表示法：不使用该类任何一个命令占一个编码，支持6、9、15、4种编码需要3、4、4、2位，共13位；外部条件3个，直接控制 对应3位测试字段；下地址字段还剩24-13-3=8位。控制存储器容量=2^{8}*24b



单选题 1分

设置

某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用编码表示法，共有33个微命令，构成5个互斥类，分别包含8、3、12、5和6个微命令，则操作控制字段至少有几位？

A 6

B 15

C 16

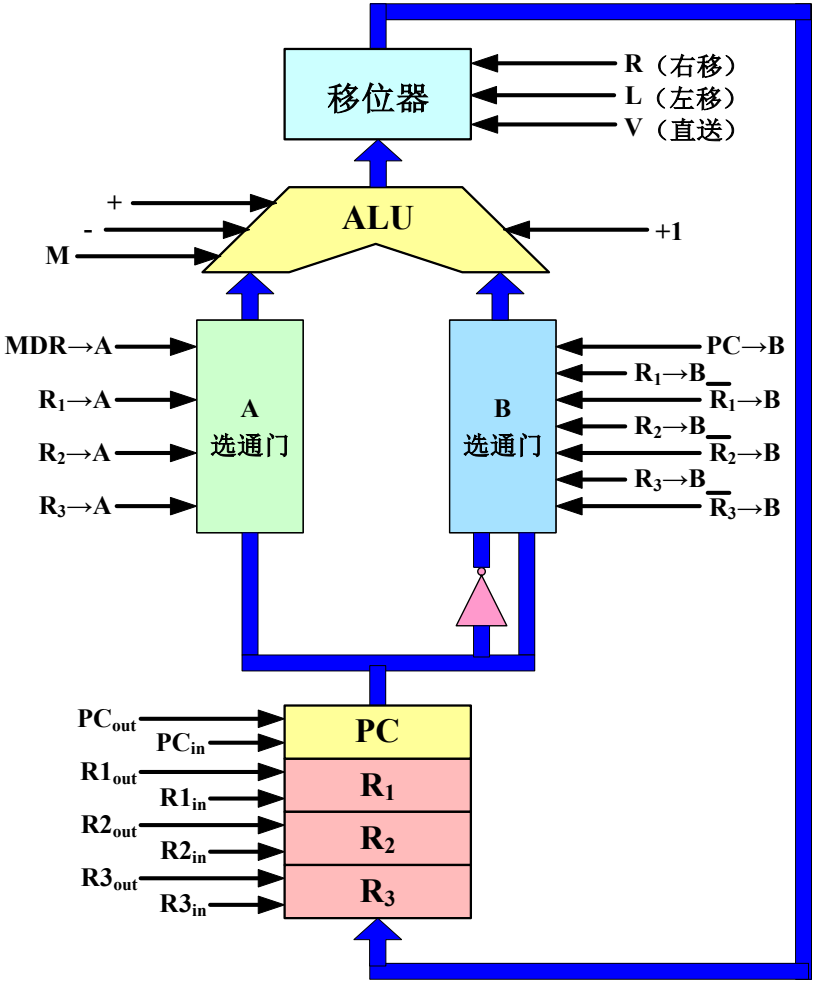
D 33

提交

微程序控制器

例3：设有一运算通路如图所示，假设操作数a和b（均为补码）已分别放在通用寄存器R1和R2中，ALU有+、-、M（传送）3种操作功能。

1. 指出相斥性微命令和相容性微命令。
2. 采用字段编码表示方式，设计适合此运算器的微指令格式。
3. 画出计算(a-b)/2→R2的微程序流程图，则该图中执行周期需要几条微指令？
4. 按设计的微指令格式，写出3的微指令控制信号。



微程序控制器

例3：设有一运算通路如图所示，假设操作数a和b（均为补码）已分别放在通用寄存器R1和R2中，ALU有+、−、M（传送）3种操作功能。

1. 指出相斥性微命令和相容性微命令。

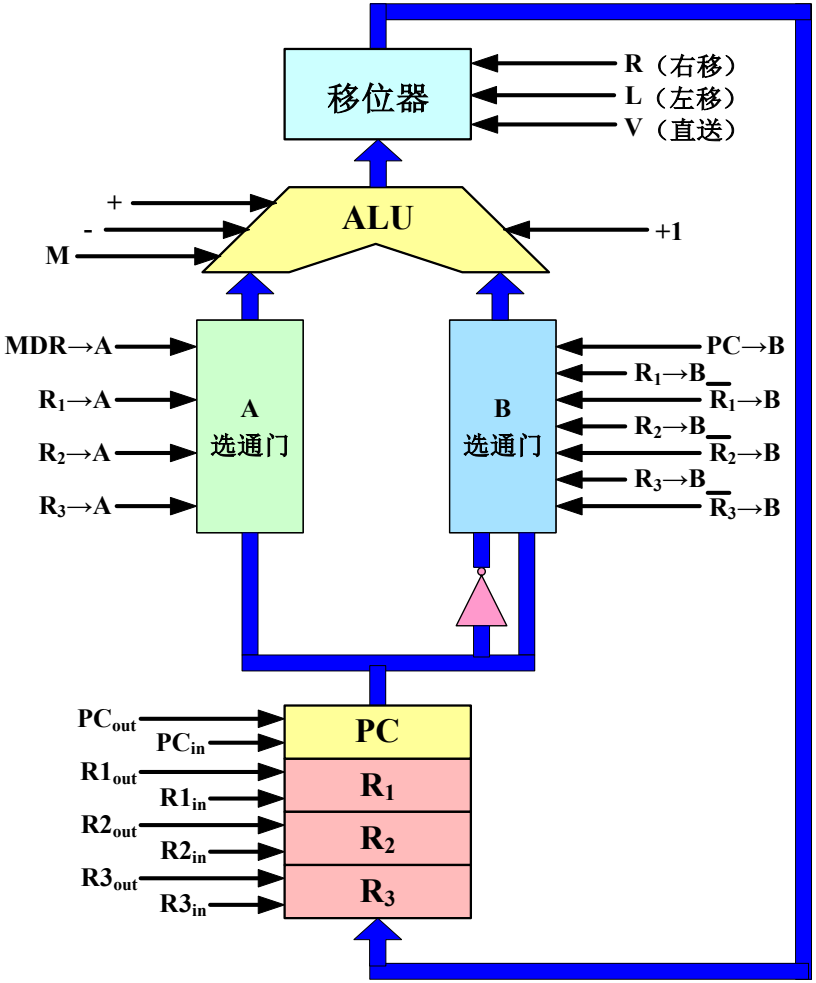
解：

相斥性微命令5组：

- 移位器（R、L、V）；
- ALU（+、-、M）；
- A选通门的4个控制信号；
- B选通门的7个控制信号；
- 寄存器的输入与输出控制信号，即输入时不能输出，输出时不能输入。

相容性微命令：

- ALU的任一信号与+1控制信号；
- 寄存器的4个输入控制信号；
- 五组相斥性微命令组与组之间是相容的。

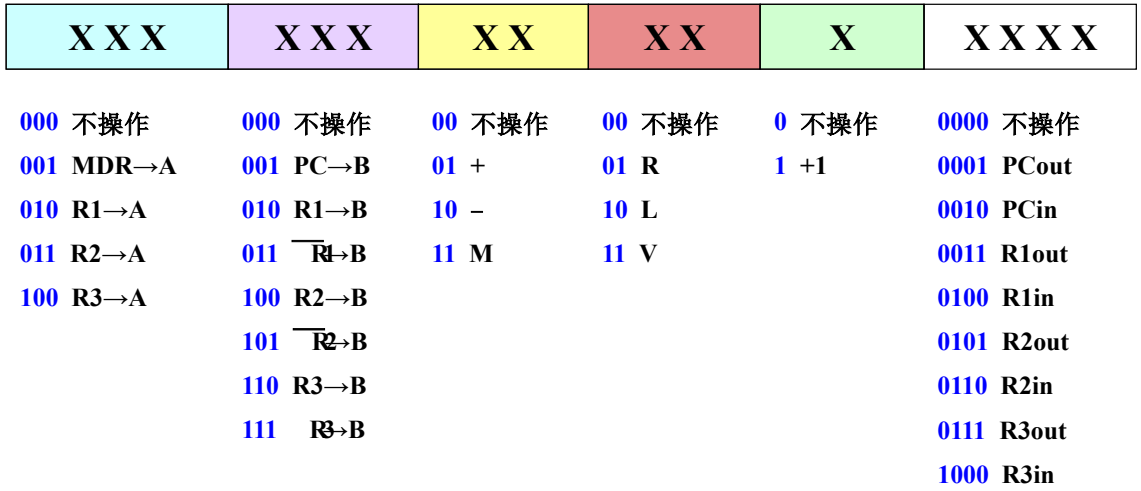


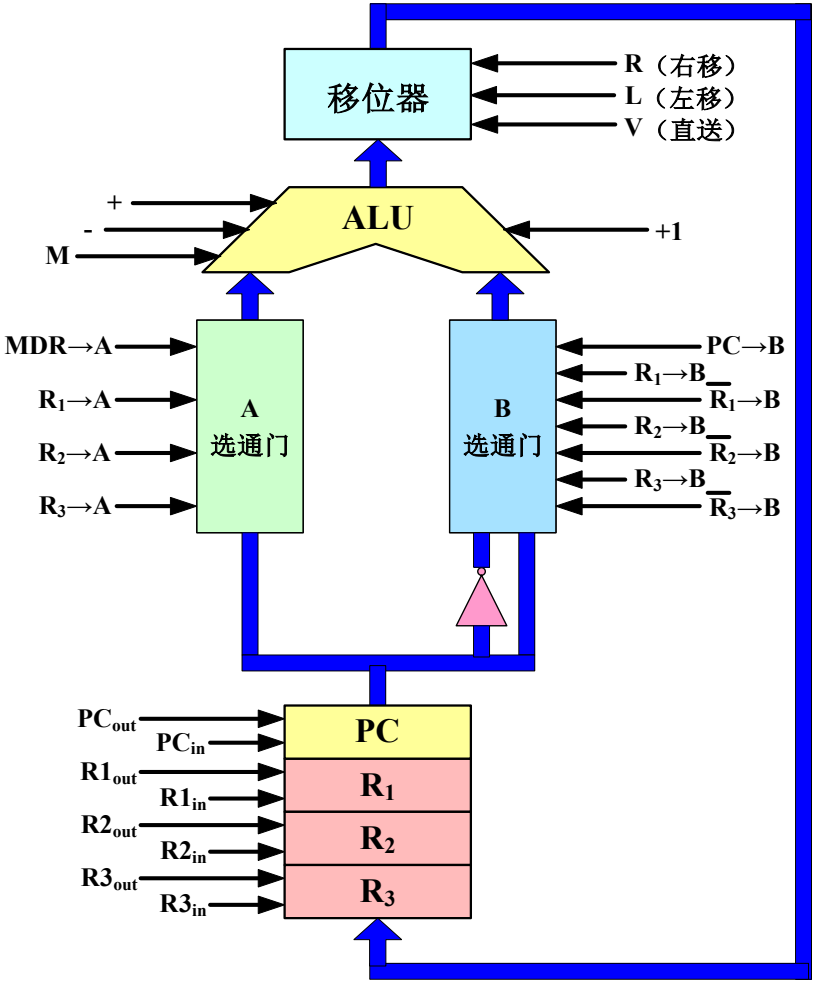
微程序控制器

例3：设有一运算通路如图所示，假设操作数a和b（均为补码）已分别放在通用寄存器R1和R2中，ALU有+、−、M（传送）3种操作功能。

2. 采用字段编码表示方式，设计适合此运算器的微指令格式。

解:





微程序控制器

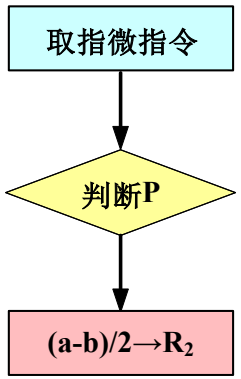
例3：设有一运算通路如图所示，假设操作数a和b（均为补码）已分别放在通用寄存器R1和R2中，ALU有+、−、M（传送）3种操作功能。

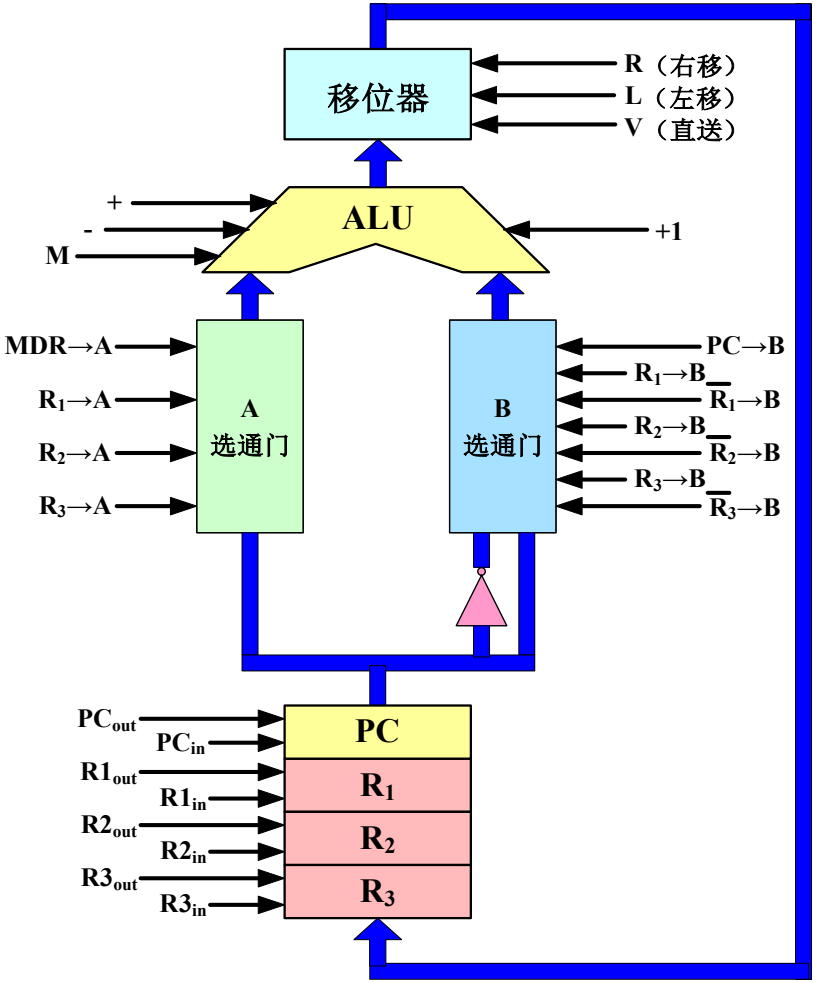
3. 画出计算(a-b)/2→R2的微程序流程图，则该图中执行周期需要几条微指令？

解:

[A-B]补=[A]补+[-B]补=[A]补+$\overline{[B]补}$+1

操作数a和b（补码）分别放在R1和R2中，执行周期只需一条微指令即可。



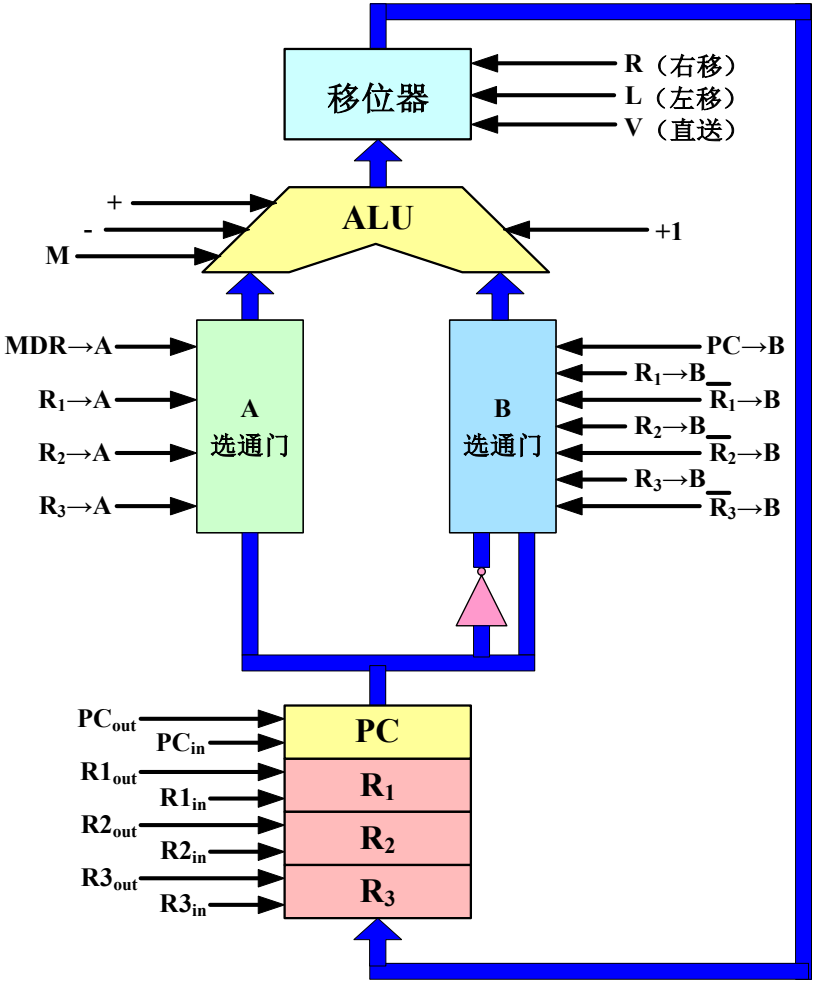


微程序控制器

例3：设有一运算通路如图所示，假设操作数a和b（均为补码）已分别放在通用寄存器R1和R2中，ALU有+、−、M（传送）3种操作功能。

4. 按设计的微指令格式，写出3的微指令控制信号。

解:
微指令操作控制字段为:
010|101|01|01|1|0110
对应的控制信号为:
R1 → A
$\overline{R2}$ → B
+
R
+1
R2in



硬布线控制器 VS 微程序控制器

	硬布线控制器	微程序控制器
工作原理	微操作控制信号由组合逻辑电路即时产生	微操作控制信号以微程序的形式存放在控制存储器中，执行指令时读出
执行速度	快	慢
规整性	不规整，繁琐	规整
应用场合	RISC	CISC
扩充性	不易扩充	易扩充